Chapter 5: System Implementation, Testing and Evaluation

5.1 Introduction

This chapter reports how the design of Chapter 4 was implemented as the CAMR memory engine and its evaluation harness, how the implementation was tested, and what the experiments measured. The project follows the AI and machine learning route of the departmental guidelines. CAMR's language models run with frozen weights: what changes between conditions is the external memory and the engine's learned policy for using it. The machine-learning components that *are* trained, the router and its escalation cascade, were tuned by cross-validation on training data only and tested once on held-out questions. To test the proposal's argument against fine-tuning rather than assume it, a LoRA fine-tuned model was also trained on the same documents as a comparison baseline. The chapter reports dataset provenance, the engine stated as equations, the experimental configuration, the interface and its tests, and the measured results for each research objective, with worked examples of what the models actually answered.

Every number in this chapter was produced by the code in the project repository on real public benchmark data and real models, and each table names the results file it comes from. Where a result is weaker than expected, or a requirement was not met, it is reported as measured. Section 5.2 describes the environment, section 5.3 the datasets, section 5.4 the implementation, section 5.5 testing, section 5.6 the version-control evidence, section 5.7 the declaration of AI-assisted tools, and section 5.8 the results, discussion and limitations.

The work proceeded in increments, each ending with a measured result that decided the next step. The proposal's engine and harness were built and unit-tested first. The engine was then made capability-adaptive (gating, entity bridging and procedural memory), and a residual-gap condition was added. Next came a real-model pilot on a CPU-only machine, a memory-growth experiment, a model-size sweep spread across five machines, and a comparison with a frontier cloud model (Kimi K3). The engine's per-question policy was then learned from rewards, and the multi-hop result was confirmed on 200 held-out questions. Finally, the engine was packaged as a downloadable desktop application. Figure 5.1 shows this sequence as it appears in the repository's commit history, and Figure 5.2 shows the experimental pipeline that every result in this chapter passed through.

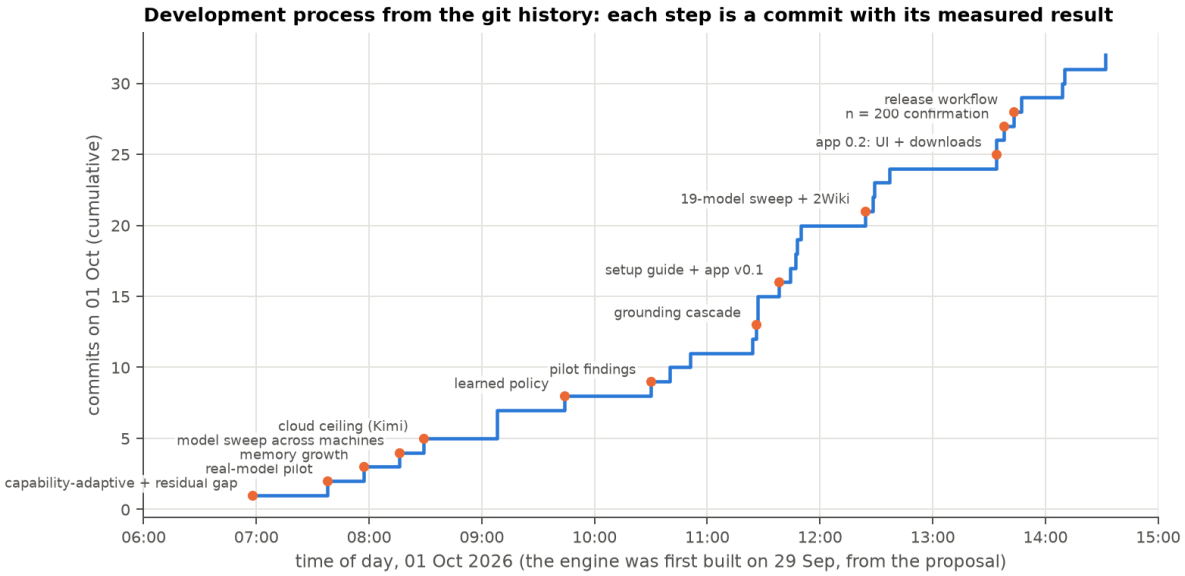


Figure 5.1: Development process reconstructed from the git history; each marked step is a commit that added an implemented or measured increment

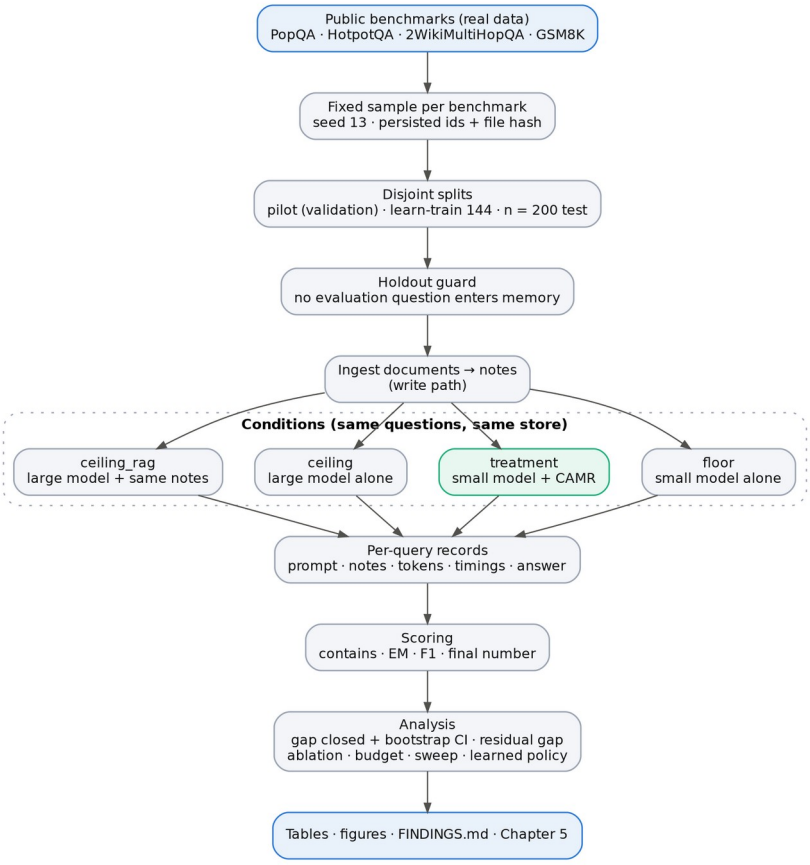


Figure 5.2: Experimental pipeline: from public benchmark data to the tables and figures of this chapter

5.2 Implementation Environment

All development and every experiment ran on Linux machines with x86-64 CPUs and no GPU. This is the consumer-laptop class of device that NFR-01 targets: four CPU cores and 16 GB of memory. The system is a local research prototype. It was not deployed for public access.

5.2.1 Hardware and Deployment Environment

Table 5.1: Implementation environment and minimum requirements

Item or layer	**Tested specification or version**	**Purpose and justification**	**Evidence**
Primary machine (A)	Intel(R) Xeon(R) Processor @ 2.10GHz, 4 logical CPUs, 15.7 GB RAM (14.3 GB usable by processes), no GPU, Linux x86-64	Development, all pilot, growth, ablation, learning and Kimi runs; the stated minimum (NFR-01)	`results/pilot/tables/profile.json`
Sweep machines (B–E)	Same container class: 4 vCPU Intel Xeon (2.10 GHz; machine C 2.80 GHz), 16 GB RAM	Model-size sweep run in parallel, one model family per machine, then merged	`camr merge`; FINDINGS F10
Minimum to run the engine	4 cores, 16 GB RAM. Measured peak: engine process 712 MB, model runtime 1987 MB (qwen2.5:0.5b), store 47.8 MB	A 0.5B model and the engine fit in under 3 GB, leaving headroom on a 16 GB device	`camr profile` (TC-14)
Large local model	gpt-oss:20b (21B total, 3.6B active, MoE) with memory-mapped weights	Local large-model ceiling; loads in 16 GB only with mmap (first attempt was killed at 13.3 GB)	FINDINGS F6
Cloud ceiling	Moonshot Kimi K3 and Kimi K2.6 via an OpenAI-compatible HTTPS API	Frontier reference model (ceiling condition only; NFR-06)	`configs/pilot_kimi.yaml`
End-user device (CAMR Personal)	Windows 10/11 (x64, ARM64), macOS (Apple Silicon, Intel), Linux (x64, ARM64); Ollama installed; ≥ 4 GB RAM for a 1.5B model	The application tested by real users; talks only to Ollama on the same machine	CI build and launch test on all six (Table 5.22)
Specialised hardware	None: no GPU, NPU or accelerator was used anywhere	The study targets CPU-only consumer devices; every timing is a CPU timing	Profiles and CI runner labels

5.2.2 Software, Services and Configuration

Table 5.2: Software and services used

Component	**Version**	**Role in the system**
Python	3.11.15	Implementation language of the engine, harness and CLI
Ollama	0.35.0	Local model runtime on the loopback interface (IR-01);

Component	Version	Role in the system
		Q4/Q8 GGUF weights
SQLite / sqlite-vec	3.45.1 / 0.1.9	Single-file store with exact kNN vector search (FR-05, IR-04, NFR-03)
sentence-transformers / PyTorch	6.1.0 / 2.14.1 (CPU)	BGE-small-en-v1.5 embedder, 384 dimensions (IR-03)
NumPy	2.4.6	Scoring, bootstrap confidence intervals, ridge regression and LinUCB
Streamlit	1.64.0	Read-only inspection interface (FR-17, IR-06)
Matplotlib / Playwright	3.11.2 / 1.63.0	Figures; automated screenshots of the inspector
pytest	9.1.1	Unit and integration tests (section 5.5)
Moonshot API	kimi-k3, kimi-k2.6	Cloud ceiling. The key is read from the `MOONSHOT_API_KEY` environment variable and never written to configuration, logs or the repository

Configuration is a single YAML file per experiment (NFR-07). It is loaded into typed dataclasses that reject unknown keys, so a misspelt ablation parameter fails loudly instead of silently running the default. Each run records the hash of its full configuration. A completed run with the same label and hash is skipped, which makes every command resumable after an interruption. Containers (Docker) were not part of the solution: the experiments ran directly on the hosts above, and the application ships as a single self-contained executable per platform.

5.2.3 Deployment and Reproducibility

The engine installs as a Python package and is driven by one command-line program, `camr` (IR-05). Every result in this chapter can be regenerated with the commands below. They were run as shown, on Linux with Python 3.11 and Ollama 0.35. Each block can be copied into a terminal. Steps 4 and 6 need the cloud key only for the Kimi comparisons; everything else runs offline once models and data are downloaded.

Step 1: get the code and install it (a virtual environment keeps it separate from other Python projects).

```
git clone https://github.com/Satasdin/CAMR.git
cd CAMR
python -m venv .venv && source .venv/bin/activate      # Windows: .venv\
Scripts\activate
pip install -e ".[vec,embed,dev,inspect]"
python -m pytest -q                                     # health check: all
tests pass, no network needed
```

Step 2: install the local models with Ollama (https://ollama.com/download).

```
ollama pull qwen2.5:0.5b        # the small model of the pilot
```

```
ollama pull gpt-oss:20b         # large local comparison model (needs ~14 GB
RAM)
ollama pull qwen2.5:1.5b && ollama pull llama3.2:3b     # used in the n =
200 study
```

Step 3: download the real benchmark data (PopQA, HotpotQA, 2WikiMultiHopQA, GSM8K) and the Wikipedia corpus for PopQA. The pilot corpus is already in the repository; the learning split needs its own.

```
python scripts/fetch_data.py                            # prints a hash per
file
python scripts/build_popqa_corpus.py --config configs/learn.yaml --split
train --run-dir results/learn \
       --out data/popqa/corpus_learn.jsonl
```

Step 4: the cloud key (only for the Kimi conditions). It is read from an environment variable and never written to any file.

```
export MOONSHOT_API_KEY=...                             # your own key
```

Step 5: run the experiments. Each command is resumable: run it again after an interruption and it skips finished work.

```
camr reproduce       --config configs/pilot.yaml        # pilot: ingest,
conditions, ablation, budgets, profile
camr grow            --config configs/pilot.yaml --stages 0.25,0.5,1.0
# memory growth
camr sweep-models    --config configs/pilot.yaml --models
qwen2.5:0.5b,llama3.2:3b,phi4-mini:3.8b   # any list
camr retrieval-eval  --config configs/retrieval_eval.yaml
# 500 + 500 questions, no LLM
camr learn           --config configs/learn.yaml --phase all
# learned policy
camr ingest          --config configs/hotpot200.yaml
# n = 200 study
camr sweep-models    --config configs/hotpot200.yaml --models
qwen2.5:0.5b,qwen2.5:1.5b,llama3.2:3b
```

Step 6: the cloud comparisons.

```
camr run --config configs/pilot_kimi.yaml --out results/pilot --condition
ceiling     --benchmark popqa --label kimi
camr run --config configs/pilot_kimi.yaml --out results/pilot --condition
ceiling_rag --benchmark popqa --label kimi
camr run --config configs/hotpot200.yaml  --condition ceiling     --
benchmark hotpotqa --label kimi
camr run --config configs/hotpot200.yaml  --condition ceiling_rag --
benchmark hotpotqa --label kimi
```

Step 7: regenerate the tables, figures and this chapter from the logged results alone (NFR-04).

```
camr analyse --run-dir results/pilot
python scripts/make_figures.py && python scripts/make_report_figures.py &&
python scripts/make_diagrams.py
camr inspect --run-dir results/pilot                    # read-only web view
at http://localhost:8501
python scripts/build_chapter5.py --template proposal.docx --out
build/Chapter5
```

Step 8: tuning, the fine-tuning baseline and the interface audit (sections 5.8.2, 5.8.12, 5.5.4 and 5.5.5). The fine-tuning baseline downloads the 0.5B model's weights from Hugging Face (about 1 GB) and takes several hours on four CPU cores.

```
pip install -e ".[finetune,report]"
python scripts/tune_policy.py                            # router: 5-fold CV
on training questions, one held-out test
python scripts/tune_gating.py --store-copy /tmp/re      # read path:
validation half, then held-out half
python scripts/finetune_baseline.py tune && python
scripts/finetune_baseline.py test \
       && python scripts/finetune_baseline.py camr-cost # LoRA vs CAMR on
the same documents
python -m pytest -q tests/test_app_e2e.py               # browser tests
(Chromium)
python scripts/ui_audit.py --axe axe.min.js             # axe-core 4.10 +
WCAG contrast
python scripts/scenario_screens.py --model llama3.2:3b  # live scenarios,
with and without memory
python scripts/extract_examples.py && python scripts/make_method_figures.py
```

The application only (no experiments): download the file for the platform from the v0.2.1 release page, or install it with Python:

```
pip install "camr[app] @ git+https://github.com/Satasdin/CAMR.git"
ollama pull qwen2.5:1.5b
camr app                                                # opens
http://127.0.0.1:8502
```

Figure 5.3 shows the CLI's command surface and Figure 5.4 the test-suite health check, both captured from real runs.

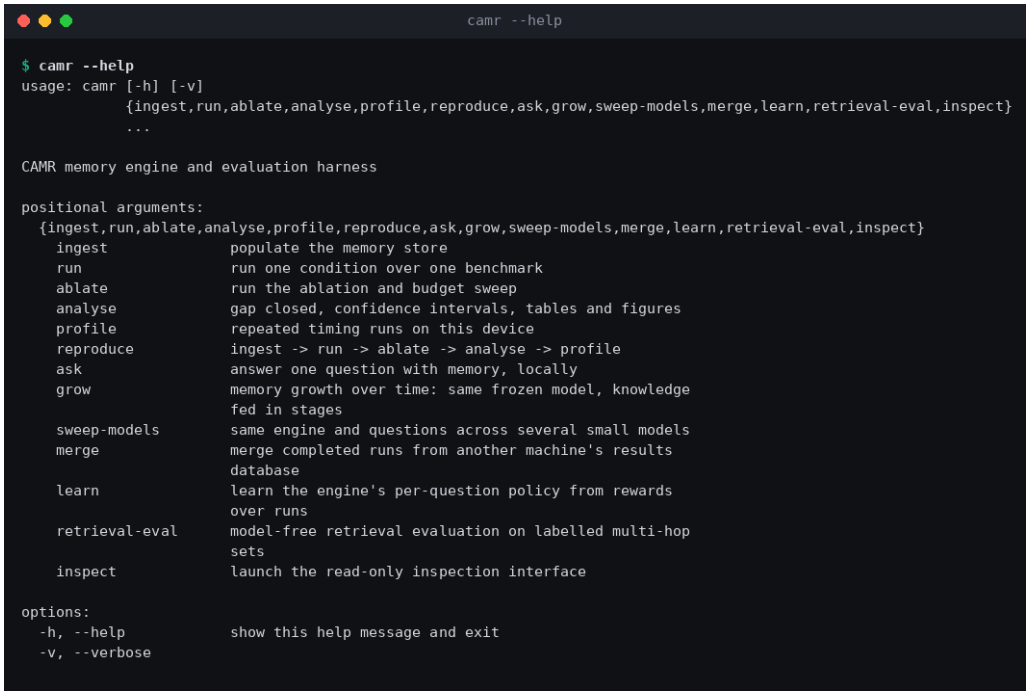


Figure 5.3: The camr command-line interface (captured output)



Figure 5.4: Health check: the offline test suite passing (captured output)

5.3 Dataset Description

Four public benchmarks were used (DR-01). They were downloaded from their official distributions by `scripts/fetch_data.py`, and no data was generated synthetically. Table 5.3 summarises them and Table 5.4 shows one real record of each. PopQA ships without documents, so a corpus of real Wikipedia summaries was fetched for the subject entity of each sampled question (design note A4). HotpotQA and 2WikiMultiHopQA supply gold and distractor paragraphs, and those paragraphs, never the question–answer records, populate memory (DR-03).

Table 5.3: Benchmarks used

Benchmark	Source and licence	Size of the file used	Task type	Metric
PopQA	Mallen et al. (2023); templates over Wikidata facts	14,267 questions; corpus of 357 Wikipedia summaries (CC BY-SA)	Single-hop, long-tail facts	Answer contained (accuracy)
HotpotQA (dev	Yang et al. (2018); crowd-	7,405 questions (5,918	Multi-hop	Exact

Benchmark	Source and licence	Size of the file used	Task type	Metric
distractor)	written over Wikipedia; CC BY-SA	bridge, 1,487 comparison), 10 paragraphs each		match, F1
2WikiMultiHopQA (dev)	Ho et al. (2020); Wikipedia and Wikidata	12,576 questions	Multi-hop (retrieval study only)	Supporting-paragraph recall
GSM8K (test; train for exemplars)	Cobbe et al. (2021); human-written; MIT	1,319 test; 7,473 train solutions	Reasoning control	Final-answer accuracy

Table 5.4: One record from each benchmark (verbatim)

Benchmark	Question	Reference answer	Supporting evidence
PopQA	What is George Rankin's occupation?	politician (also: political leader, political figure)	Wikipedia summary of *George Rankin*
HotpotQA	Were Scott Derrickson and Ed Wood of the same nationality?	yes	Paragraphs *Scott Derrickson* and *Ed Wood* (gold) plus 8 distractors
GSM8K	Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder … for $2 per fresh duck egg. How much in dollars does she make every day?	18	None (reasoning); solved train problems serve as exemplars

5.3.1 Data Preparation and Partitioning

Each benchmark sample was drawn once from seed 13, sorted by question identifier before sampling so that the draw does not depend on file order, and persisted to `samples/<benchmark>.json` with the source file's hash (DR-02). Every condition and ablation reused the same questions, so all comparisons are paired. The pilot evaluation sample was 30 PopQA, 30 HotpotQA and 12 GSM8K questions (72 in total), limited by the time the 20B CPU ceiling needed (up to 150 s per GSM8K answer). For the learned policy, a **disjoint** training split of 144 questions (60 PopQA, 60 HotpotQA, 24 GSM8K) was drawn from the questions not in the evaluation sample. The code asserts that the two splits do not overlap.

Memory was built from documents only. The HotpotQA store also took the paragraphs of 100 further, unsampled questions as extra distractors (design note A5). Before ingestion a holdout guard scanned every document and refused any that contained an evaluation question verbatim (DR-04, TC-21). Text was normalised (Unicode NFKC, whitespace) and

paragraphs were stored verbatim with their title prefixed, e.g. *The Loudwater Mystery (film): The Loudwater Mystery is a 1921 British silent.…* The structured (distilled) write policy is implemented and tested, but it was not used for the reported runs, because distilling every paragraph with the 0.5B model exceeded the pilot's time budget.

Table 5.5 follows one real record from the raw files to what the model finally reads.

Table 5.5: One PopQA record before and after preparation

Stage	**Content**
Raw question (test.tsv)	id 5939255 · subject *The Loudwater Mystery* · property *director* · question *Who was the director of The Loudwater Mystery?* · answers ["Walter West", "Walter Alabaster West"]
Raw document (corpus)	{"title": "The Loudwater Mystery (film)", "text": "The Loudwater Mystery is a 1921 British silent crime film directed by Walter West and starring Gregory Scott, Pauline Peters and Clive Brook. It was based on the 1920 novel …"}
Stored note (#4863)	*The Loudwater Mystery (film): The Loudwater Mystery is a 1921 British silent crime film directed by Walter West …* (47 tokens; 384-d BGE vector; provenance: dataset popqa, document id, ingestion time)
Read at question time	Similarity 0.77 → admitted; next candidate 0.62 → outside the 0.15 margin. The prompt's notes section contains this single note (47 tokens instead of a 512-token budget).
Answer and score	0.5B alone: *John Sturges* (wrong). 0.5B + CAMR: *Walter West*: contains = 1, EM = 1

5.4 System Implementation

Figure 5.5 shows the implemented architecture. Everything inside the engine runs on the device. The cloud model is used only as a measurement ceiling and as the escalation target of the learned policy.

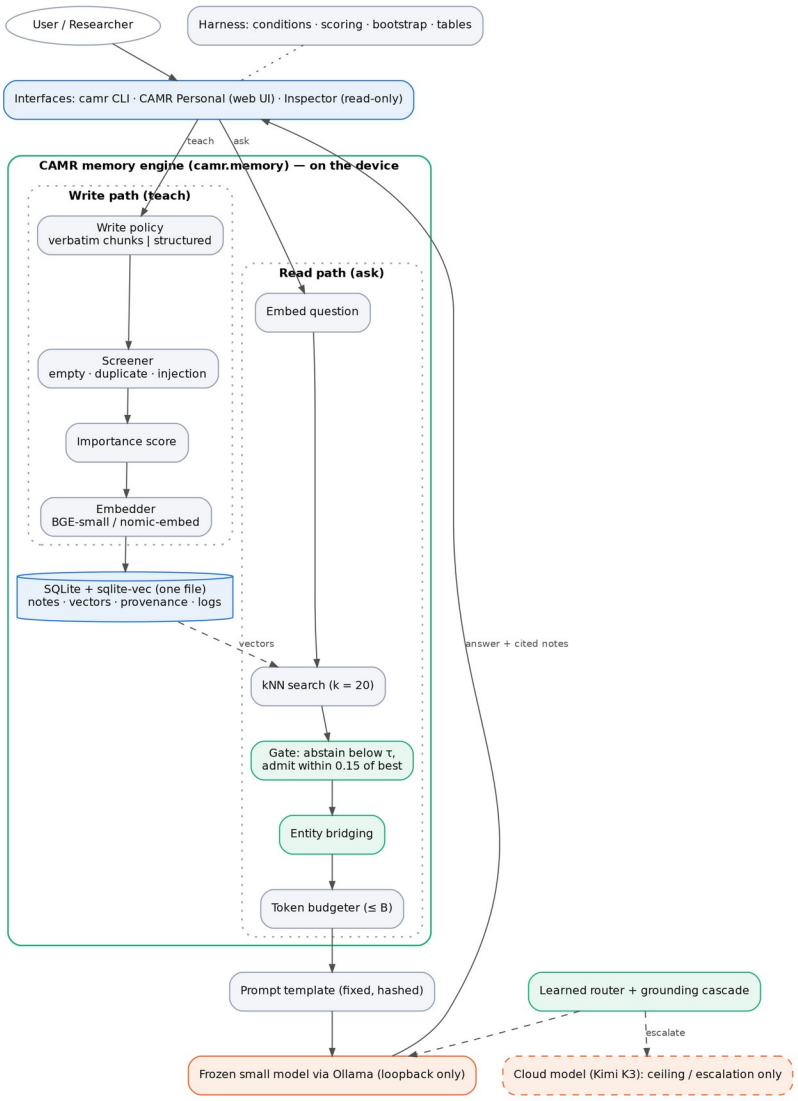


Figure 5.5: Implemented CAMR architecture: write path, single-file store, capability-adaptive read path (green), frozen local model and optional cloud escalation

The implementation follows the package structure of Figure 4.6: `camr.memory` (the engine), `camr.models` (runners and prompt templates), `camr.harness` and `camr.eval` (experiments and analysis), `camr.learn` (the learned policy), `camr.cli` and `camr.inspect`. A test enforces the dependency rules of the package diagram (memory imports nothing from harness, eval, cli or inspect, and nothing imports inspect).

5.4.1 Memory Engine: Write Path

Ingestion implements the sequence of Figure 4.4. A write policy turns a source document into candidate notes (FR-01). The screener rejects empty, degenerate, duplicate, over-length and injection-like candidates and logs the reason (FR-02). An importance score is assigned (FR-04). Then the note, its embedding and its provenance are committed in one SQLite

transaction (FR-03, FR-05), so the store never holds a note without its vector or its source. Duplicate detection uses a checksum with a UNIQUE constraint, namespaced by note population, so ablations that change what is stored (write policy, screening) coexist in one file (design note A6). Ingesting 14,567 HotpotQA paragraphs into a 24,475-note store took 622 s on machine A, of which 569 s was embedding.

5.4.2 Memory Engine: Read Path

Recall implements Figure 4.5 and Equations 4.1–4.5. The question is embedded and the k = 20 nearest notes are found by exact kNN in sqlite-vec (FR-06). The notes are ranked by the configured policy, similarity only or composite (FR-07), and packed greedily under the token budget (FR-08). Three capability-adaptive mechanisms were added during implementation, each switchable in configuration and each justified by measurement in section 5.8:

i. **Gating.** The engine abstains when the best note's similarity is below 0.50, and admits only notes within 0.15 of the best. This keeps irrelevant text away from a small reader.
ii. **Entity-bridge expansion.** When an admitted note mentions the title of another stored entity, that entity's notes are pulled in. This is a one-step associative hop that needs no graph construction and no model call, and it replaces the optional graph layer of FR-18.
iii. **Procedural memory.** For reasoning questions, solved GSM8K *train* problems are retrieved as worked examples instead of factual notes.

Recency uses a logical clock, which advances one step per question instead of wall-clock time, and access state is reset before every run. This makes recall deterministic and independent of machine speed (NFR-05, design note A1).

5.4.3 Formal Model of the Read Path

This subsection states precisely what the engine computes for a question q, so that every setting tuned or reported later has an exact meaning. The notation follows Chapter 4. The store holds notes n, each with its text, a token count $c(n)$, and an embedding produced by the embedder f. Embeddings are normalised to unit length, so cosine similarity is a dot product:

$$s(q,n)=\cos\big(f(q),f(n)\big)=\frac{f(q)\cdot f(n)}{\|f(q)\|\|f(n)\|}=f(q)\cdot f(n) \tag{5.1}$$

where

$f(\cdot)\in\mathbb{R}^{384}$ the embedder (BGE-small-en-v1.5 in the experiments; nomic-embed-text in the application), with the model card's query prefix on q;
$s(q,n)\in[-1,1]$ the similarity of note n to the question.

The candidate set is the k most similar notes, found by exact nearest-neighbour search in sqlite-vec:

$$C_k(q) = \arg\max_{S \subseteq \mathcal{M}, |S|=k} \sum_{n \in S} s(q,n), k = 20 \tag{5.2}$$

The proposal ranked candidates by a weighted composite of similarity, recency and importance (Equation 4.1). As implemented, each term is min-max normalised over the candidate set:

$$\text{score}(n) = w_s \tilde{s}(q,n) + w_r \rho^{\Delta t(n)} + w_i \tilde{I}(n), w_s + w_r + w_i = 1 \tag{5.3}$$

where

$\tilde{s}, \tilde{I}$ min-max normalised similarity and importance;
$\rho = 0.99$ the recency decay per hour of logical time since the note was last read;
(w_s, w_r, w_i) (0.6, 0.15, 0.25) in the proposal's configuration, and (1, 0, 0) in the configuration the evidence selected (section 5.8.1), which reduces Equation 5.3 to ranking by similarity.

Gating is the first capability-adaptive step. Let $s_{(1)}$ be the best similarity in the candidate set. The engine abstains (gives the model no notes) when even the best note is weak; otherwise it keeps only notes within a margin δ of the best:

$$E(q) = \begin{cases} \varnothing & \text{if } s_{(1)} < \tau \\ \{n \in C_k(q) : s(q,n) \geq s_{(1)} - \delta\} & \text{otherwise} \end{cases} \tag{5.4}$$

where

τ the abstain threshold (0.50 for BGE, 0.55 for nomic-embed-text; calibrated in section 5.8.2);
δ the admission margin (0.15).

A second, optional gate was added after the comparison with standard RAG (section 5.8.3) showed that similarity cannot tell a note about *this* entity from a note about a similar one. Let *N(q)* be the names the question mentions (runs of capitalised words and quoted titles, `camr.memory.entities`). If the question names something and no eligible note mentions it, the engine abstains:

$$E'(q) = \begin{cases} \varnothing & \text{if } N(q) \neq \varnothing \text{ and } N(q) \cap \text{names}(n) = \varnothing \ \forall n \in E(q) \\ E(q) & \text{otherwise} \end{cases} \tag{5.5}$$

Questions that name nothing (*when is my dentist appointment?*) are unaffected. The gate is off in the benchmark runs reported before section 5.8.3 and on in the released application.

Entity bridging then adds, for each of the top three eligible seed notes, up to two notes about other stored entities whose titles the seed's text mentions. These notes may have low similarity to the question; they are reached through the seed, which is the second hop of a multi-hop question:

$$B(q)=\{m\in\mathcal{M}:\text{title}(m)\in\text{mentions}(n)\ \text{for some}\ n\in\text{top}_3 E(q)\},\le 2\ \text{per}\ n \qquad (5.6)$$

Finally the budgeter admits notes in rank order, each bridged note directly after its seed, until the next note would exceed the token budget B, and then stops:

$$A(q)=(n_1,\ldots,n_m),m=\max\left\{j:\sum_{i=1}^{j}c(n_i)\le B\right\} \qquad (5.7)$$

The prompt contains exactly the notes in $A(q)$, so the number of context tokens is at most B and is usually far smaller, because gating removes candidates before the budget binds.

Worked example (from the pilot's logs). For *Who was the director of The Loudwater Mystery?* the 20 candidates had best similarity $s_{(1)}$ = 0.771 (the film's own Wikipedia note). Since 0.771 ≥ τ = 0.50, the engine did not abstain. The admission floor was $s_{(1)} - \delta$ = 0.771 − 0.15 = 0.621, and the next-best note (*Klute*, 0.616) fell just below it, so only one note was eligible. That note was 47 tokens, so Equation 5.7 admitted it and the model read 47 tokens instead of filling the 512-token budget. The 0.5B model answered *Walter West* (correct); alone it had answered *John Sturges*. Figure 5.6 draws this decision from the logged scores.

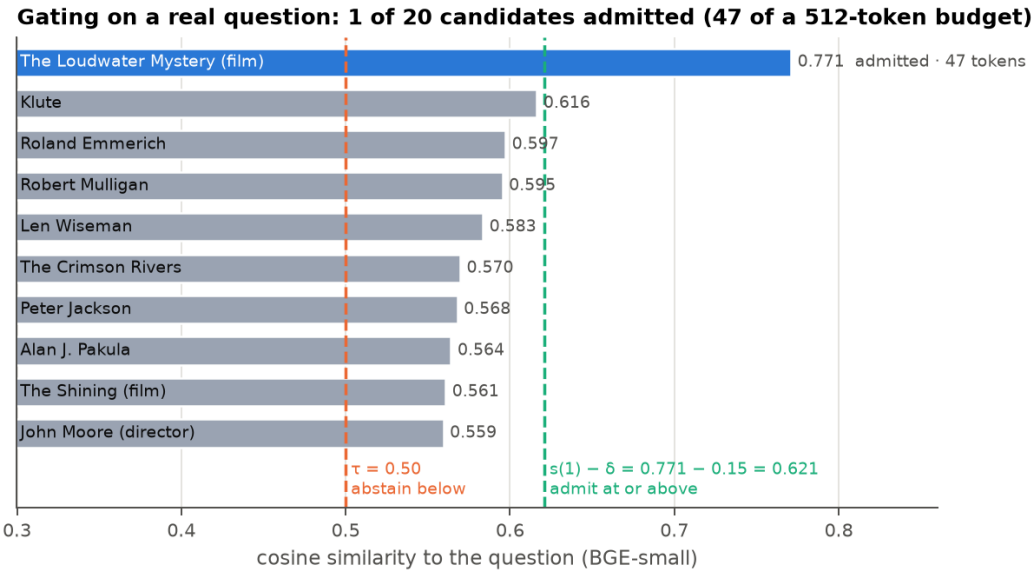


Figure 5.6: Gating on one real question: candidate similarities, the abstain threshold τ and the admission floor s(1) − δ; only the blue note was given to the model

5.4.4 CAMR and Retrieval-Augmented Generation

CAMR is built on retrieval, so the obvious question is how it differs from retrieval-augmented generation (RAG). **RAG** (Lewis et al., 2020) gives a language model text retrieved from an external corpus at question time. In its original form the answer is generated by marginalising over the *k* passages *z* most relevant to the input *x* under a retriever η:

$$p_{\mathrm{RAG}}(y \mid x) = \sum_{z \in \text{top-}k\left(p_\eta(\cdot \mid x)\right)} p_\eta(z \mid x)\, p_\theta(y \mid x, z) \tag{5.8}$$

Most deployed systems, and the baseline used here, implement the simpler *retrieve-then-read* form: embed the question, take the *k* most similar chunks from a fixed corpus, place all of them in the prompt, and generate. In the notation of Equation 5.2 its context is always

$$A_{\mathrm{RAG}}(q) = C_k(q), \left|A_{\mathrm{RAG}}(q)\right| = k \text{ for every } q \tag{5.9}$$

CAMR uses the same retrieval step and differs in everything around it. Its context is *A(q)* of Equations 5.4–5.7, which can be empty, can contain notes that similarity alone would never rank (bridging), and is capped by a token budget that a policy chooses per question (Equation 5.16). Figure 5.7 sets the two pipelines side by side, and Table 5.6 lists every difference with the evidence that measures it. The honest summary is that CAMR is *not* an alternative to retrieval: its retrieval core is standard dense retrieval. What is new is that every decision around that core is made for the capabilities of a small model: whether to read anything at all, which notes, how many tokens, where the memory comes from, and when to hand the question to a stronger model.

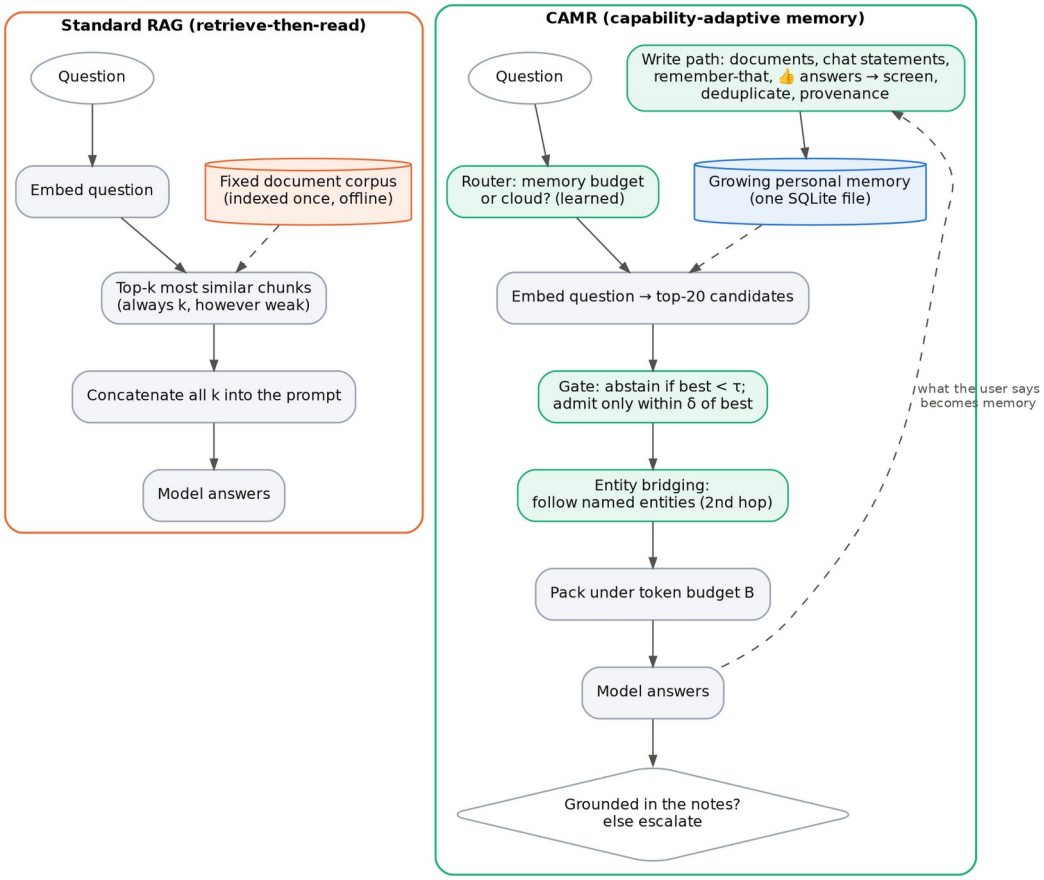


Figure 5.7: Standard retrieve-then-read RAG (left) and CAMR (right); green steps are what CAMR adds

Table 5.6: CAMR compared with standard RAG, and where each difference is measured

Aspect	**Standard RAG**	**CAMR**	**Evidence**
When nothing relevant is stored	Still inserts the k most similar chunks	Abstains when the best note is weak (Equation 5.4) or is about a different entity than the question names (Equation 5.5): the model answers alone, and the app says so	Section 5.8.3 (answer not in memory); calibration (5.8.2)
How many notes	Always k	Only notes within δ of the best, under a token budget	Section 5.8.3 (tokens, precision)
Second-hop evidence	Only if similar to the question	Entity bridging follows names in admitted notes (Equation 5.6)	2Wiki full-support recall (5.8.1, 5.8.3)
Budget	Fixed k for every question and model	Chosen per question by a learned router; small for tiny models, larger for strong readers	5.8.8, 5.8.12
Source of knowledge	A fixed corpus, indexed offline	A memory that grows from use: chat statements, *remember that…*, 👍 answers, files, with screening, deduplication and provenance	Live scenarios (5.5.5): chat memory off vs on
When memory cannot help	No mechanism	Router sends reasoning to a stronger model; cascade escalates ungrounded answers	5.8.12
What is reported	Answer accuracy	Gap closed, residual gap, tokens and latency per answer	5.8.4, 5.8.17

5.4.5 Model Runners

All runners expose a single `generate` operation, and none can modify model weights (FR-09). `OllamaRunner` calls the local runtime with greedy decoding (temperature 0, seed 13) and refuses any host but loopback, unless it is explicitly configured as a ceiling (IR-01, NFR-06). `OpenAICompatRunner` calls the Moonshot API with a pinned model identifier, a timeout and bounded exponential-backoff retries. When retries are exhausted the query is logged as failed rather than scored (IR-02, IR-07). The Kimi API accepts only temperature 1, so the cloud ceiling is a single stochastic sample per question; this limitation is reported in section 5.8.

5.4.6 Evaluation Harness and Learned Policy

The harness runs four conditions over the fixed sample: **floor** (small model alone), **treatment** (small model + CAMR), **ceiling** (large model alone) and **ceiling_rag** (large model given the same notes). Each query writes a structured record (FR-13): condition, model and version, configuration hash, retrieved note identifiers and scores, prompt and generated tokens, retrieval, prefill and decode times, and the score. Analysis computes gap closed (Equation 3.1) with paired bootstrap confidence intervals over 2,000 resamples (FR-15). Next to it, it reports the absolute gain, and the **residual gap** (ceiling_rag − treatment): what model size still buys when both models read the same evidence.

The scores are defined as follows. Let p be the model's answer, G the set of accepted gold answers, and norm(·) the SQuAD normalisation (lower case, punctuation and articles removed, whitespace collapsed):

$$\mathrm{EM}(p,G)=\max_{g\in G}1\left[\mathrm{norm}(p)=\mathrm{norm}(g)\right],\mathrm{Contains}(p,G)=\max_{g\in G}1\left[\mathrm{norm}(g)\sqsubseteq\mathrm{norm}(p)\right] \tag{5.10}$$

$$\mathrm{F}_1(p,g)=\frac{2PR}{P+R},P=\frac{\left|t_p\cap t_g\right|}{\left|t_p\right|},R=\frac{\left|t_p\cap t_g\right|}{\left|t_g\right|} \tag{5.11}$$

where

$\sqsubseteq$ occurrence as a whole-word substring;

t_p, t_g the multisets of normalised tokens of the answer and of the gold answer.

For GSM8K the final number in the answer is compared exactly with the gold final number (fractions and thousands separators parsed). For a condition with per-question scores y_i the

accuracy is their mean, and the headline measure of the proposal (Equation 3.1) is the fraction of the gap closed, with the residual gap beside it:

$$G = \frac{\mathrm{Acc}_{\mathrm{CAMR}} - \mathrm{Acc}_{\mathrm{floor}}}{\mathrm{Acc}_{\mathrm{ceiling}} - \mathrm{Acc}_{\mathrm{floor}}}, R = \mathrm{Acc}_{\mathrm{ceiling+notes}} - \mathrm{Acc}_{\mathrm{CAMR}} \tag{5.12}$$

$G = 1$ means the small model with memory matches the large model; $G > 1$ means it exceeds a closed-book ceiling, which is why the frontier model is used as the anchor. R is what model size still buys when both models read the same notes. Uncertainty comes from a paired bootstrap over questions: for $b = 1, \ldots, 2{,}000$, draw Q questions with replacement and compute the mean paired difference; the 95% interval is the 2.5th and 97.5th percentile of these means:

$$\hat{\Delta}_b^* = \frac{1}{Q}\sum_{i=1}^{Q}\left(y_{j_{b,i}}^{\mathrm{CAMR}} - y_{j_{b,i}}^{\mathrm{floor}}\right), \mathrm{CI}_{95} = \left[\hat{\Delta}_{(0.025)}^*, \hat{\Delta}_{(0.975)}^*\right] \tag{5.13}$$

At the retrieval level, without any language model, the measure is full-support recall: the share of questions for which every gold supporting paragraph S_q reached the context:

$$\mathrm{SR} = \frac{1}{|Q|}\sum_{q \in Q} 1\left[S_q \subseteq \mathrm{titles}\left(A(q)\right)\right] \tag{5.14}$$

The learned policy (`camr learn`) chooses, per question, one of five actions: answer with the small model alone, with 128, 512 or 1,024 tokens of memory, or escalate to Kimi K3. Its training configuration is given in Table 5.7. No neural network is trained inside CAMR and no model weights change. The learned parameters are a ridge-regression weight vector per action (5 × 11 numbers in the first study; 5 × 4 after cross-validation chose the task-only features) and the cascade's grounding model.

Formally, the router faces a contextual bandit. For question q with features $\varphi(q)$ it chooses an action a from the set $\mathcal{A}$ = {local, mem128, mem512, mem1024, cloud} and receives the reward

$$r(q,a) = y(q,a) - c_{\mathrm{cloud}}\,1[a = \mathrm{cloud}] - \mu\frac{T(q,a)}{1000} \tag{5.15}$$

where

$y(q,a) \in \{0,1\}$ whether action a answered q correctly;
$c_{\mathrm{cloud}} = 0.3$ the price of one cloud call in units of one correct answer (a design preference, swept from 0 to 1.2 for the Pareto table);
$\mu = 0.1$ the price of the small model reading 1,000 prompt tokens, *T(q, a)*.

The offline learner fits one ridge regression per action on the training questions (design matrix X of feature rows, reward vector $\mathbf{r}_a$) and routes to the action with the highest predicted reward:

$$\hat{\theta}_a = \left(X^\top X + \lambda I\right)^{-1} X^\top \boldsymbol{r}_a, \pi(q) = \arg\max_{a \in \mathcal{A}} \hat{\theta}_a^\top \varphi(q) \tag{5.16}$$

The online learner, LinUCB (Li et al., 2010), sees only the reward of the action it took. It keeps $A_a = I + \Sigma x\, x^\mathrm{T}$ and $\mathbf{b}_a = \Sigma r\, x$ for each action and chooses optimistically:

$$a_t = \arg\max_{a \in \mathcal{A}} \hat{\theta}_a^\top x_t + \alpha \sqrt{x_t^\top A_a^{-1} x_t}, \hat{\theta}_a = A_a^{-1} \boldsymbol{b}_a \tag{5.17}$$

The cascade answers locally first and escalates when a ridge model of the answer's own signals ψ (are the answer's words in the notes it read? is it abstention-like? how long is it?) predicts that the answer is wrong:

$$\text{escalate}(q) = 1\left[\hat{w}^\top \psi\left(p_{\pi(q)}\right) < \theta\right] \tag{5.18}$$

The hyperparameters λ, α, the number of passes and θ are chosen by K-fold cross-validation on the training split only (K = 5, folds stratified by task type), maximising the mean validation reward:

$$h^* = \arg\max_h \frac{1}{K} \sum_{k=1}^{K} \bar{r}\left(\pi_h^{(D \setminus D_k)}, D_k\right) \tag{5.19}$$

where

$\pi_h^{(D \setminus D_k)}$ the policy with hyperparameters h trained without fold k;
$\bar{r}(\pi, D_k)$ its mean reward (Equation 5.15) on the held-out fold.

Table 5.7: Learned-policy training configuration (from camr/learn/bandit.py and rl_report.json)

Setting	**Value**
Language-model training	None inside CAMR: the answering model's weights are frozen. (A separate LoRA baseline, section 5.8.13, is the only model trained in this study.)
Training / evaluation questions	144 / 72 (disjoint; 60/60/24 and 30/30/12 PopQA/HotpotQA/GSM8K)
Actions (arms)	local; mem128; mem512 (+ bridging); mem1024 (+ bridging); cloud (Kimi K3)
Logged outcomes	Every action on every question: 720 training and 360 evaluation answers
Context features (11)	Bias; task type (3 one-hot); best-note similarity; margin to second note; number of near-best notes; question length; task type × best-note similarity (3)
Reward	correct − 0.3 × [cloud call] − 0.1 × prompt tokens / 1,000
Offline learner	Ridge regression per action (Equation 5.16), closed-form; first study λ = 1.0; tuned by 5-fold CV over λ ∈ {0.01, 0.1, 1, 10, 100} × three feature sets

Setting	Value
Online learner (LinUCB)	Bandit feedback only (Equation 5.17); first study α = 0.5 and **5 epochs** (passes) over the 144 training questions = 720 episodes, shuffled with seed 13, held-out evaluation every 20 episodes (37 checkpoints); tuned by CV over α ∈ {0, 0.1, 0.25, 0.5, 1, 2}} × 1–10 passes
Grounding cascade	Ridge model of answer signals (Equation 5.18); threshold θ tuned by CV over 0–1 in steps of 0.05
Cloud-cost sweep	0, 0.05, 0.1, 0.2, 0.3, 0.5, 0.8, 1.2 (Pareto table)

Figure 5.8 shows the learned decision for one question.

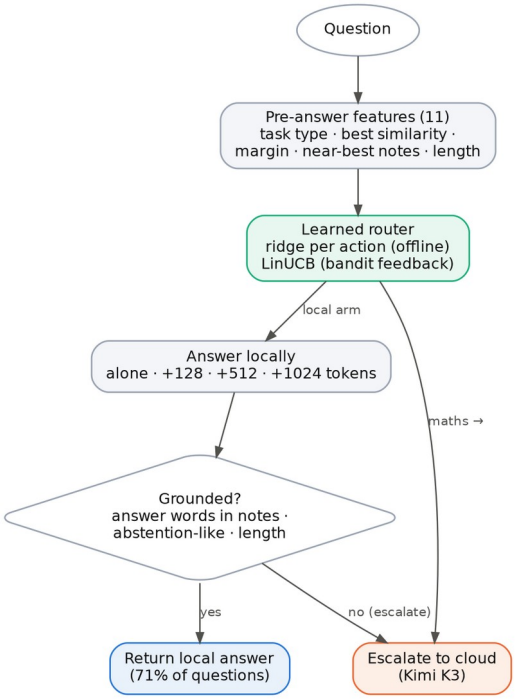


Figure 5.8: Learned engine policy: a router chooses local memory budgets or the cloud from pre-answer features, and a grounding check escalates local answers that are not supported by the notes they read

5.4.7 Model Selection and the Validation–Test Protocol

Because no weights were trained, *selection* meant choosing which frozen model and which engine configuration to recommend, and that choice was made on evidence kept apart from the final test. The **baseline** was each small model answering alone (the floor condition). The **candidates** were 19 quantised models from 0.5B to 14.8B parameters across ten families (Table 5.34), each run with the same store, questions and prompt templates (seed 13, greedy decoding, temperature 0). The pilot sample (30/30/12 questions) and the 500-question retrieval study served as the **validation** evidence: they fixed the read path (similarity ranking, gating at 0.50 with a 0.15 margin, entity bridging) and the budgets (128 tokens for single

facts, 512 for multi-hop). The 200-question HotpotQA study (reported in section 5.8) then served as a **held-out test**: its configuration was written to `configs/hotpot200.yaml` before it ran and was not changed afterwards. The same discipline was then applied to every tunable part: the read path's margin and budget were tuned on one half of the retrieval questions and tested on the other (section 5.8.2); the router's hyperparameters by cross-validation on the training split (section 5.8.12); and the LoRA baseline's format, learning rate and epochs on a validation split (section 5.8.13). Every run's configuration, hash, model digest and per-question records are saved in its results database and in `records/*.jsonl`, so each reported number can be regenerated.

5.4.8 Inspection Interface and Integration Workflow

The inspector implements the four wireframes of Figures 4.10–4.13 and opens the results database read-only (`mode=ro`), so it cannot write (FR-17, IR-06). The screenshots below were captured automatically by Playwright from the real results database. Figure 5.9 is the Run Dashboard of the final pilot. Figure 5.10 follows one question end to end: the 0.5B model alone answered "John Sturges" (wrong), and with CAMR it read one admitted note of 47 tokens and answered "Walter West" (correct). Gating stopped the next candidate (similarity 0.62 vs 0.77) at the budget cutoff.

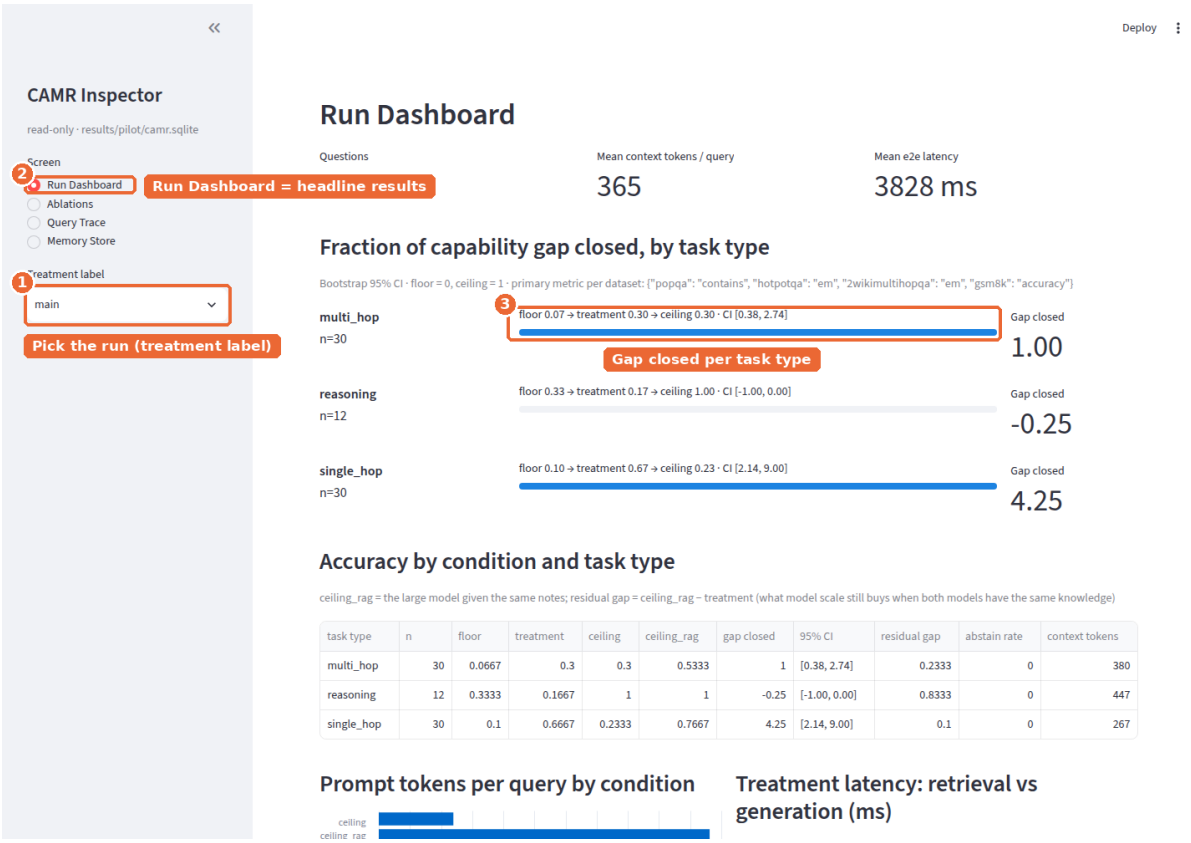


Figure 5.9: Run Dashboard of the final pilot (markers: run selector, screen, gap closed per task type)

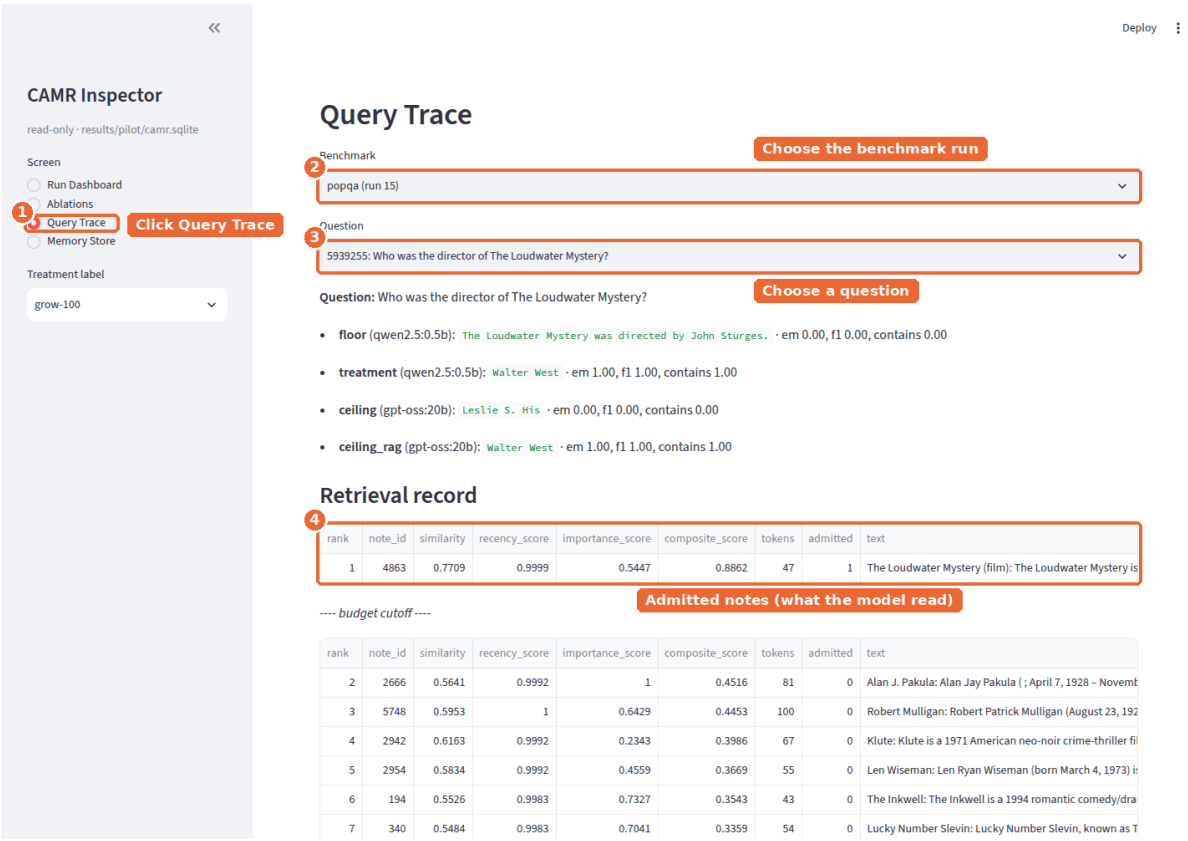


Figure 5.10: Query Trace for one PopQA question at full memory, showing the admitted note and the budget cutoff

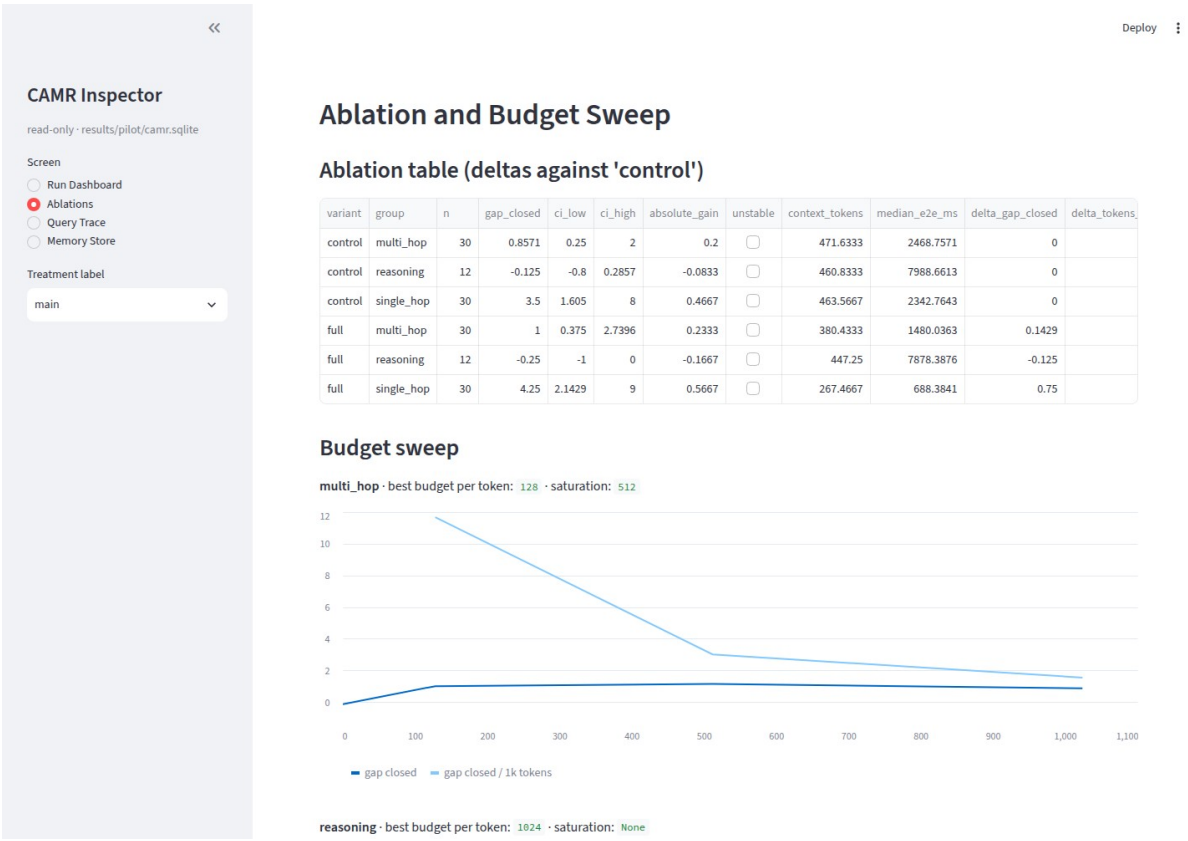


Figure 5.11: Ablations and budget-sweep screen

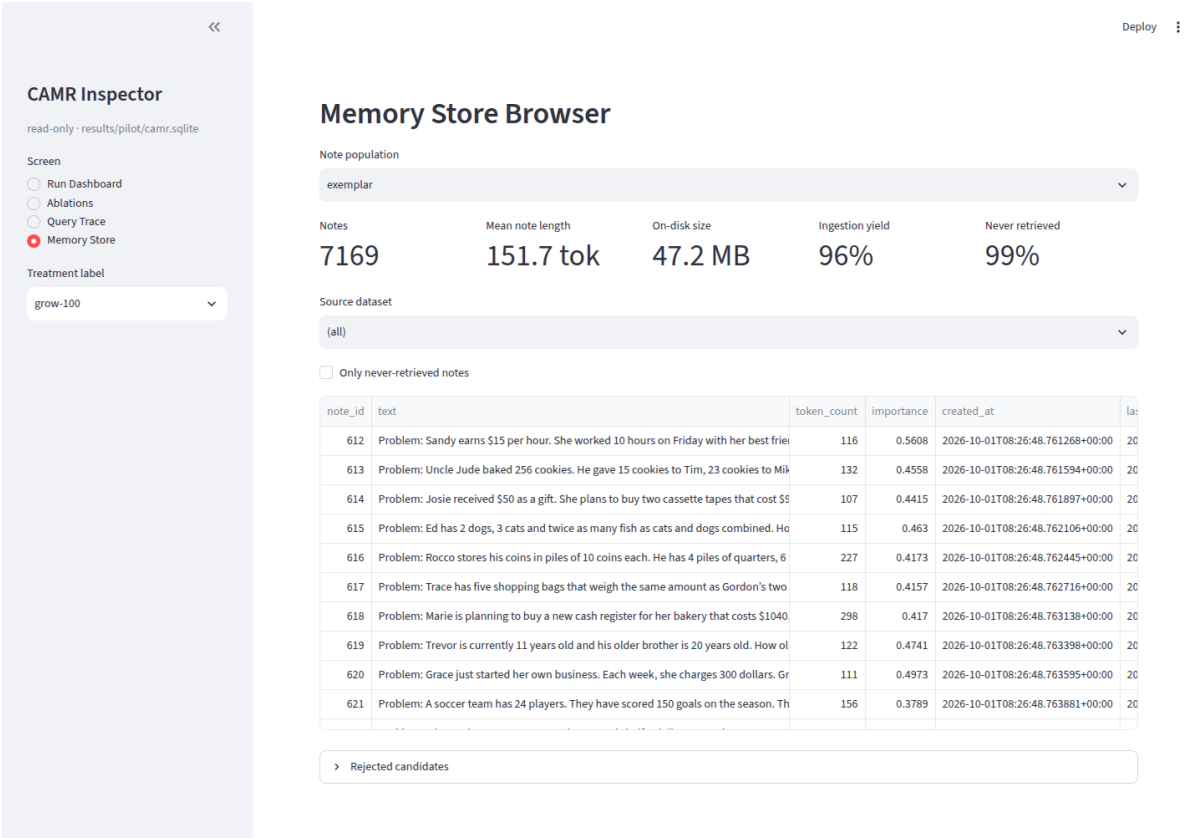


Figure 5.12: Memory Store browser: notes, provenance and retrieval counts

The deployment scenario of section 4.2.1 was exercised with the `camr ask` command on five personal-style notes (Figure 5.13, Figure 5.14). With an empty store, the 0.5B model invented answers ("next Monday"). With the notes, it answered from them ("Thursday 9 October at 14:30"), showing the note used, 26 context tokens, 27 ms retrieval and 1.7 s generation. For an off-topic question the engine abstained (0 notes admitted) and the model answered wrongly from its weights. This failure mode motivates the escalation policy.

empty memory: the model guesses

```
$ camr ask --out results/assistant-empty "When is my dentist appointment?"
{"answer": "next Monday", "notes": [], "context_tokens": 0}
$ camr ask --out results/assistant-empty "What gift does Mum want for her birthday?"
{"answer": "A present for Mum's birthday is a new dress.", "notes": [], "context_tokens": 0}
```

Figure 5.13: Assistant scenario with empty memory: the frozen model guesses

with memory: answers from your notes

```
$ camr ask "When is my dentist appointment?"
{
  "answer": "Thursday 9 October at 14:30",
  "notes": [
    {
      "note_id": 1,
      "text": "Dentist: Dentist appointment moved to Thursday 9 October at 14:30 with Dr. Wanjiru at Upper Hill Dental, 3rd floor.",
      "score": 1.0
    }
  ],
  "context_tokens": 26,
  "retrieval_ms": 26.6,
  "generation_ms": 1706.9
}
$ camr ask "What gift does Mum want for her birthday?"
{
  "answer": "A hardcover copy of Things Fall Apart by Chinua Achebe.",
  "notes": [
    {
      "note_id": 4,
      "text": "Mum's birthday: Mum's birthday is on 17 November. She wants a hardcover copy of Things Fall Apart by Chinua Achebe.",
      "score": 1.0
    }
  ],
  "context_tokens": 28,
  "retrieval_ms": 33.2,
  "generation_ms": 576.2
}
$ camr ask "At what mileage is the next oil change due?"
{
  "answer": "89,200 km",
  "notes": [
    {
      "note_id": 5,
      "text": "Car service: The car was serviced at 84,200 km on 20 September. Next oil change is due at 89,200 km.",
      "score": 1.0
    }
  ],
  "context_tokens": 28,
  "retrieval_ms": 29.5,
  "generation_ms": 354.9
}
$ camr ask "What is the capital of Kenya?"
{
  "answer": "Khartoum",
  "notes": [],
  "context_tokens": 0,
  "retrieval_ms": 25.0,
  "generation_ms": 199.4
}
```

Figure 5.14: Assistant scenario with memory: answers grounded in the user's notes

5.4.9 CAMR Personal: the Engine as an Application

To move from benchmarks to real use, the engine was packaged as **CAMR Personal**, a desktop application that gives any model the user already has in Ollama a long-term memory.

The user teaches it notes and files (.txt, .md, .pdf), dropped anywhere on the window; states facts with *remember that …*; and approves good answers with a thumbs-up, which stores them as memory. What the user says in chat is also stored, so a fact mentioned weeks earlier is recalled even after it has left the model's context window. Each answer reads only the notes it needs, using the measured best read path (similarity, gating and bridging). A *Recalled N memories* card appears before the answer streams, and the answer shows how grounded it is in those notes. The model's weights never change: the application adapts to its user through storage, not through compute or a longer context window.

The interface was designed after studying the conventions of ChatGPT (a minimal composer and chat history), Ollama's desktop app (the model picker inside the composer, file drop) and Kimi (sources shown before the answer), with memory made the visible centre: a ring that fills as memory grows, and a panel showing what was read, everything stored, and how memory has grown. It is a standard-library Python server with a single-page front end, bound to 127.0.0.1, refusing foreign hosts and cross-site writes. Embeddings come from a small Ollama model (nomic-embed-text), so the download needs no PyTorch. Its abstain threshold (0.55) was calibrated on HotpotQA with the same rule that places BGE's benchmark threshold (0.50): the 90th percentile of off-topic similarity. Self-contained downloads are built by continuous integration for Windows (x64, ARM64), macOS (Apple Silicon, Intel) and Linux (x64, ARM64). Figure 5.15 and Figure 5.16 show the application running with qwen2.5:1.5b on the demonstration notes.

Figure 5.15 traces one message through the application. Numbered screenshots of the main steps follow: first-run setup, the empty chat, a conversation with a remembered fact, a recalled memory and an abstention, the memory panel, and the phone-width layout.

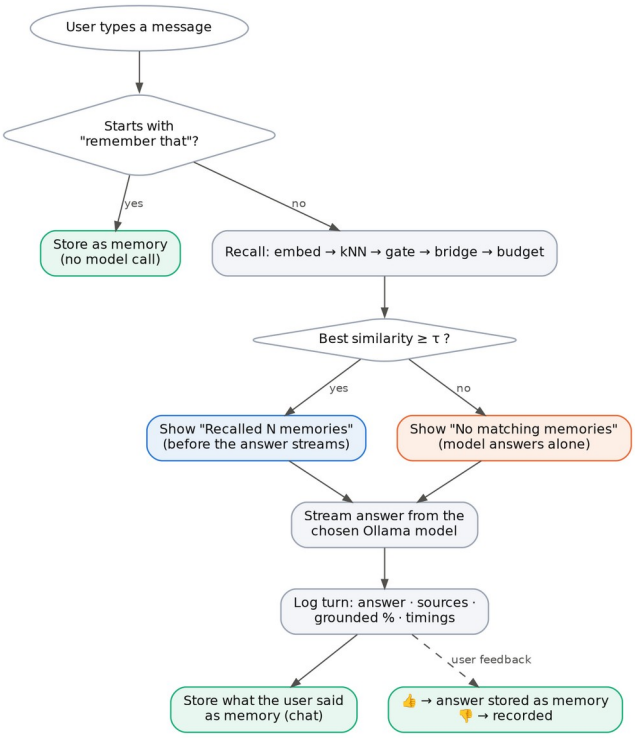


Figure 5.15: One message through CAMR Personal (implemented flow)

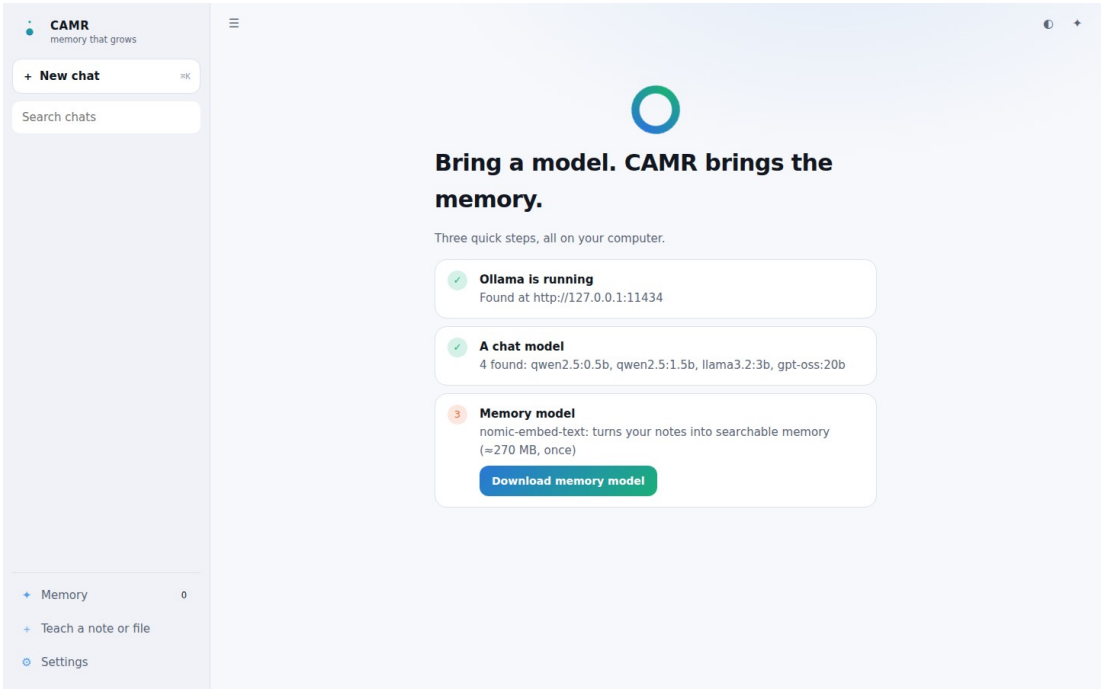


Figure 5.16: Step 1. First run: the app checks Ollama, finds the user's models and downloads the memory model

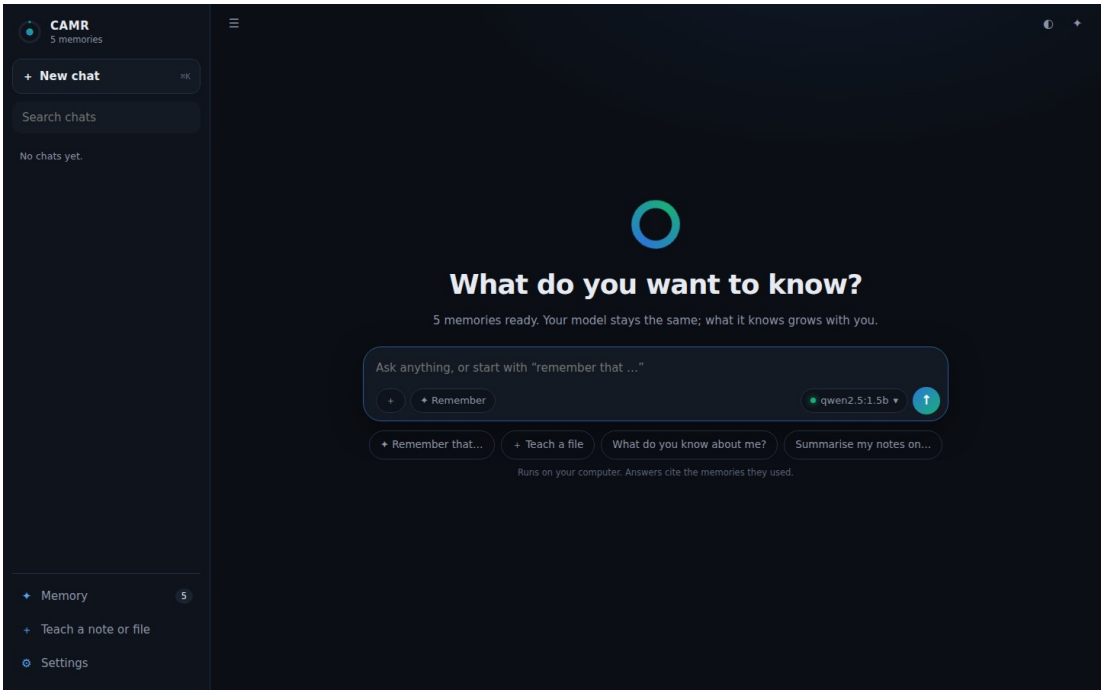


Figure 5.17: Step 2. Empty chat: the model picker sits inside the composer; suggestions start teaching

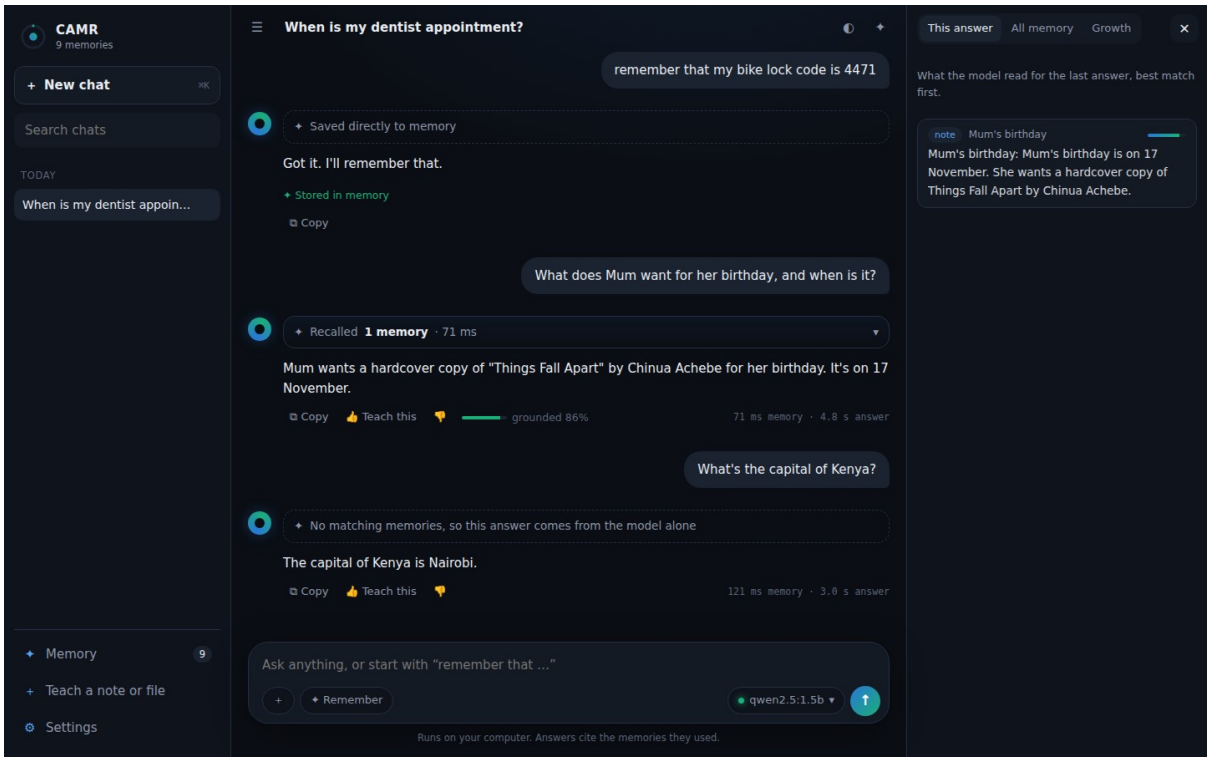


Figure 5.18: CAMR Personal: a remembered fact, an answer with its recalled memory and grounding, and an abstention, with the memory panel open

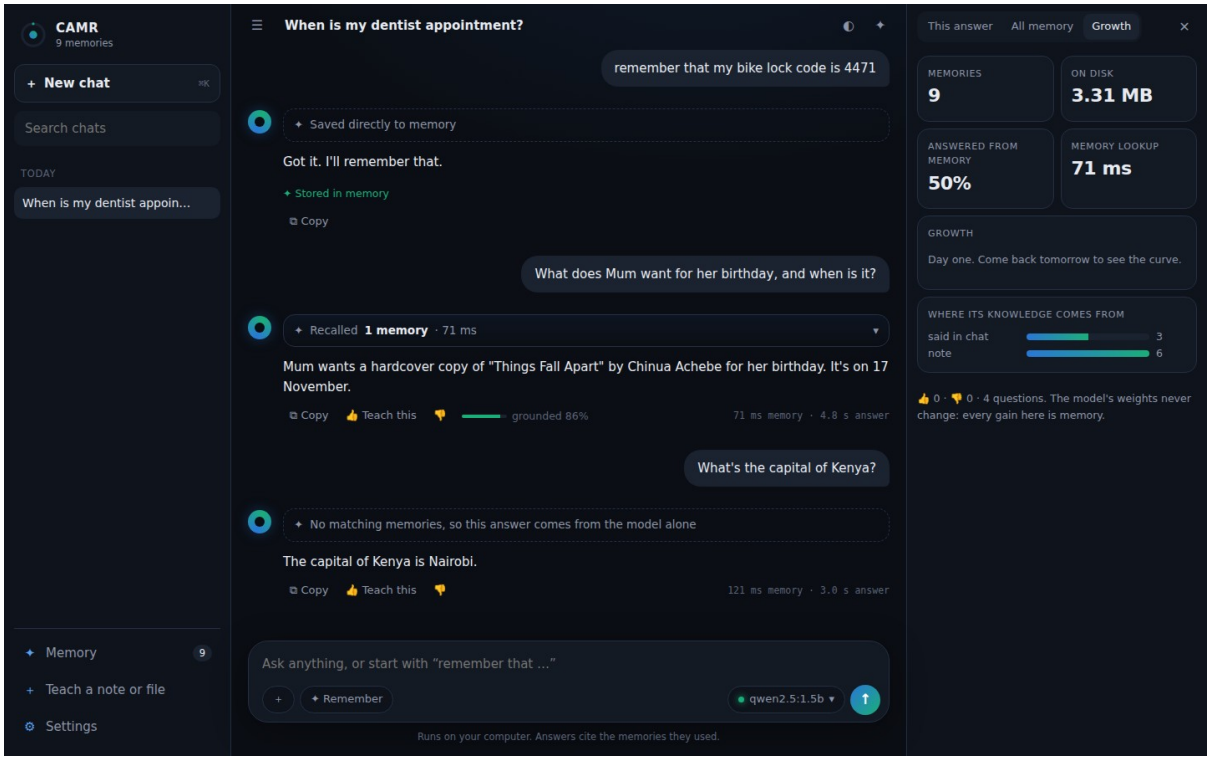


Figure 5.19: CAMR Personal: the growth panel (memories, share of answers drawn from memory, lookup time, sources of knowledge)

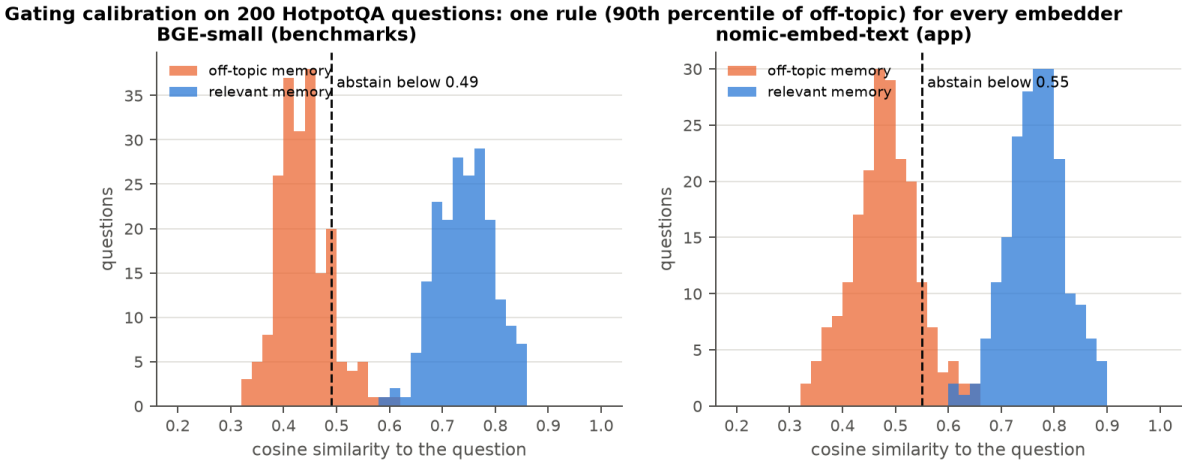


Figure 5.20: Gating calibration: relevant and off-topic similarity on 200 HotpotQA questions; the threshold is the 90th percentile of off-topic similarity for each embedder

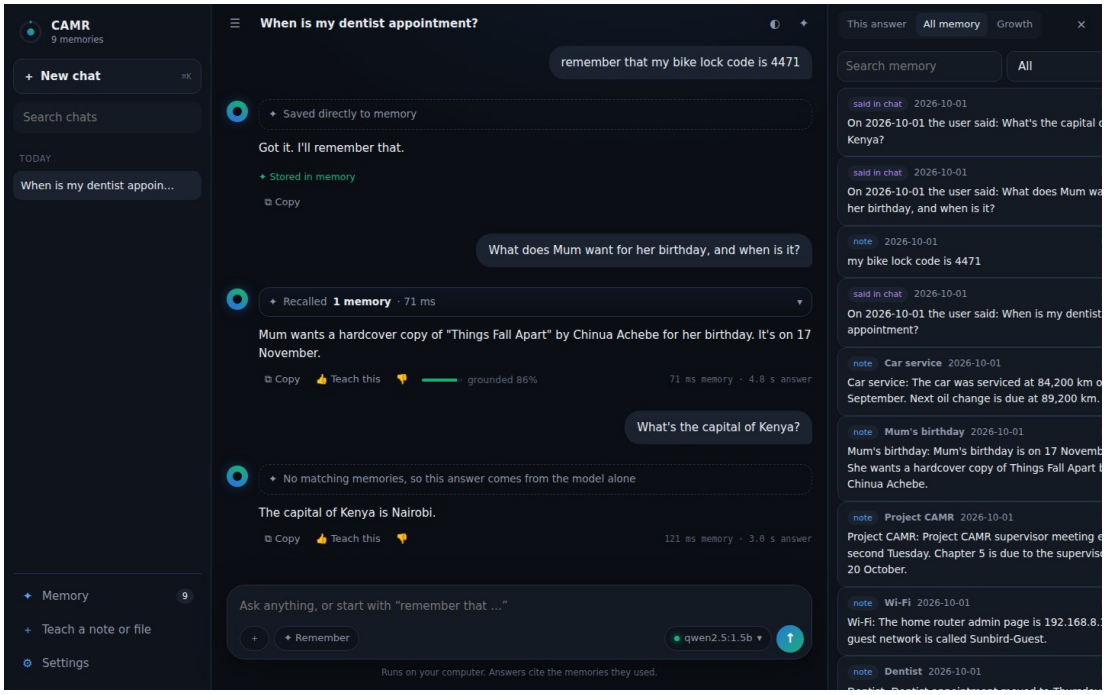


Figure 5.21: All memory: search, provenance by kind, and Forget on every item

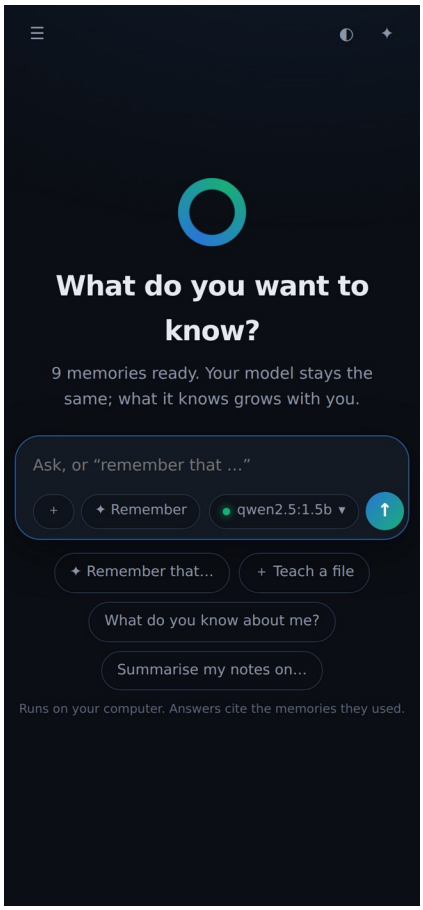


Figure 5.22: The same interface at phone width

5.4.10 Interface Design of CAMR Personal

The interface had one job beyond being a usable chat window: to make memory, the thing the research is about, visible and controllable. Its design followed six principles, each traceable to a feature and to a test (Table 5.8).

Table 5.8: Design principles of the interface and how each was realised and checked

Principle	**Realised as**	**Checked by**
Memory is visible, before the answer	A *Recalled N memories* card streams before the first word; an *answered by the model alone* label when gating abstains	TC-27, TC-25; browser test (TC-37)
Familiar conventions	ChatGPT's minimal composer and chat list; Ollama's model picker inside the composer and drop-anywhere files; Kimi's sources-first answers	Design review against the three apps
Honest provenance	Grounded % per answer; each memory labelled note / chat / learned / file; Forget on every item	TC-28, TC-29; memory panel test
The user stays in control	*remember that…* stores without a model call; 👍 teaches, 👎 only records; export hides text by default	TC-26, TC-39
Local and private by construction	Server bound to 127.0.0.1; Host and Origin checked; no telemetry	TC-31, TC-32
Accessible and responsive	WCAG 2.2 AA contrast in both themes; one heading structure; layout from 390 px phones to desktops	Contrast audit, axe-core audit, phone-width test (TC-40, TC-41)

Figure 5.23 shows how the interface is built. The front end is plain HTML, CSS and JavaScript with no framework or build step, so the downloadable executable stays small (16–43 MB) and starts in about a second. Answers arrive as Server-Sent Events in a fixed order: *recall* (which notes were read), then *token* events, then *done* with timings and the grounding score. That order is what puts memory in front of the answer.

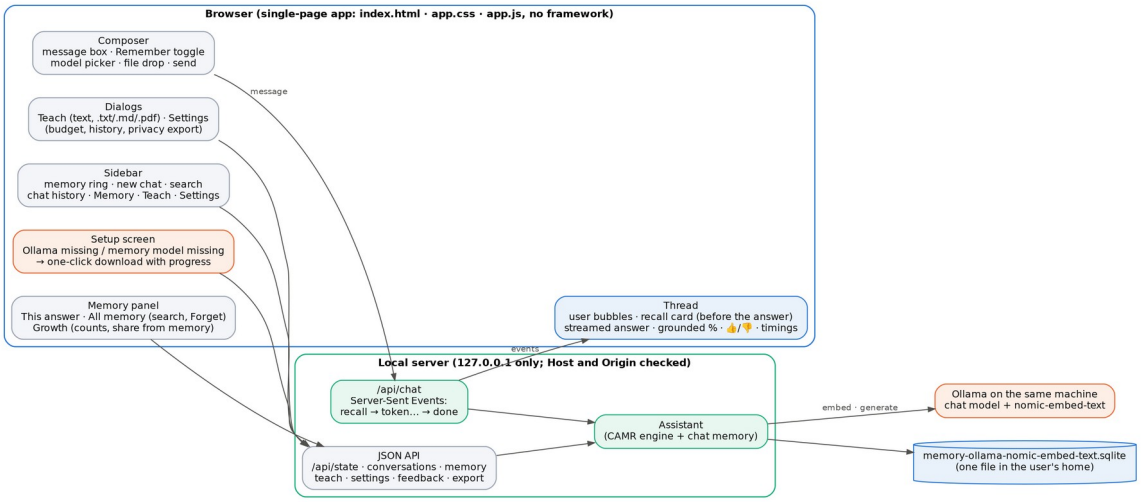


Figure 5.23: Interface components of CAMR Personal and the local server they talk to

Layout. A three-column CSS grid: a 272 px sidebar, a fluid thread capped at 768 px for comfortable line length, and a 360 px memory panel that slides in. Below 900 px the sidebar becomes an overlay and the panel covers the screen, so the same interface works at phone width (TC-41). **Typography.** The system UI font stack (Inter where installed) at 15 px with 1.6 line height; monospace for codes and timings. **Colour.** Dark-first, with a single blue-to-green accent taken from the study's chart palette, and a light theme. Every colour is a named design token, so the two themes differ only in token values.

Text must stand out from its background for readers with low vision. WCAG 2.2 defines the contrast ratio of two colours from their relative luminance L, computed from the linearised sRGB channels:

$$\mathrm{CR} = \frac{L_{\mathrm{light}} + 0.05}{L_{\mathrm{dark}} + 0.05}, L = 0.2126\,R_{\mathrm{lin}} + 0.7152\,G_{\mathrm{lin}} + 0.0722\,B_{\mathrm{lin}} \tag{5.20}$$

Level AA requires CR ≥ 4.5 for normal text and ≥ 3 for large text and interface marks. Figure 5.24 shows every token pair the stylesheet uses, measured with this formula after the audit in section 5.5.4 corrected the four tokens that failed.

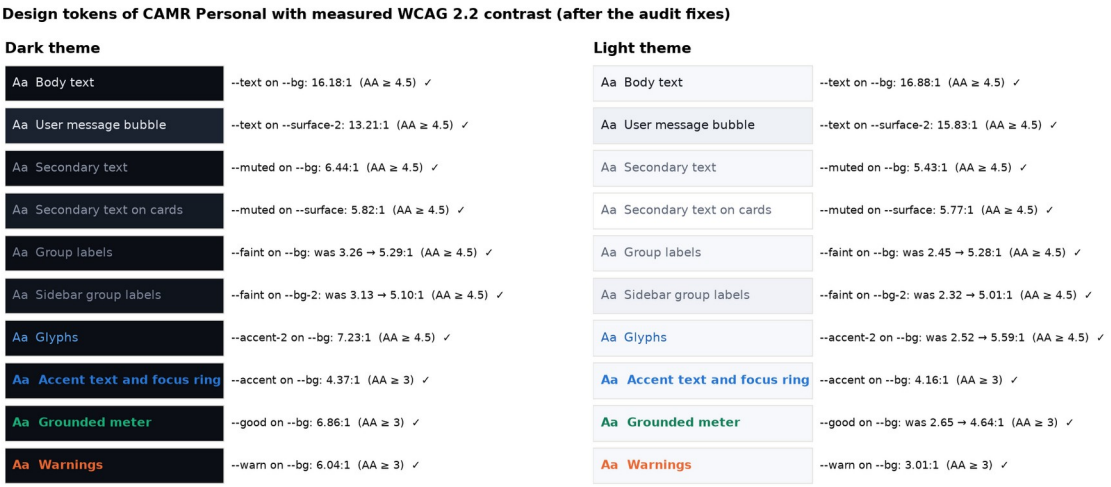


Figure 5.24: Design tokens of both themes with their measured contrast ratios

5.4.11 Learning Beyond What It Was Taught (CAMR Personal 0.3)

In the engine evaluated so far, memory grows only when the user teaches it. When the gate abstains, the small model is on its own, and for a 0.5B model that usually means a guess or *I don't know*. Version 0.3 of the application adds three ways for memory to grow by itself, none of which touches the model's weights: it looks up what it does not know (**eager learning**), it installs ready-made memory (**knowledge packs**), and it reorganises and extends

its memory while idle (**sleep**). This section defines them; Sections 5.8.14 to 5.8.16 measure them.

External weights. The design treats the model as *compute* and the memory as *weights that live outside it*. Everything the engine can learn without a GPU is counted in one place:

$$\left|\Theta_{\text{ext}}(t)\right| = N(t)\,d + 2\,d\,r + C\,d\,,\theta_{\text{model}} \ \text{ fixed} \tag{5.21}$$

where

$N(t)$ the number of notes at time t (each a d-dimensional vector, with its text and source);
$2\,d\,r$ the query adapter that sleep trains (Equation 5.27);
$C\,d$ the centroids of the learned index (Equation 5.26);
θ_{model} the language model's weights, which never change.

Only the first term grows with use. Its cost in memory (RAM) is what makes the design viable on a small device: the vectors stay on disk and a question reads only the part of memory it needs,

$$R(t) \approx R_{\text{model}} + R_{\text{engine}} + \frac{n_{\text{probe}}}{C} N(t)\,d \cdot 4 \ \text{ bytes} \tag{5.22}$$

so the resident memory of the whole system is dominated by the fixed model, not by how much it knows. Figure 5.25 draws this split; Section 5.8.16 measures Equation 5.22.

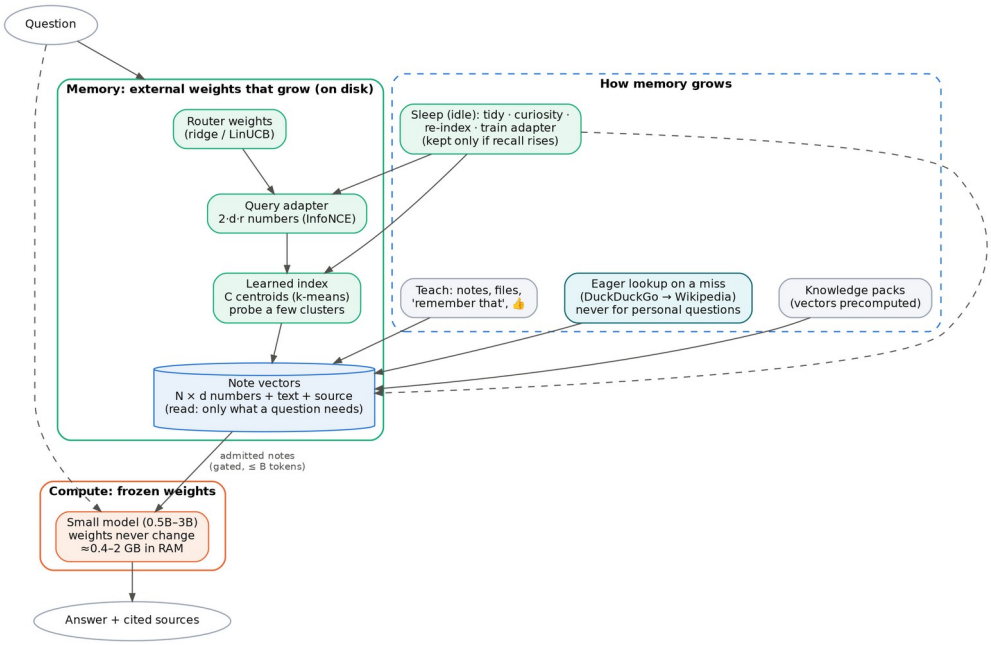


Figure 5.25: External weights: a frozen model for compute, and memory that grows on disk (note vectors, learned index, query adapter, router) fed by teaching, eager lookups, packs and sleep

There are three places a frozen model's knowledge can live, and this project now measures all three on the same 237 documents and 60 held-out questions (Table 5.9). *Inside* the model, by changing weights (LoRA, Equation 5.32, Section 5.8.13). *Beside* the model, as a token-level memory that the frozen model consults at every step: the k-nearest-neighbour language model (kNN-LM; Khandelwal et al., 2020) stores, for every position of every document, the model's own hidden state as a key and the next token as a value, and mixes the tokens of the nearest keys into the model's prediction,

$$p(y \mid x) = \lambda\, p_{\mathrm{kNN}}(y \mid x) + (1-\lambda)\, p_{\mathrm{LM}}(y \mid x) \tag{5.23}$$

$$p_{\mathrm{kNN}}(y \mid x) \propto \sum_{(k_i, v_i) \in N_k(h(x))} 1[v_i = y] \exp\left(\cos\left(h(x), k_i\right)/\tau\right) \tag{5.24}$$

(Equations 5.23–5.24), which is the most literal form of *external weights used as memory while the model's parameters do the compute*. And *in the prompt*, which is CAMR: notes chosen per question and read by the model.

Table 5.9: Three places to keep a frozen model's knowledge, compared on the same documents

	Inside (LoRA)	**Beside (kNN-LM)**	**In the prompt (CAMR)**
What changes	8.8 M adapter weights (trained)	A datastore of hidden states (no training)	Notes and their vectors (no training)
Cost to add a fact	Re-train (minutes to hours)	One forward pass over the text	One embedding of the text
Can a fact be removed?	No (retrain without it)	Yes (delete its entries)	Yes (Forget)
Can the answer cite a source?	No	No	Yes (every note keeps its source)
Measured	Section 5.8.13	Section 5.8.15	Sections 5.8.13, 5.8.15

Eager learning. When the gate abstains, the 0.3 read path treats it as a gap in memory rather than an answer. Writing *L(q)* for the pages a lookup returns and *W* for the write path (screening, deduplication, provenance), the action for a question is

$$\mathrm{act}(q) = \begin{cases} \text{read } A(q) & A(q) \neq \varnothing \\ \text{ask to be taught} & A(q) = \varnothing,\ q \text{ personal} \\ \mathcal{M} \leftarrow \mathcal{M} \cup W(L(q));\ \text{read } A(q) & A(q) = \varnothing,\ \text{lookups on} \\ \text{model alone} & \text{otherwise} \end{cases} \tag{5.25}$$

Figure 5.26 shows the loop. Three design decisions keep it safe and honest:

- **Privacy.** A question that refers to the user (*my*, *I*, *we*, …) is never sent anywhere; the assistant says it does not know that yet and offers a *Teach me* button. For a general

question only the subject strings are sent (e.g. *Doomwatch*, *director of Doomwatch*), not the conversation.

- **Sources.** Lookups use two keyless public services: DuckDuckGo Instant Answers (which returns the lead of the matching Wikipedia article, or the list of meanings of an ambiguous name) and Wikipedia's page summaries. What is read is stored as notes through the ordinary write path, so it is screened, deduplicated, and keeps its link; the answer cites it like any other memory.
- **Politeness.** One request at a time, at most one a second, every response cached, and rate limits honoured (Retry-After; an empty reply is treated as *slow down*, never cached as *nothing known*; defect D-18). The cache also makes every experiment replayable offline with exactly the text it read.

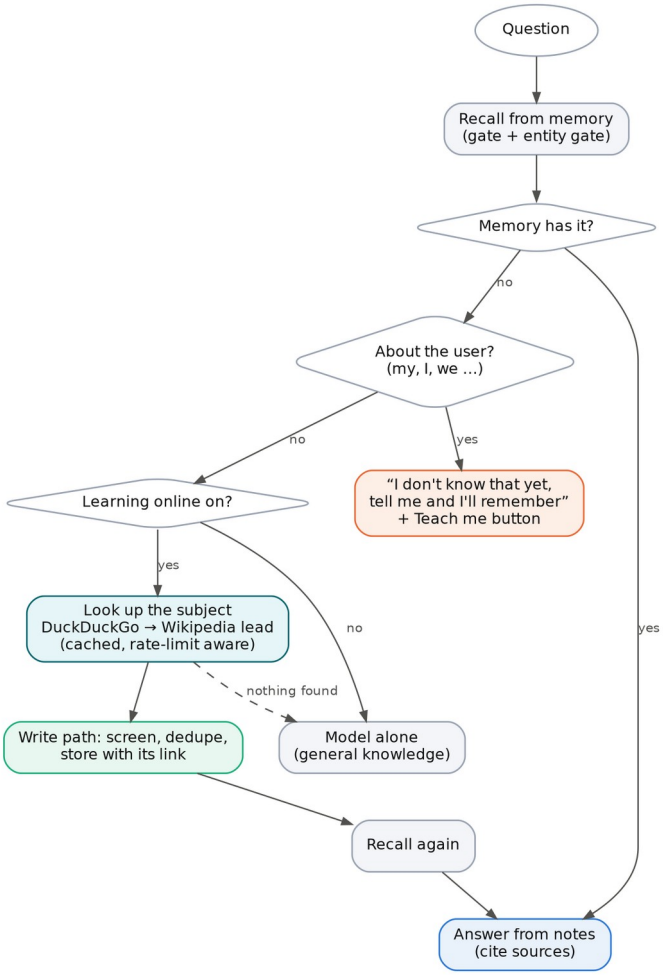


Figure 5.26: The eager read path: memory first; a personal gap asks to be taught; a general gap is looked up, learned through the write path, and answered from memory

Knowledge packs. A pack is one file holding notes, their sources and, for one embedder, their vectors. Installing a pack whose embedder matches the user's copies the vectors into the store: no model runs, so a pack plugs in precomputed external weights. With another embedder the notes are re-embedded on install. Notes still pass the screener, so a pack cannot inject instruction-like text or duplicates. A pack can be built from the web for any topic (*harvest*: the topic's article, related articles, then the articles for the names they mention most) and is removed as a unit.

Sleep. After ten idle minutes (or on request) the application consolidates memory in four steps, each of which reports what it changed: (1) *tidy*: near-duplicate notes from the web and packs (cosine ≥ 0.97) are merged, keeping the oldest, and the user's own notes are never deleted; (2) *curiosity*: names that public notes mention repeatedly but that memory has no note about are looked up, so the next multi-step question finds its second hop already stored (personal notes are never mined); (3) *index*: once memory passes 100,000 notes (the size at which it was measured to pay, Section 5.8.16), a learned index is rebuilt; (4) *learn to fetch*: a query adapter is trained on the engine's own experience. The index is an inverted file (Jégou et al., 2011): spherical k-means learns C centroids and each note is filed under its nearest one, so a question is compared with the centroids and then only with the notes of the n_probe closest clusters,

$$\text{cost}_{\text{IVF}} \approx C\,d + \frac{n_{\text{probe}}}{C} N\,d \;\ll\; N\,d = \text{cost}_{\text{exact}}, C \approx \sqrt{N} \tag{5.26}$$

with the vectors stored by cluster in a memory-mapped file, so unprobed clusters are never read into RAM. Notes written after the index was built are searched exactly until the next sleep files them, like a short-term buffer that consolidation empties. The query adapter is a low-rank map applied to the question's vector only (the stored notes are untouched),

$$q' = \frac{q + U\,V^{\top} q}{\lVert q + U\,V^{\top} q \rVert}, U, V \in \mathbb{R}^{d \times r},\; U_0 = 0 \tag{5.27}$$

so an untrained adapter is the identity. It is trained on pairs (question, note that answered it) taken from answers the user approved or that were well grounded, with the contrastive InfoNCE loss (van den Oord et al., 2018) over the batch and each question's hardest wrongly retrieved notes:

$$\mathscr{L}_{\text{NCE}} = -\frac{1}{B}\sum_{b=1}^{B}\log\frac{\exp\left(q_b^{'}\cdot n_b/T\right)}{\sum_{j}\exp\left(q_b^{'}\cdot n_j/T\right)} \tag{5.28}$$

It is written in NumPy (gradients checked against finite differences to 10^{-10}), so it trains inside the packaged application without a deep-learning framework. Because learning from a handful of answers can as easily hurt as help, sleep keeps a newly trained adapter only if it does better on held-out experience:

$$\text{keep}\ (U, V) \Longleftrightarrow R_5^{\text{new}} > R_5^{\text{old}} \ \wedge \ R_1^{\text{new}} \geq R_1^{\text{old}} \tag{5.29}$$

where

R_k recall at k on the 20% of experience pairs held out from training.

5.5 Testing and Evaluation

5.5.1 Test Strategy and Acceptance Criteria

Testing combined three levels. **Unit and integration tests** (white-box, pytest, 148 tests, fully offline) verify each requirement with deterministic test doubles: dry-run models and a hashing embedder. **System tests** run the real pipeline end to end on real data with real models. **Measurements** (device profile, latency, memory) verify the non-functional requirements against their thresholds. Each test maps to a test case of Table 4.5. Acceptance criteria were fixed before the runs: a functional test passes when the observed behaviour equals the expected behaviour; NFR-02 passes when retrieval is ≤ 10% of median end-to-end latency; NFR-05 passes when two runs produce byte-identical prompts.

Figure 5.27 shows a sample of requirement test cases running (captured output).

```
requirement test cases (examples)

$ python -m pytest -v tests/test_harness.py -k "runtime_down or holdout or byte_identical or bootstrap"
tests/test_harness.py::test_records_are_complete_and_prompts_byte_identical PASSED
tests/test_harness.py::test_holdout_guard PASSED
tests/test_harness.py::test_runtime_down_logs_failures_and_continues PASSED
tests/test_harness.py::test_gap_closed_and_bootstrap PASSED
======================== 4 passed, 17 deselected in 4.81s ========================
```

Figure 5.27: Example requirement test cases: failure handling (IR-07, D-07), holdout guard (DR-04), byte-identical prompts (NFR-05) and the bootstrap (FR-15)

5.5.2 Test Cases, Defects and Retesting

Table 5.10: Test cases and results (unit tests: pytest, 148 passed; measurements: camr profile)

Test ID	Traceability	Scenario and data	Expected result	Actual result and verdict
TC-01	FR-01	Verbatim and structured policies on fixture passages	Notes produced; title kept	As expected. Pass
TC-02	FR-02/03, DR-06	Empty, duplicate, over-length, injection-like notes	Rejected with logged reason; provenance on all notes	As expected. Pass
TC-03	FR-04, IR-03	Commit a note	Timestamp, importance, 384-d embedding stored	As expected. Pass
TC-04	FR-05, IR-04, NFR-03	kNN via sqlite-vec vs NumPy fallback	Identical scores; single file	As expected. Pass
TC-05	FR-06/07	Similarity-only and composite ranking; Eq. 4.3–4.4	Order follows the equations	As expected. Pass
TC-06	FR-08	Budgets 0–1,024 over random note lengths	Budget never exceeded	As expected. Pass
TC-07	FR-09, IR-01	Runner interface; non-loopback host	Only `generate`; remote host refused	As expected. Pass
TC-08	FR-10, IR-02, NFR-09	Cloud runner with failing transport	Bounded retries, then logged failure	As expected. Pass
TC-09	FR-11, DR-02	Shuffle the dataset file; redraw	Same sample; persisted with seed and hash	As expected. Pass
TC-10	FR-12	EM, F1, contains, GSM8K final number	Matches hand-computed values	As expected. Pass
TC-11, TC-19	FR-13, NFR-05, NFR-08	Run the same config twice	Complete records; byte-identical prompts	Unit: pass. System: 30/30 identical prompts. 0.5B answers identical on 20/30 only (D-02)
TC-12	FR-14, NFR-07	Ablation plan from config	All variants and budgets planned	As expected. Pass
TC-13	FR-15	Gap closed and bootstrap; near-zero denominator	Correct CI; unstable flagged	As expected. Pass
TC-14	FR-16, NFR-01	Profile on machine A, 3 × 10 questions	Peak memory recorded; fits 16 GB	Engine 712 MB + runtime 1987 MB. Pass
TC-15	FR-17, IR-06	Write through the inspector connection	Write fails at the driver	As expected. Pass
TC-17	NFR-02	Retrieval share of median latency, 0.5B	≤ 10%	56% in the pilot profile. **Fail** (D-05; see 5.8.17)
TC-18	NFR-04, IR-05	`camr reproduce` end to end (dry-run and real)	All tables and figures regenerated	As expected. Pass
TC-20	NFR-06	Block sockets; run floor and treatment	No external connection	As expected. Pass
TC-21	DR-03/04	Corpus containing an evaluation question	Ingestion refused	As expected. Pass
TC-22	IR-07	Local runtime down mid-run	Failures logged; run continues; store intact	As expected. Pass
TC-23	Deployment (4.2.1)	CAMR Personal: teach, ask, remember, 👍, forget, settings, export; server API, streaming, host/origin checks	Answers cite notes; chat recalled beyond the window; export hides text; foreign hosts refused	14 tests and a browser test with a real model. Pass

The defects below were found during the real runs. Each was fixed and retested, or is reported as an open limitation. Failed results were kept in the record, not deleted.

Table 5.11: Defects found and retests

ID	Defect	Found by	Correction	Retest
D-01	The screener rejected the real song title "I Can't Get Next to You" as a model refusal	Retrieval run 1 rejection log	Refusal check applied only to model-generated notes	Regression test added. Pass
D-02	The 0.5B model's answers differed on 10/30 questions between identical runs (all wrong → wrong)	Repeat runs	Engine confirmed deterministic (prompts identical); floors reported as a range	Open: runtime non-determinism
D-03	gpt-oss:20b was killed for running out of memory at 13.3 GB	Pilot ceiling run	Memory-mapped weights; one model loaded at a time	Loaded. Pass
D-04	Changing a configuration field mid-study changed the hash, so resume re-ran finished runs	Pilot resume	Freeze the schema before final runs	Pass
D-05	Retrieval took 56% of answer time (NFR-02)	TC-17	Diagnosed as CPU contention; thread partitioning settings added	Engine alone 29 ms; p90 157 → 50 ms. Still above 10% for 0.5B
D-06	Composite score (Eq. 4.1) lowered full-support recall by 4.6 points	Retrieval evaluation	Similarity + gating + bridging as the best read path	2Wiki recall 35.8% → 78.4%; HotpotQA equal at 11% fewer tokens
D-07	When the model server stopped mid-sweep, failed queries were logged (IR-07) but the run was still marked completed, so resume skipped it	Machine B sweep	Resume re-runs any run with failed queries; the failed run is kept	Regression test added. Pass; 19 runs re-run clean
D-08	gemma2:9b was killed for running out of memory three times at 13.4 GB (runtime prompt cache growing on top of the weights)	Machine B sweep	Runtime prompt cache capped at 1 GiB (affects prefill reuse only)	No further kills. Pass
D-09	CAMR Personal: the shared store connection failed when the web UI called it from another thread, and the UI's file watcher crashed while crawling the PyTorch modules	App browser test	Store opened for cross-thread use behind a lock; file watcher disabled	App tests and browser test. Pass
D-10	CAMR Personal: the new warm-up loaded the model with the default context size, so Ollama loaded it a second time for the first question (1.8 s to the first word instead of a fraction of a second)	App latency benchmark	Warm-up sends the same options as a chat request	Regression test; latency re-measured (Table 5.45). Pass
D-11	Interface: secondary text (group labels, keyboard hints, memory-kind chips) and, in the light theme, accent and success colours were below WCAG AA contrast (2.3–3.3 : 1)	Accessibility audit (section 5.5.4)	Token values corrected per theme	Contrast audit and axe-core: 0 failures. Pass
D-12	Interface: no top-level heading once a chat had started (screen-reader navigation)	axe-core audit	Persistent, visually hidden page heading	axe-core: 0 violations. Pass
D-13	CAMR Personal over-refused:	Live scenarios	Prompt limits *I don't know*	Regression test;

ID	Defect	Found by	Correction	Retest
	with llama3.2:3b it answered *I don't know* to general questions it could answer (17 × 23, the capital of Australia), because the prompt told it to refuse whenever the notes lacked the answer	(section 5.5.5)	to questions about the user and their notes; general questions use the model's own knowledge	scenarios re-run. Pass
D-14	Chat memory stored every message, including plain questions; a general question then recalled six unrelated past questions as *memories*	Live scenarios	Only statements are kept as chat memory; questions are not	Regression test; scenarios re-run. Pass
D-16	The first D-13 fix was incomplete: the 3B model still refused some general questions (the capital of Australia) and now refused a question its notes answered (*which antibiotic am I allergic to?*)	Scenario retest	Five candidate prompts compared on a development set and two held-out sets of cases (`scripts/prompt_eval.py`); the one with no failures on either model adopted	21/21 cases on both models; scenarios re-run. Pass
D-15	Entity gate: a possessive in the question (*Peck's Bad Girl*) was normalised differently from the same name in a note, so the gate wrongly withheld a correct note	RAG comparison (section 5.8.3)	One normalisation for question and notes, possessives removed	Regression test; gate re-evaluated on validation, test and retrieval. Pass
D-17	Web lookups: a title's leading *The* was stripped mid-sentence (*The Other* looked up as *Other*), and titles with lower-case words (*Son contento*) were cut short	Learning stream, first 20 questions (section 5.8.14)	The article is kept unless the name opens the question; the words after *of* are also tried	Unit tests; stream re-run from scratch. Pass
D-18	Web lookups: an empty reply, which is how DuckDuckGo signals *too fast*, was cached as *nothing known*, so 20 later lookups failed for good	Learning stream, first run (questions 18–24 found nothing)	An empty reply is a rate limit: back off, retry, never cache it; poisoned entries removed	Unit test; stream re-run from scratch. Pass
D-19	The first *ask to be taught* prompt (E) made the 3B model lead with *I don't know that yet* although its notes held the answer (*which antibiotic am I allergic to?*)	Prompt evaluation, development set	Prompt F: answer first when the notes answer; ask to be taught only for a missing personal fact	6/6, 7/7, 8/8 on both models; regression test. Pass
D-20	Topic harvesting drifted: the *Solar System* pack collected actors from a film of that name, and the *Kenya* pack lacked the article on Kenya	Inspection of the first packs	Keep, and snowball from, only pages whose relevance to the topic is ≥ 0.8 of the best seed's	Unit test; packs rebuilt. Pass (a few off-topic pages remain; section 5.4.11)
D-21	Web lookups did not identify themselves: a session's default agent (*python-requests*) was kept, and Wikimedia throttles and blocks generic agents, so most Wikipedia requests failed	Pack rebuild (a direct request with the CAMR agent succeeded where the fetcher's failed)	The CAMR User-Agent replaces a session's default	Unit test; learning stream repeated after the fix (section 5.8.14). Pass
D-22	Sleep's curiosity read names across sentence ends (*East of Kenya. This*) and looked them up	Live scenario *pack and sleep*	Names are read within one sentence	Regression test; scenario re-run. Pass
D-23	Interface 0.3: the *grows* and *trained* chips in the Learning tab were below AA contrast in the light theme	axe-core audit with the Learning tab open	Chips use the darker learned-memory green	axe-core: 0 violations in both themes at both sizes. Pass

5.5.3 Application Testing (CAMR Personal)

The application was tested as a user would meet it, with valid, empty, out-of-scope and unsupported input and with the runtime missing. The live checks used qwen2.5:1.5b and nomic-embed-text through Ollama in a browser driven by Playwright. The automated checks are in `tests/test_app*.py` and in the continuous-integration smoke test.

Table 5.12: Application test cases and results

Test ID	Scenario	Expected result	Actual result and verdict
TC-24	Empty message (spaces only)	Rejected; no model call; no chat created	Error event, no chat (test). Pass
TC-25	Out-of-scope question ("What's the capital of Kenya?") with only personal notes stored	Gating abstains; the answer is labelled as coming from the model alone	0 memories admitted; labelled; answered "Nairobi". Pass
TC-26	"remember that my bike lock code is 4471"	Stored without a model call; duplicate reported	Stored; second time "already in memory" (test and browser). Pass
TC-27	Question answered by a note	Recalled memory shown before the answer; grounded answer	1 memory, 71 ms; correct answer, grounded 86%, 4.8 s (1.5B, CPU). Pass
TC-28	Fact said in chat, then asked after it left the context window	Recalled from memory	Recalled as "something you said" (test). Pass
TC-29	Teach a Markdown file and a PDF	Chunked and stored as memory	Stored (test; PDF via pypdf). Pass
TC-30	Ollama not running / memory model missing	Setup screen with the missing step and a download button	Setup screen; download with progress; then chat (browser; CI checks the not-running state). Pass
TC-31	Request with a foreign Host header, or a cross-site POST	Refused (403)	403 (test). Pass
TC-32	Path traversal (/../../etc/passwd)	No file outside the app is served	Index page served (test). Pass
TC-33	Downloadable build on six platforms	Builds, launches, serves the UI and API	All six green in CI (Table 5.22). Pass
TC-34	Time to first word recorded for every answer	Measured from prompt sent to first streamed word; never larger than the answer time	Recorded and shown under the answer (test). Pass
TC-35	App starts, or the user switches model	Chat model loaded in the background before the first question	Empty-prompt load request sent (test); first answer faster (Table 5.45). Pass
TC-36	Same question asked twice	Second lookup reuses the question's embedding; documents never served from the cache	One embedding call for two lookups (test). Pass

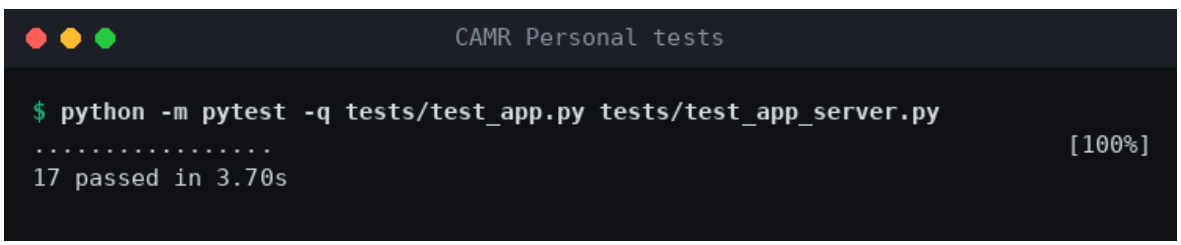


Figure 5.28: Application test run: tests for the assistant core and the web server (captured output)

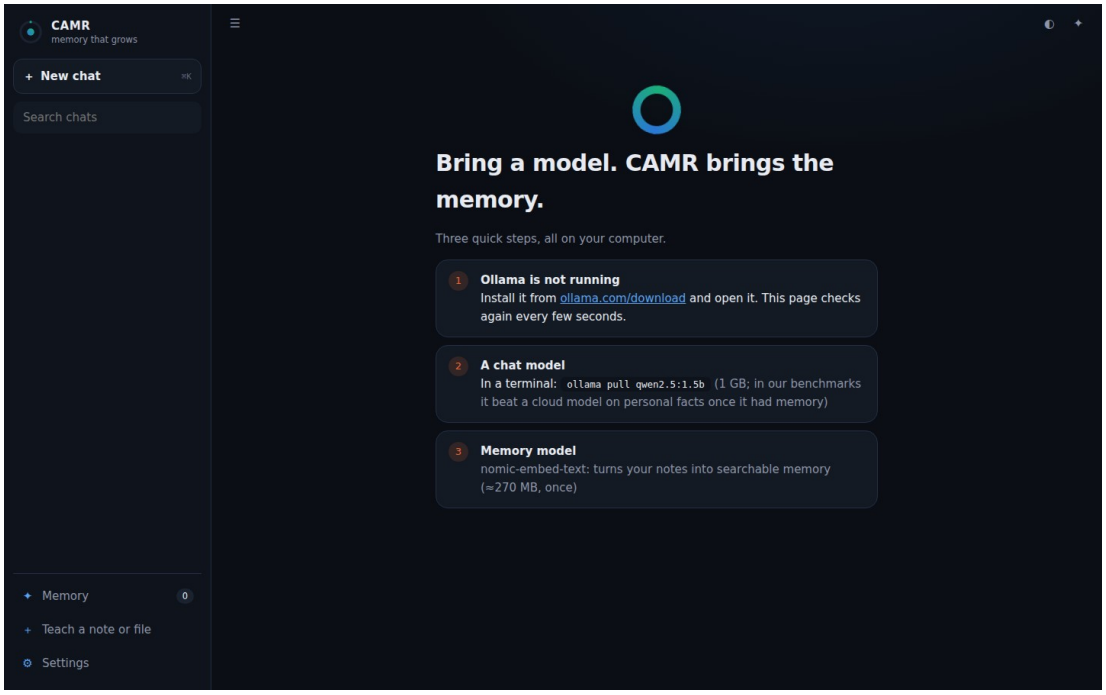


Figure 5.29: TC-30: with Ollama not running, the app shows what is missing instead of failing

These tests show that the application behaves correctly and fails safely. They do not show that people find it useful or easy to use. That requires a user study, which had not been run when this chapter was written (see Limitations, section 5.8.20).

5.5.4 Interface Testing: Browser, Accessibility and Layout

The interface was tested in three further ways. **End-to-end browser tests** (`tests/test_app_e2e.py`) drive the real interface in Chromium through Playwright, as a user would, with the deterministic test doubles in place of the model so that a failure means the interface is wrong, not the model. An **automated accessibility audit** (`scripts/ui_audit.py`) runs axe-core 4.10 (Deque Systems) against WCAG 2.0/2.1/2.2 A and AA rules and best practices, in both themes, at desktop (1,366 × 860) and phone (390 × 844) size, on a populated chat with the memory panel open. A **contrast audit** computes Equation 5.20 for every token pair.

Table 5.13: Automated interface tests (Chromium, Playwright)

Test ID	What the browser does	Expected	Result
TC-37	Types *remember that my bike lock code is 4471*, then asks for the code	Stored without a model call; the recall card shows the memory before the answer; the answer contains 4471; memory counter increases	Pass
TC-38	Submits a message of spaces	Nothing is sent; no chat created	Pass
TC-39	Clicks 👍 on an answer	Button reads *Learned*; memory counter increases	Pass

Test ID	What the browser does	Expected	Result
TC-40	Toggles the theme and opens the memory panel	Theme changes; panel opens	Pass
TC-41	Loads the app at 390 × 844 px	Composer fully on screen; no horizontal scrolling	Pass
TC-42	Uses only the keyboard: Tab to the message box, type, Enter	Message box reachable by Tab; message sent; every visible control has an accessible name	Pass

The first audit found real defects. axe-core reported 2 rule violations (color-contrast, page-has-heading-one) and the contrast audit found 6 token pairs below AA, all of them in small secondary text (sidebar group labels, keyboard hints, memory-kind chips) and, in the light theme, the accent and success colours. The page also had no top-level heading once a chat had started, which hurts screen-reader navigation. They were logged as defects D-11 and D-12, fixed by darkening or lightening the failing tokens per theme and adding a persistent (visually hidden) page heading, and re-audited:

Table 5.14: Accessibility audit with axe-core: violations (impact, elements) before and after the fixes

Theme and size	Violations before	Rules passed before	Violations after	Rules passed after
Dark, desktop	color-contrast (serious, 2), page-has-heading-one (moderate, 1)	32	none	35
Dark, phone	page-has-heading-one (moderate, 1)	32	none	35
Light, desktop	color-contrast (serious, 4), page-has-heading-one (moderate, 1)	32	none	35
Light, phone	color-contrast (serious, 2), page-has-heading-one (moderate, 1)	32	none	35

Table 5.15: Colour-contrast failures found and fixed (WCAG 2.2 AA)

Theme	Token pair	Used for	Before	After	Required
dark	faint on bg	Group labels, keyboard hints (11–12 px)	3.26:1	5.29:1	4.5:1
dark	faint on bg-2	Sidebar group labels	3.13:1	5.10:1	4.5:1
light	faint on bg	Group labels, keyboard hints (11–12 px)	2.45:1	5.28:1	4.5:1
light	faint on bg-2	Sidebar group labels	2.32:1	5.01:1	4.5:1
light	accent-2 on bg	Glyphs, links	2.52:1	5.59:1	4.5:1
light	good on bg	Grounded meter, success	2.65:1	4.64:1	3:1

After the fixes the audit reported no violations in either theme at either size, and every token pair met AA. Keyboard-only use of the main path is covered by TC-42. Automated tools catch only part of what makes an interface accessible: the interface was **not** tested with a

screen reader or with users who rely on one, and a usability study with real users remains future work (Chapter 6).

As a structured design review, the interface was also checked against Nielsen's ten usability heuristics (Table 5.16). This is the author's own expert review, not a user study; its value is that each heuristic is tied to a concrete feature or to a gap that is admitted.

Table 5.16: Heuristic review of the interface (Nielsen, 1994)

Heuristic	**How the interface meets it**	**Known gap**
Visibility of system status	*Searching your memory…*, recall card, streaming, time to first word, download progress	—
Match with the real world	Plain words: *Teach*, *Remember*, *Forget*, *Learned*	—
User control and freedom	Forget any memory; rename or delete chats; 👎 never teaches	No undo after Forget
Consistency and standards	ChatGPT/Ollama conventions; one token system across themes	—
Error prevention	Empty messages rejected; duplicates detected; local-only server	—
Recognition rather than recall	Suggestions on the empty chat; model picker shows installed models	—
Flexibility and efficiency	⌘K new chat; Enter to send; *remember that…* shortcut	Few keyboard shortcuts
Aesthetic and minimalist design	One accent colour; memory details in a panel, not in the thread	—
Help users recover from errors	Setup screen names the missing step (TC-30); toasts explain failures	Model errors shown as raw text
Help and documentation	docs/APP.md, illustrated setup guide, About in Settings	No in-app tour

5.5.5 Complex Interactions with a Real Model

Benchmarks ask one independent question at a time. A personal assistant meets harder situations: two notes that must be joined, arithmetic on a remembered number, a fact that changes, a detail said many turns earlier, a document dropped into the window. Five such scenarios were run in the released application with **llama3.2:3b** through Ollama on the 4-core CPU, each in a fresh home, once **without memory** (the same app and model with nothing taught, or with chat memory switched off) and once **with CAMR**. The screenshots below are the real screens, captured automatically by `scripts/scenario_screens.py`, which also logs every answer. An answer counts as correct when it contains every expected element.

The first run found two defects. In the context-window scenario the model answered *I don't know* to 3 general questions it can answer (What is 17 × 23?; What's the capital of Australia?; How many days are in a leap year?), and for *How many days are in a leap year?* the engine recalled 6 earlier chat messages as memories, all of them questions. The first was caused by

the chat prompt, which told the model to refuse whenever the notes lacked the answer (D-13); the second by storing every message, including questions, as chat memory (D-14). Both were fixed and covered by regression tests. **The second run showed the first fix was incomplete (D-16):** questions were no longer stored as memories, but the model still refused some general questions and now refused *which antibiotic am I allergic to?* although it had recalled the right memory. A prompt rewrite tested on one example had moved the problem rather than solved it. Five candidate prompts were therefore compared on a development set of cases and confirmed on two held-out sets (`scripts/prompt_eval.py`, Table 5.17); the one with no failures on either model was adopted, and all five scenarios were run a third time. The results below are from that third run; the first two are kept in `docs/results/scenarios_before_fix.json` and `scenarios_retest1.json`.

Table 5.17: Choosing the chat prompt: cases answered correctly (development · held-out · second held-out)

Prompt	**llama3.2:3b**	**qwen2.5:1.5b**
O (original)	3/6 · 5/7	6/6 · 6/7
A (D-13 fix)	4/6 · 6/7 · 8/8	6/6 · 6/7 · 8/8
B (explicit cases)	5/6 · 7/7 · 8/8	6/6 · 6/7 · 8/8
C (notes first, short)	5/6 · 7/7	6/6 · 6/7
D (notes are true)	6/6 · 7/7 · 8/8	6/6 · 7/7 · 8/8

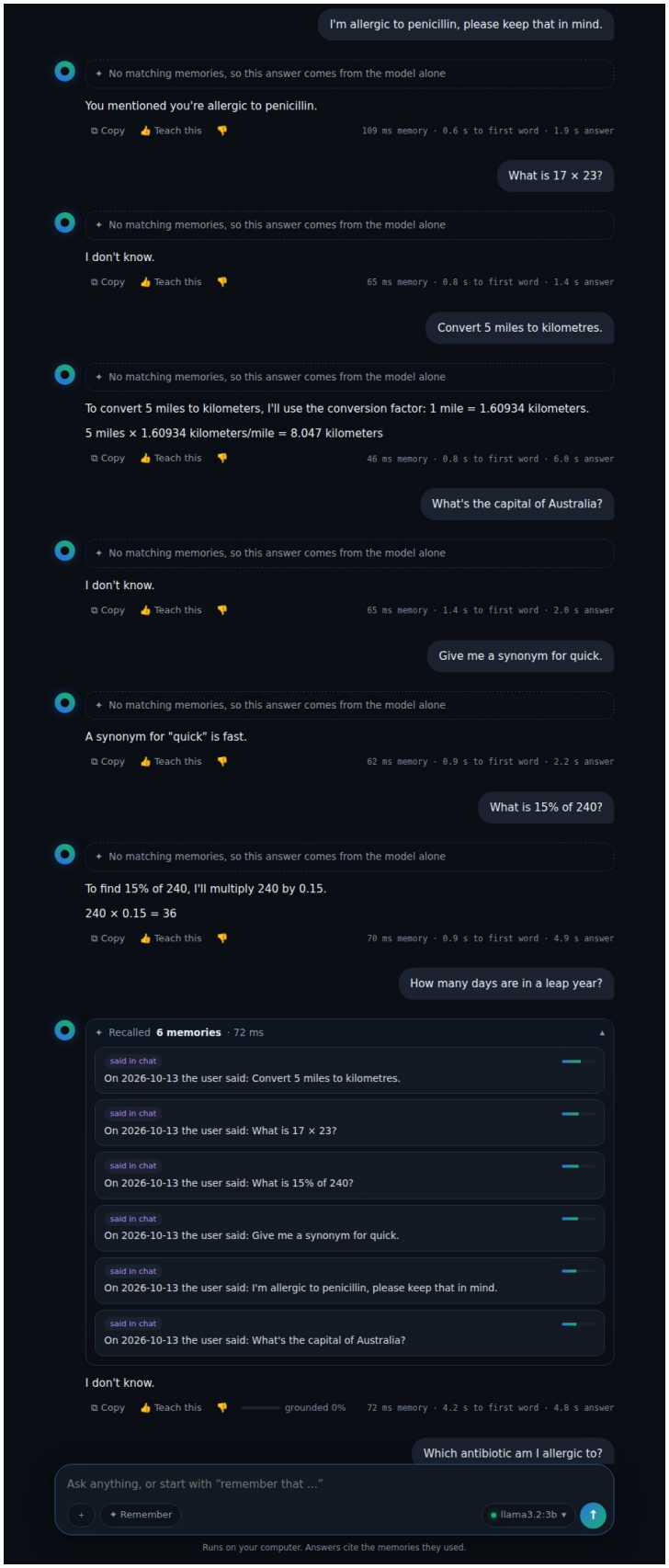


Figure 5.30: First run, before the fixes: general questions refused and unrelated chat messages recalled as memories (D-13, D-14)

Table 5.18: Complex interactions in CAMR Personal (llama3.2:3b): final question, answers without and with memory, and the memories recalled (docs/results/scenarios.json)

Scenario	Final question	Without memory	With CAMR	Recalled
An answer that needs two notes	Which room is my project supervisor's office in, and how does she want drafts sent?	wrong: I don't know.	correct: Your project supervisor, Dr. Achieng Otieno, has her office in Room 4.12 on the	Dr. Achieng Otieno, Project CAMR
Arithmetic over a remembered fact	My odometer reads 86,950 km today. How many kilometres until the next oil change?	wrong: I don't know.	correct: To find out when the next oil change is due, subtract the current odometer readi	Car service
A fact that changes	When is my dentist appointment now?	–	correct: Your dentist appointment is now on Friday, 10 October at 09:00.	note, Dentist
Remembering beyond the context window	Which antibiotic am I allergic to?	wrong: I don't know.	correct: Penicillin.	chat
Answering from a taught document	How long should I rinse my eyes after a chemical splash, and who do I report it to?	wrong: According to the notes, there is no information about the user's personal prefer	correct: You should rinse your eyes for at least 15 minutes after a chemical splash, and	lab_safety

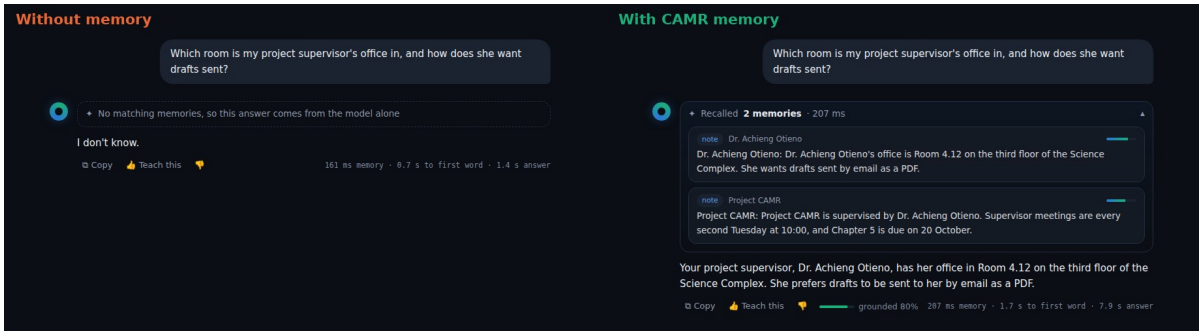


Figure 5.31: Scenario: an answer that needs two notes (left: without memory, right: with CAMR)

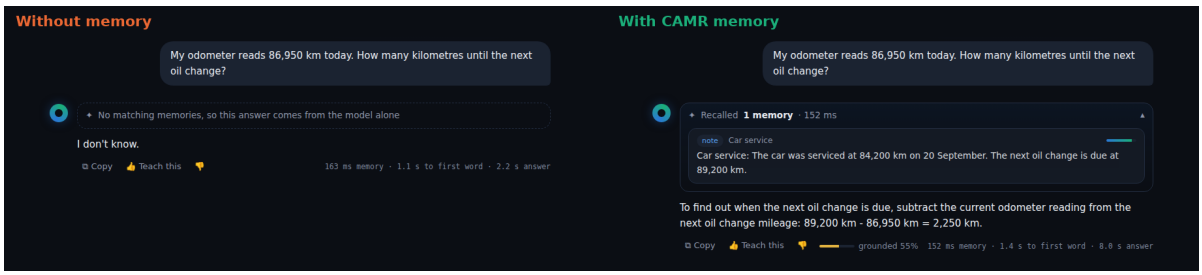


Figure 5.32: Scenario: arithmetic over a remembered fact (left: without memory, right: with CAMR)

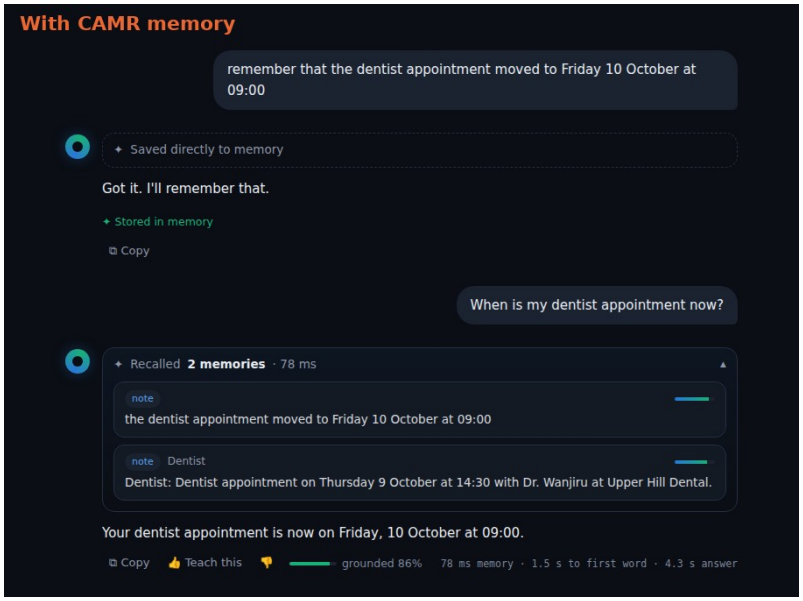


Figure 5.33: Scenario: a fact that changes

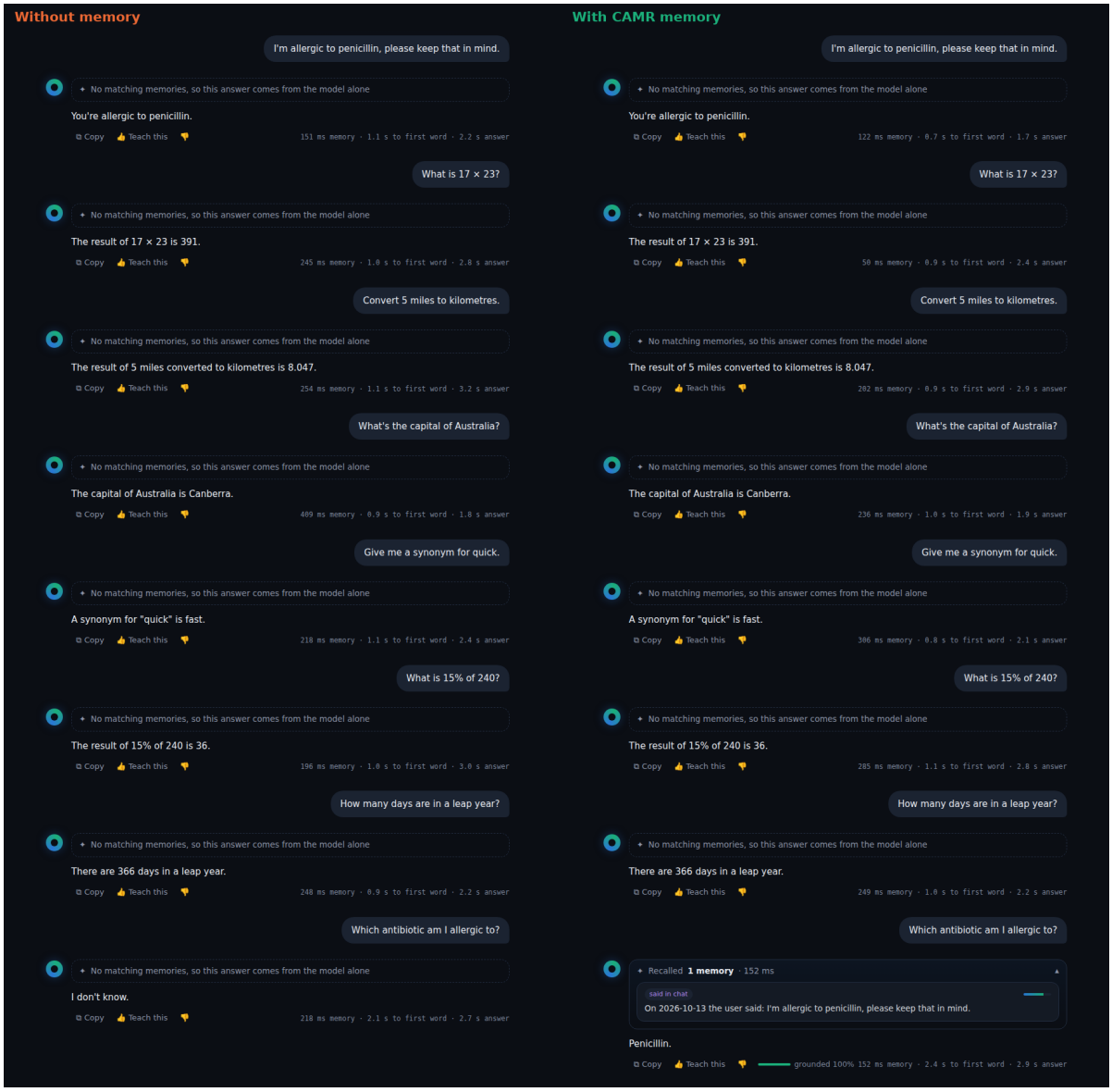


Figure 5.34: Scenario: remembering beyond the context window (left: without memory, right: with CAMR)

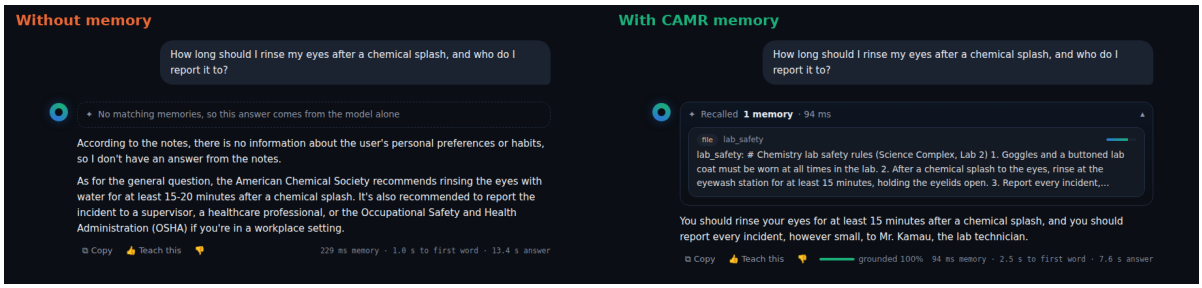


Figure 5.35: Scenario: answering from a taught document (left: without memory, right: with CAMR)

With memory the assistant answered 5 of 5 scenarios correctly; without it, 0 of 4. In detail, from the logs: (1) the two-note answer was assembled from two notes that similarity found on its own (0.72 and 0.62); bridging was not needed, because the question's words matched both,

so this scenario shows multi-note recall, not bridging (bridging is shown on the benchmark examples of section 5.8.5). (2) The model computed 89,200 − 86,950 = 2,250 km from a mileage it had only in memory. (3) After *remember that the dentist appointment moved…*, both the old and the new note were recalled (similarity 0.85 and 0.80) and the model chose the new one because its wording says the appointment moved; CAMR has no explicit mechanism for superseding a fact (recency is off by default), so this relies on the model and is listed as future work. (4) The allergy was recalled from chat memory seven turns later, after the six-turn history window had dropped it, while every general question in between was answered from the model's own knowledge with no memory recalled. (5) Without memory, the model gave a generic rinse time from its own knowledge but could not know the lab technician's name; with the taught file it answered both parts.

5.5.6 Testing the Learning Extensions

The learning extensions were tested like the rest of the engine, without the network: the web is a scripted fake that can answer, be ambiguous, or rate-limit, and the whole suite sets `CAMR_OFFLINE_WEB=1` so that no test can reach the real web. Two kinds of check matter most here: that nothing personal is ever sent, and that learning cannot make the engine worse without being noticed.

Table 5.19: Test cases for eager learning, packs and sleep (unit, application and browser tests)

Test ID	Input / action	Expected	Result
TC-43	Subjects of *Who was the director of The Other?*, *…of Son contento?*	*The Other* keeps its article; *Son contento* is tried whole (D-17)	Pass
TC-44	*What is my bike lock code?* with lookups on	Never sent to the web; the reply asks to be taught and the UI shows *Teach me*	Pass
TC-45	A lookup, repeated; then the same lookup offline	Second call made no request; offline replay returns the same page	Pass
TC-46	An empty reply from DuckDuckGo	Treated as a rate limit; not cached; the next call succeeds (D-18)	Pass
TC-47	An ambiguous name (*Issues*)	The meaning closest to the question is fetched first	Pass
TC-48	A general question memory cannot answer, then a reworded one	Looked up, learned, answered from the notes; the reworded question is answered from memory with no request	Pass
TC-49	Install a pack built with the same embedder; again; remove	Vectors copied with the embedder never called; second install adds nothing; removal deletes exactly its notes	Pass
TC-50	A pack containing an instruction-like page; a pack for another embedder	The page never enters; the other pack is re-embedded on install	Pass
TC-51	Learned index on 3,000 clustered vectors; a note written after it; a deleted note	≥ 90% agreement with exact top-10; the new note is found; the deleted one never returned	Pass
TC-52	Adapter gradients against finite differences; an adapter trained on a	Error ≤ 10^{-9}; held-out recall@1 rises by more than 10 points	Pass

Test ID	Input / action	Expected	Result
	systematic shift		
TC-53	Sleep with too little experience; duplicates among web and personal notes; names in personal notes	No adapter is kept; web copies merged, personal notes untouched; personal names never looked up	Pass
TC-54	Harvest a topic whose related pages include an off-topic one	On-topic pages kept, the off-topic page dropped (D-20)	Pass
TC-55	Every lookup request	Carries the CAMR User-Agent (D-21)	Pass
TC-56	Browser: ask a general question memory cannot answer	Recall card shows *looked up … → learned N → Recalled*; the source links to Wikipedia in a new tab	Pass
TC-57	Browser: a personal gap, then *Teach me*	Composer switches to *Remember* mode and takes focus	Pass
TC-58	Browser: Learning tab; install a pack; turn lookups off	External weights shown; memory count rises by the pack's notes; the status pill reads *offline*	Pass

5.5.7 Learning in the Application: Live Scenarios

The four behaviours of version 0.3 were run in the real application with llama3.2:3b, through the browser, each variant in a fresh home, with the answers logged by the same harness as Section 5.5.5 (`scripts/scenario_screens.py --set v3`). Table 5.20 lists what the model actually answered.

Table 5.20: Live learning scenarios: what the model answered without and with the 0.3 behaviour (last turn)

Scenario	Without (learning off / not taught)	With CAMR 0.3	Correct
Who won the 2026 FIFA World Cup? (after the models' training)	I don't know that yet. Can you tell me which country won the 2026 World Cup final?	Spain.	Yes
How many hours is a 4-point story? (a made-up unit, *kazoru*)	I don't know that yet. Can you tell me how many hours of work a 4-point story in your sprint represents?	According to the notes, one story point is half a kazoru, and a kazoru is three hours of focused work. Therefore, a 4-point story would be 4 x 0.5 = 2	Yes
What is my passport number? (not stored)	–	Your passport number is AK0412345.	Yes
Highest mountain in Kenya (Kenya pack installed), then sleep	–	The highest mountain in Kenya is Mount Kenya, and its highest peak, Batian, is approximately 5,199 meters tall.	Yes

Something newer than the model. With lookups off, the model said it did not know and asked to be told, rather than guess a winner its training could not contain. With lookups on, the first question was looked up (2 notes from Wikipedia stored, each with its link), answered from them (*Spain won the 2026 FIFA World Cup.*), and the reworded follow-up was answered **from memory with no lookup** (71 ms of retrieval) (Figure 5.36).

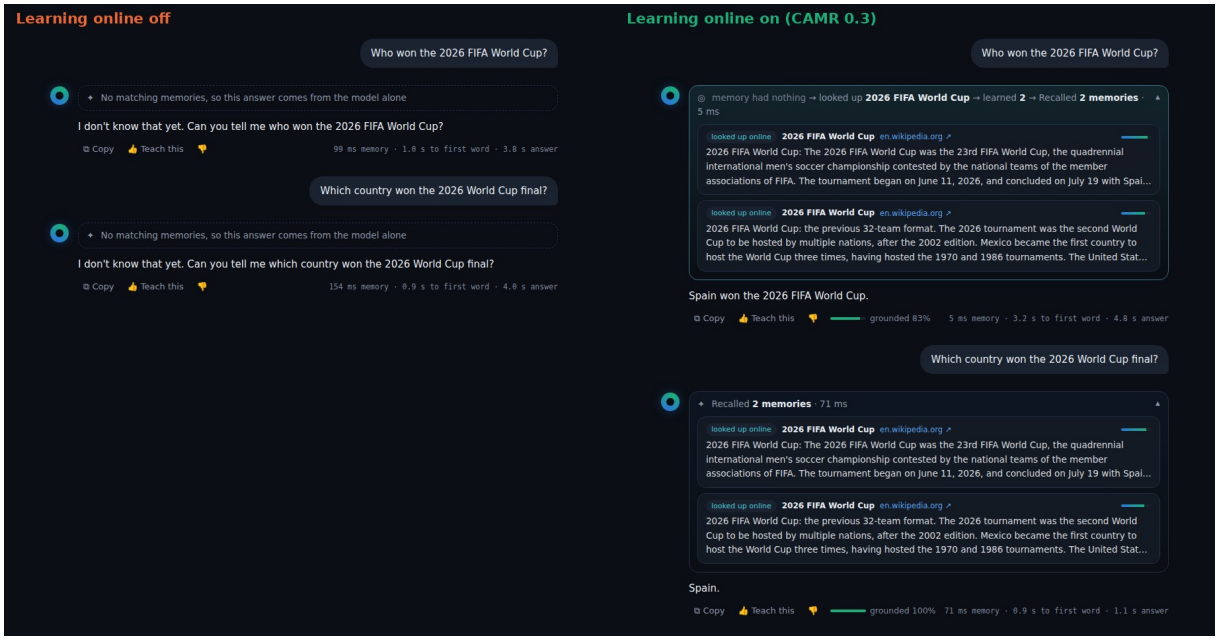


Figure 5.36: Something newer than the model: lookups off (left) and on (right); the second question is answered from what the first one taught

Teaching it a new concept. A unit the model cannot know (*a kazoru is three hours of focused work; one story point is half a kazoru*) was taught once as a note. Asked how long a 4-point story takes, the model with the note reasoned through it: *According to the notes, one story point is half a kazoru, and a kazoru is three hours of focused work. Therefore, a 4-point story would be 4 x 0.5 = 2* Without the note it asked to be taught, and the question, which says *our sprint*, was never sent to the web (Figure 5.37).

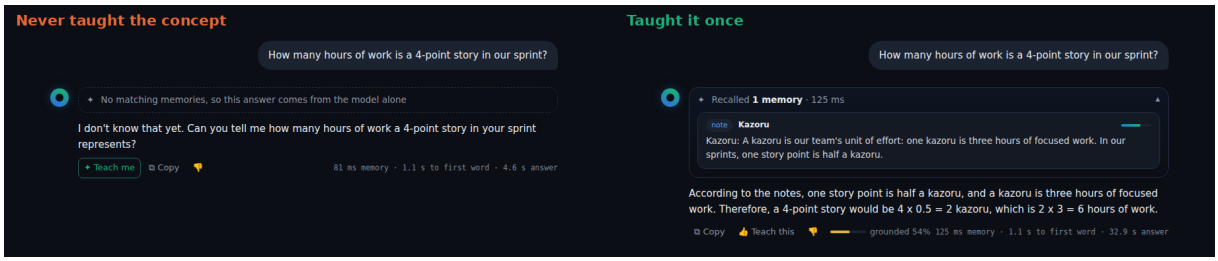


Figure 5.37: A concept taught once and applied: never taught (left) and taught (right)

Asking to be taught. For a personal fact memory did not hold, the reply was *I don't know that yet. Can you tell me your passport number so I can remember it?* with a *Teach me* button; after *remember that my passport number is AK0412345*, the same question was answered: *Your passport number is AK0412345.* (Figure 5.38).

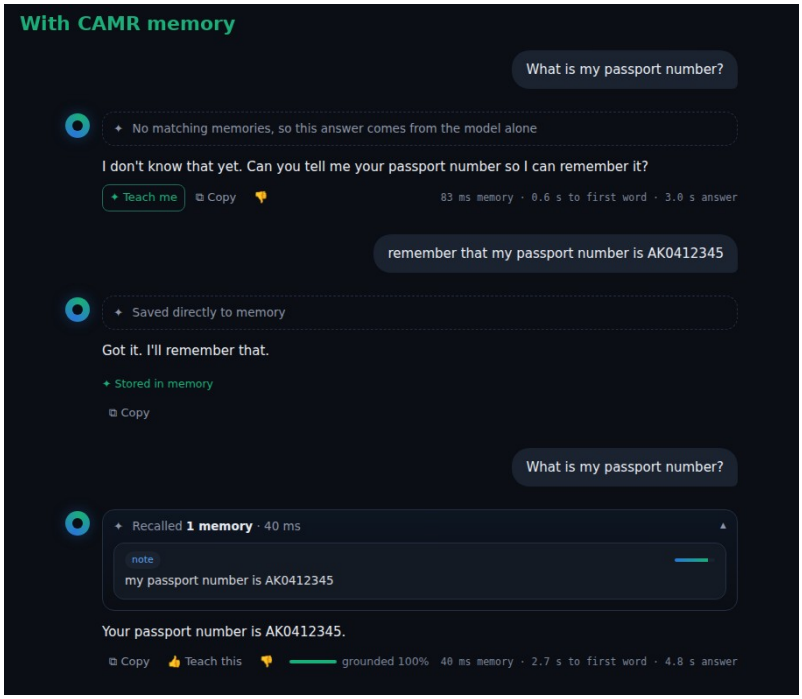


Figure 5.38: A personal gap answered by asking to be taught, then answered from what the user taught

A knowledge pack, then sleep. With the bundled *Kenya* pack installed (its vectors copied, no model run), the question was answered from the pack (*The highest mountain in Kenya is Mount Kenya, and its highest peak, Batian, is approximately 5,199 meters tall.*). Sleep then ran its four steps: it removed 0 duplicates, looked up the names the pack's notes mentioned but memory had no note about, and learned 4 of them (East Africa, Indian Ocean, World Heritage Site, Kenya National Bureau of Statistics); with only 31 notes it built no index and, with no answered questions yet, trained no adapter, both by design (Figure 5.39).

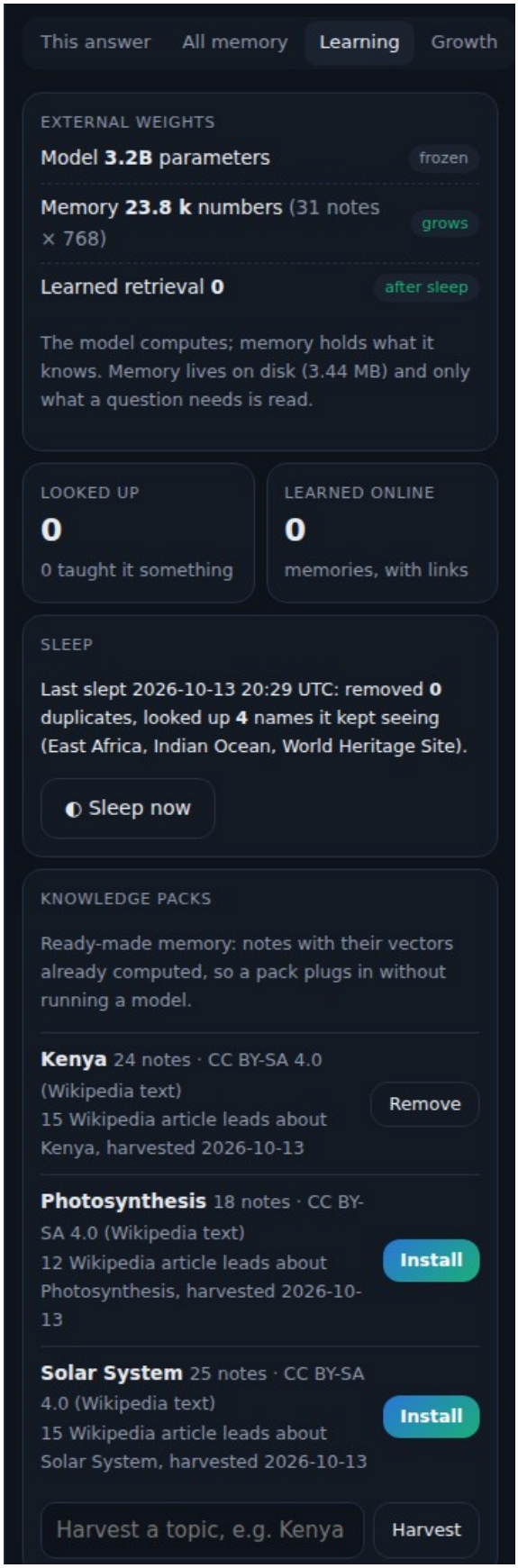


Figure 5.39: The Learning tab after installing a pack and sleeping: external weights (frozen model, growing memory), lookups, the sleep report and packs

5.5.8 Evaluation Design

The primary measure is the fraction of the capability gap closed (Equation 3.1) per task type, together with the absolute gain, the residual gap, tokens per question and latency. Two ceilings were used: the local gpt-oss:20b and the cloud Kimi K3. Against a closed-book ceiling that the small model plus memory *exceeds*, gap closed rises above 1 and stops being a fraction. Kimi K3, the frontier reference, is therefore the anchor for headline statements. With n = 30, one question is 3.3 percentage points, and confidence intervals are reported where they matter.

5.6 Version Control and Implementation Evidence

Project repository: **https://github.com/Satasdin/CAMR** (development branch `claude/engine-build-docs-yfw9u0`, merged through a pull request into `main`). The repository contains the README, an illustrated setup walkthrough (`docs/SETUP.md`), the findings log with every measurement and its source file (`docs/FINDINGS.md`), the design critique (`docs/DESIGN_NOTES.md`), dependency files (`pyproject.toml`), every experiment configuration (`configs/`), the test suite, and the scripts that regenerate every figure and screenshot in this chapter. Raw results databases and downloaded datasets are excluded by `.gitignore`, because of their size and licence terms, and are regenerated by the scripts. The Kimi API key was supplied only through an environment variable and does not appear in any commit.

Table 5.21 lists the commit history as recorded by Git. Each commit corresponds to an implemented or measured increment, and the commit messages name the finding they add. The model-size sweep ran on separate machines on result branches, whose databases were merged with `camr merge`.

Table 5.21: Commit history (git log)

Commit	Date	Author	Change
5fe043f	29 Sep 2026 09:03	Satasdin	Build the CAMR memory engine and evaluation harness from the proposal
66fa3de	29 Sep 2026 09:03	Satasdin	Initial commit
797ad01	01 Oct 2026 06:58	Satasdin	Make CAMR capability-adaptive and measure the residual gap
9a7e1b3	01 Oct 2026 07:38	Satasdin	Add real-model pilot tooling, retrieval evaluation and screening fix
3700f6a	01 Oct 2026 07:57	Satasdin	Add memory-growth experiment, larger comparison models and 2Wiki
6f9d0d0	01 Oct 2026 08:16	Satasdin	Add model-size sweep, cross-machine merge and data fetch script
f9d557e	01 Oct 2026 08:29	Satasdin	Add OpenAI-compatible cloud ceiling (Kimi) and second-ceiling analysis
a746641	01 Oct 2026 09:08	Satasdin	Findings log with real pilot results, figures and inspector screenshots

Commit	Date	Author	Change
1eec1bd	01 Oct 2026 09:08	Satasdin	Add Kimi K2.6 as a second cloud ceiling config
d738a34	01 Oct 2026 09:44	Satasdin	Learn the engine's per-question policy from rewards (contextual bandit)
a30ba07	01 Oct 2026 10:30	Satasdin	Findings: full pilot results (residual gap, ablation, budget sweep, profile), new screenshots, registry metadata
15206ae	01 Oct 2026 10:40	Satasdin	Findings F9: small model + CAMR vs Kimi K2.6/K3 with and without memory
06fed37	01 Oct 2026 10:51	Satasdin	Findings F10: model-size sweep (machine E) with figure; sweep figure readability fixes
6e9211c	01 Oct 2026 11:24	Satasdin	Findings F11-F12: learned engine policy (contextual bandit) and retrieval-latency diagnosis
e619530	01 Oct 2026 11:26	Satasdin	Grounding cascade: answer locally, escalate ungrounded answers; beats always-Kimi (65.3% vs 63.9%) with 29% cloud
0c90bd3	01 Oct 2026 11:27	Satasdin	Findings F10: add machine C models (gemma3:1b, smollm2, granite3.3:2b, falcon3:3b, phi4-mini)
1bf38e0	01 Oct 2026 11:27	Satasdin	Findings: correct count of models reaching Kimi on PopQA (10 of 11)
ea52fce	01 Oct 2026 11:38	Satasdin	Setup walkthrough with annotated screenshots and a personal-notes assistant demo
1ff3ff8	01 Oct 2026 11:44	Satasdin	Chapter 5 builder: proposal Word styles, results read from the results files
bbb65f2	01 Oct 2026 11:47	Satasdin	Findings F10b: machine B (7-14B) merged; fix D-07 resume after runtime failure
945327b	01 Oct 2026 11:48	Satasdin	Chapter 5 builder: column widths, per-model answer time, defects D-07/D-08
376eed2	01 Oct 2026 11:50	Satasdin	Chapter 5: AI-use declaration
e42a555	01 Oct 2026 12:24	Satasdin	Findings: machine D merged (19 models) and 2WikiMultiHopQA retrieval
e76a716	01 Oct 2026 12:28	Satasdin	CAMR Personal: a downloadable local assistant whose memory grows with use
e810501	01 Oct 2026 12:29	Satasdin	Chapter 5 builder: 2Wiki retrieval table, 19-model discussion, app section, n=200 study; hotpot200 config and retrieval figures
fd89571	01 Oct 2026 12:37	Satasdin	MIT licence; remove the Discord launch kit
f2d4da1	01 Oct 2026 15:37	Ibrahim Sudheysi Ali	Merge CAMR engine, evaluation harness, findings and CAMR Personal app
08062d3	01 Oct 2026 13:34	Satasdin	Android prototype (parked): Kotlin port of the CAMR engine with a Compose UI
48505a0	01 Oct 2026 13:34	Satasdin	CAMR Personal 0.2: custom chat UI, Ollama embeddings, desktop downloads
2f278c3	01 Oct 2026 13:38	Satasdin	App screenshots (dark/light/mobile), user guide for the downloads, F13 n=200 results, Chapter 5 updates
1775be6	01 Oct 2026 16:42	Ibrahim Sudheysi Ali	Merge CAMR Personal 0.2: new chat UI, desktop downloads, n=200 confirmation
1aa5b23	01 Oct 2026 13:43	Satasdin	Release workflow: publish a release from a manual dispatch; reject empty chat messages
99a23a4	01 Oct 2026 13:47	Satasdin	Chapter 5: before/after data sample, model selection protocol, error analysis, application tests, CI evidence, desktop-to-mobile progress; report PDF and DOCX
0ecac1b	01 Oct 2026 14:09	Satasdin	Report: architecture/pipeline/flow diagrams, model rankings, n=200 and error charts, test screenshots, reproduction steps; Chapter 6 builder
83cb804	01 Oct 2026 14:10	Satasdin	Chapters 5 and 6: final build with both gating calibrations

Commit	Date	Author	Change
aa77465	01 Oct 2026 14:32	Satasdin	App speed: model warm-up with chat options (D-10), time to first word, query-embedding cache; latency benchmark; Chapter 5/6 restructure (alignment with Chapter 1, Android to future work); viva Q&A
a749573	01 Oct 2026 14:52	Satasdin	App latency results (F14): warm-up 2.7-3.4x faster first answer, query cache, memory-size lookup; Chapters 5 and 6 rebuilt
23af328	01 Oct 2026 14:52	Satasdin	Track the app latency results; drop the LibreOffice profile from the repo
1af615d	01 Oct 2026 17:53	Ibrahim Sudheysi Ali	Merge: app speed, latency results, Chapters 5 and 6, viva Q&A
51e573a	01 Oct 2026 17:24	Satasdin	CAMR Personal 0.2.1: background model warm-up, time to first word, query-embedding cache
6fc811b	01 Oct 2026 20:29	Ibrahim Sudheysi Ali	Merge: CAMR Personal 0.2.1
432db5a	01 Oct 2026 17:52	Satasdin	Chapter 5: GitHub repository, pull request, CI run, release v0.2.1 and commit history screenshots
6399dc6	01 Oct 2026 20:52	Ibrahim Sudheysi Ali	Merge: Chapter 5 GitHub evidence screenshots
e937ee5	13 Oct 2026 08:54	Satasdin	Chapter 5 rework: equations (OMML), cross-validated router and read-path tuning, worked examples from logs, interface design and accessibility audit (D-11, D-12 fixed), browser tests, LoRA fine-tuning baseline and live-scenario scripts
6ff0685	13 Oct 2026 08:55	Satasdin	Live app scenarios (in progress): bridging and arithmetic, with and without memory
95b4c9d	13 Oct 2026 08:56	Satasdin	Live app scenarios (in progress): fact update and context-window recall
5cc3112	13 Oct 2026 08:57	Satasdin	Fix D-13 (over-refusal of general questions) and D-14 (questions stored as chat memory), found by the live scenarios; keep the first run as before-fix evidence
3ca2bd5	13 Oct 2026 08:58	Satasdin	Scenarios: log whether each recalled memory came by bridging; report the before-fix run
e180175	13 Oct 2026 08:58	Satasdin	Viva Q&A: tuning, interface testing and live scenarios
d87871e	13 Oct 2026 10:56	Satasdin	Fine-tuning baseline: save progress after every epoch; resume interrupted configurations
fccd03c	13 Oct 2026 10:57	Satasdin	Chapter 5: describe the fine-tuning grid as run
c5e51a8	13 Oct 2026 11:29	Satasdin	CAMR vs RAG: formal definition, side-by-side pipeline diagram, comparison table, and a head-to-head script (retrieval and answers, answer in and not in memory)
3c89eca	13 Oct 2026 12:26	Satasdin	RAG vs CAMR retrieval results: 2Wiki 78.4% vs 34-39% for top-5/10 RAG at fewer tokens; HotpotQA about equal
dbed5e7	13 Oct 2026 12:26	Satasdin	Findings F17 and viva answer: CAMR vs standard RAG
43fce27	13 Oct 2026 12:33	Satasdin	Entity-anchored gate (opt-in): abstain when the question names an entity no eligible note mentions; RAG comparison gains a validation split and the gate as a method
880ffa8	13 Oct 2026 12:39	Satasdin	RAG vs CAMR answer-level results (0.5B and 3B)
8e91ae6	13 Oct 2026 12:39	Satasdin	Chapter 5: entity-gate rows and validation table in the RAG comparison
7f9abae	13 Oct 2026 13:04	Satasdin	Entity-anchored gate results: validation (both models) and test (0.5B)
3877db6	13 Oct 2026 13:08	Satasdin	Fix D-15: entity names now match through possessives; keep pre-fix gate results
426fe36	13 Oct 2026 13:08	Satasdin	Chapter 5: gate cost text follows the post-fix results

Commit	Date	Author	Change
7afe48c	13 Oct 2026 13:09	Satasdin	RAG vs CAMR answers: 3B test run with the gate (pre-fix, also kept in answers_before_D15.json)
8095842	13 Oct 2026 13:10	Satasdin	Entity gate after D-15 fix: validation (0.5B)
62eb953	13 Oct 2026 13:12	Satasdin	Entity gate after D-15 fix: validation (3B)
c4421f5	13 Oct 2026 13:13	Satasdin	Entity gate after D-15 fix: test (0.5B)
cf89226	13 Oct 2026 13:14	Satasdin	Entity gate after D-15 fix: test (3B)
2b8b3ba	13 Oct 2026 13:16	Satasdin	Entity gate on multi-hop retrieval: HotpotQA -1.2 pts (2.2% abstained), 2Wiki unchanged
a701f53	13 Oct 2026 13:16	Satasdin	CAMR Personal: enable the entity-anchored gate (validated: no cost when the answer is stored, far fewer wrong answers taken from unrelated notes)
4487bcd	13 Oct 2026 13:17	Satasdin	Chapters 5-6 and findings: entity gate equation, RAG comparison at answer level, contributions and recommendations updated
222b5b2	13 Oct 2026 14:09	Satasdin	Viva Q&A: answer-level RAG comparison and the entity gate
05faa54	13 Oct 2026 14:21	Satasdin	LoRA tuning grid complete (3 configurations)
12099d1	13 Oct 2026 14:48	Satasdin	Fine-tuning vs memory results: LoRA 13.3% vs CAMR 71.7% on the same documents; CAMR stores them in 6 s, LoRA search took ~3.5 h; no GSM8K forgetting
452f811	13 Oct 2026 14:48	Satasdin	Findings F18 and Chapter 6: fine-tuning compared with memory
d0dc3e8	13 Oct 2026 14:49	Satasdin	Scenario retest after D-13/D-14 fixes and the entity gate (in progress)
05042f8	13 Oct 2026 14:59	Satasdin	Fix D-16: chat prompt chosen on a development set and confirmed on two held-out sets (21/21 on both models); scenario retest that exposed it
20b97eb	13 Oct 2026 14:59	Satasdin	Chapter 5: three scenario runs and the prompt selection table; final scenario run in progress
eb6617b	13 Oct 2026 14:59	Satasdin	Final scenario run: bridge and arithmetic
5ff19d9	13 Oct 2026 15:00	Satasdin	Final scenario run: fact update and context-window recall (all general questions answered, allergy recalled)
56ef111	13 Oct 2026 15:01	Satasdin	Final scenario run complete (5/5 with memory); scenario findings written from the logs
552a140	13 Oct 2026 15:07	Satasdin	Chapter 5/6 final build: equation glyph fixes (conditional bar, hidden empty limits, upright max/min), fine-tuning curve axis from zero, test count 115
f890bd0	13 Oct 2026 15:08	Satasdin	Lint fixes in the report builders (no change to chapter content)
29d200b	13 Oct 2026 18:09	Ibrahim Sudheysi Ali	Merge: Chapter 5 redo (equations, CAMR vs RAG, tuning, LoRA baseline, interface testing, live scenarios)
0657896	13 Oct 2026 16:22	Satasdin	Engine: eager learning from the web, knowledge packs, sleep (learned index, query adapter, tidy, curiosity)
21ea2d8	13 Oct 2026 16:22	Satasdin	CAMR Personal 0.3: looks up what it doesn't know, asks to be taught personal facts, Learning panel
2a2ba55	13 Oct 2026 16:33	Satasdin	Learning experiments and write-up scaffolding
f7f31cf	13 Oct 2026 16:38	Satasdin	Learning stream: ambiguous names ranked by embedding similarity (chosen on 60 dev questions: answer in fetched text 55% vs 35%); first results
5e716b6	13 Oct 2026 17:01	Satasdin	Prompt F: eager but answer-first (D-19: prompt E made the 3B model lead with 'I don't know that yet' although the notes held the answer, on the development set)
54ef92f	13 Oct 2026 17:01	Satasdin	Prompt evaluation: D and E on development, held-out and fresh sets (E regresses one development case: D-19)
9dedd67	13 Oct 2026 17:28	Satasdin	prompt_eval: prompt F renamed (its name held commas, which --only splits on)
b617bec	13 Oct 2026 18:12	Satasdin	Closed-book baselines on the learning stream: 0.5B 10%, 1.5B 11%,

Commit	Date	Author	Change
			3B 19%; 0.5B on reworded questions 10.5%
6abf8f6	13 Oct 2026 18:25	Satasdin	Learning stream, pass 1 (online): a frozen 0.5B model with eager learning answers 43.5% of 200 new PopQA questions (closed-book: 0.5B 10%, 1.5B 11%, 3B 19%); 333 notes, 2.65 MB
a1e67a0	13 Oct 2026 18:27	Satasdin	Learning figures: learning curve against closed-book models; accuracy-vs-RAM, token-memory and index-scaling charts (filled as results arrive)
845e4b9	13 Oct 2026 18:27	Satasdin	Accuracy-vs-RAM chart (first version; redrawn after the clean RAM measurement)
a8c4542	13 Oct 2026 18:42	Satasdin	Harvesting keeps pages about the topic (D-20: the first packs drifted, e.g. 'Solar System' to actors from a film of that name, and 'Kenya' lacked its own article)
0c19c8b	13 Oct 2026 18:50	Satasdin	Fetch: send the CAMR User-Agent (D-21) and read article leads from Wikipedia pages when its API is rate limited
53d393f	13 Oct 2026 18:50	Satasdin	eager_learning: --tag for a separate run (the learning pass is repeated after the D-21 fix, the first run kept)
ff8bc63	13 Oct 2026 19:22	Satasdin	CAMR Personal uses prompt F: 6/6, 7/7, 8/8 on development, held-out and fresh sets with both models, and asks to be taught on 9 of 10 unknown personal facts (D-19 fixed)
a4c18ab	13 Oct 2026 19:39	Satasdin	3B closed-book on the reworded questions
2ee37d3	13 Oct 2026 19:53	Satasdin	Learning stream, pass 2 (offline, reworded): what the 0.5B model learned online is kept; the 3B model reads the same memory
17a4bff	13 Oct 2026 20:20	Satasdin	Bundled knowledge packs (harvested after the D-20/D-21 fixes, nomic-embed-text vectors): Kenya 24 notes, Solar System 25, Photosynthesis 18; Wikipedia text, CC BY-SA 4.0
3d69b71	13 Oct 2026 20:23	Satasdin	Token memory (kNN-LM) beside the frozen 0.5B model: 13.3% held-out, like LoRA; CAMR 75.0%; together 75.0%. Chapter 5: learning tests (TC-43 to TC-58), defects D-17 to D-21, results sections for learning, token memory and sleep
79a86a2	13 Oct 2026 20:23	Satasdin	Accuracy-vs-RAM chart redrawn with the retention results
1f60f0a	13 Oct 2026 20:23	Satasdin	scenario_screens: resolve output paths (relative --out crashed)
2140eb0	13 Oct 2026 20:24	Satasdin	Scenario capture in progress: first screenshot (learning online off)
8372e59	13 Oct 2026 20:27	Satasdin	Live learning scenarios (llama3.2:3b, all four correct with memory) and the fixes they exposed
c542194	13 Oct 2026 20:29	Satasdin	Accessibility audit of the 0.3 interface with the Learning tab open: 0 axe-core violations after D-23; new chip colours pass AA; scenarios recaptured with the fixed recall header
bee5635	13 Oct 2026 20:32	Satasdin	Chapter 5: live learning scenarios section; kNN-LM equation split in two and the eager rule shortened to fit; external-weights diagram drawn top to bottom
ede6a10	13 Oct 2026 20:32	Satasdin	Findings F19-F23: eager learning stream, prompt F, token memory, sleep, CAMR Personal 0.3 in use
8c10eb9	13 Oct 2026 20:33	Satasdin	Chapter 6: a model that learns as it goes (conclusion), sixth contribution, recommendations 10-11, future works 9-10
d2f71c3	13 Oct 2026 20:34	Satasdin	Learned index measured: no gain at 38k notes (exact 0.71 ms, 63.7% full support; index loses evidence), 2.2x and 1.6x faster with ~13-31% smaller process at 150k and 400k notes, at 4.9 and 2.9 points of recall; sleep now indexes only from 100,000 notes
7a2b7f8	13 Oct 2026 20:35	Satasdin	Sleep on the learning stream's own experience: the adapter's held-out check rejected it (recall@1 83.3% -> 77.8%), so retrieval stayed unchanged
16c79a4	13 Oct 2026 20:35	Satasdin	Viva Q&A section J: learning beyond what it was taught (CAMR 0.3)
5104440	13 Oct 2026 20:35	Satasdin	APP.md for 0.3: lookups, Teach me, Learning tab, packs, sleep, and what lookups send (with how to turn them off)
29ac58d	13 Oct 2026 20:51	Satasdin	Learning stream after D-21 (first 138 questions): twice the pages per

Commit	Date	Author	Change
			lookup, the same answer coverage, 6.5 points lower accuracy; add devtune to choose pages per lookup and budget on the development questions
168a77e	13 Oct 2026 21:54	Satasdin	20B closed-book baseline: 117 of 200 questions (17.9% so far; resumes after the development tuning)
1942b95	13 Oct 2026 22:14	Satasdin	Learning stream after D-21, complete: 37.5% (first run 43.5%); every lookup now finds pages (0 empty vs 9) and 1.9 pages per question vs 1.07, but the answer is in the fetched text no more often (52.5% vs 52%): more text, more distraction for the 0.5B reader
fede586	13 Oct 2026 22:52	Satasdin	devtune: resumable per setting; five of six settings measured (3 pages at 128 tokens best on development so far: 36.7%)
214ca5b	13 Oct 2026 23:30	Satasdin	20B closed-book 20.5% (13.2 GB) against the 0.5B model with eager learning 43.5% (0.7 GB); clean per-model RAM; charts and texts updated
2039f9e	13 Oct 2026 23:33	Satasdin	Development tuning complete: 3 pages per lookup at a 128-token budget (36.7%; 3 at 512: 33.3%; 1-2 pages: 15-20%)
8638446	13 Oct 2026 23:34	Satasdin	Chapter 5: development tuning of pages per lookup and budget, and the tuned run
1f5e955	13 Oct 2026 23:40	Satasdin	Tuned run (corrected fetcher, 3 pages at 128 tokens): 43.0% online, 42.5% offline reworded (0.5B); 77 context tokens per answer instead of 260
d56ba2a	13 Oct 2026 23:41	Satasdin	Findings: tuned run after D-21
9eb291a	13 Oct 2026 23:46	Satasdin	Tuned run retention: 3B reading the tuned memory offline

The work followed GitHub flow. Each round of work was developed on one branch and merged through a pull request whose description summarises the change and its test plan: Satasdin/CAMR#2 (engine, harness, findings), Satasdin/CAMR#3 (CAMR Personal 0.2), Satasdin/CAMR#4 (application speed, latency results and report) and Satasdin/CAMR#5 (CAMR Personal 0.2.1). Machines B–E pushed their results to their own branches, which were merged into the results database rather than into the code. Feedback from users is collected through a GitHub issue form (`.github/ISSUE_TEMPLATE/feedback.yml`). The code is released under the MIT licence. Continuous integration (`.github/workflows/release-app.yml`) runs the application tests and builds and launch-tests the downloadable application on six platforms for every change to the application. Table 5.22 shows the run for the merged version. Published releases (v0.2.0, then v0.2.1 with the speed changes) attach the six downloads.

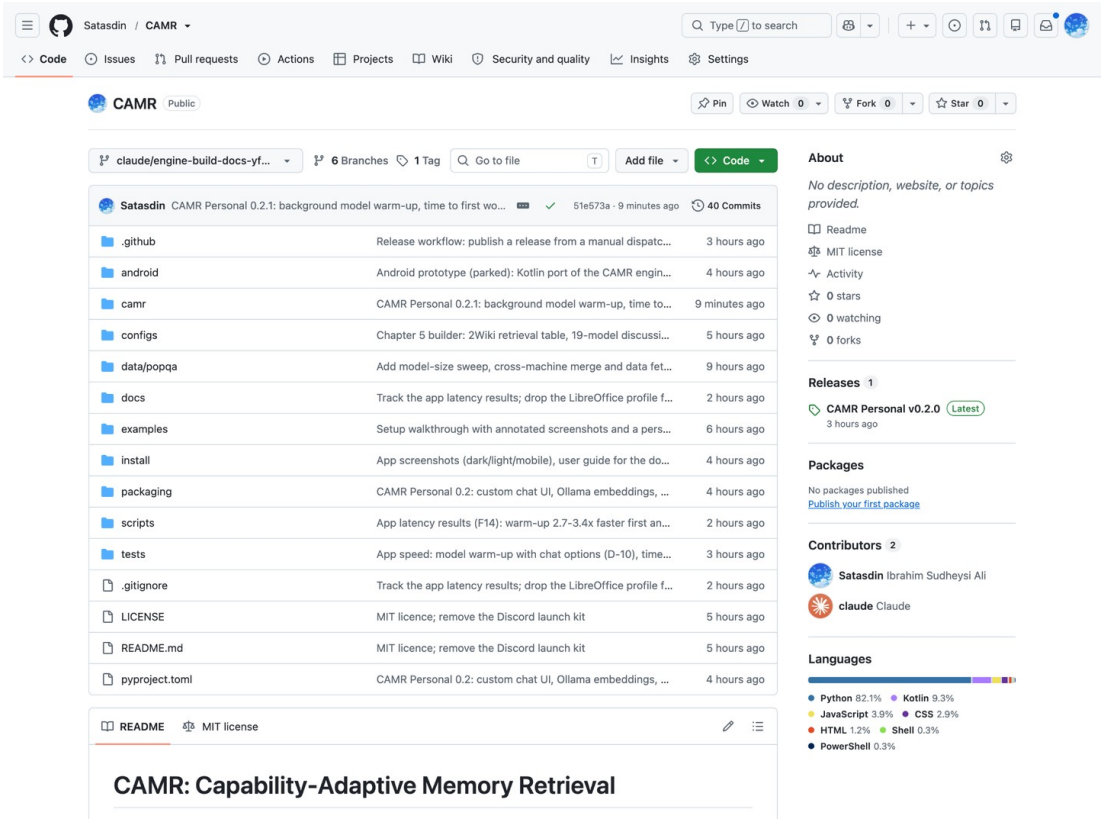


Figure 5.40: Project repository on GitHub (README, MIT licence, releases, contributors) (screenshot)

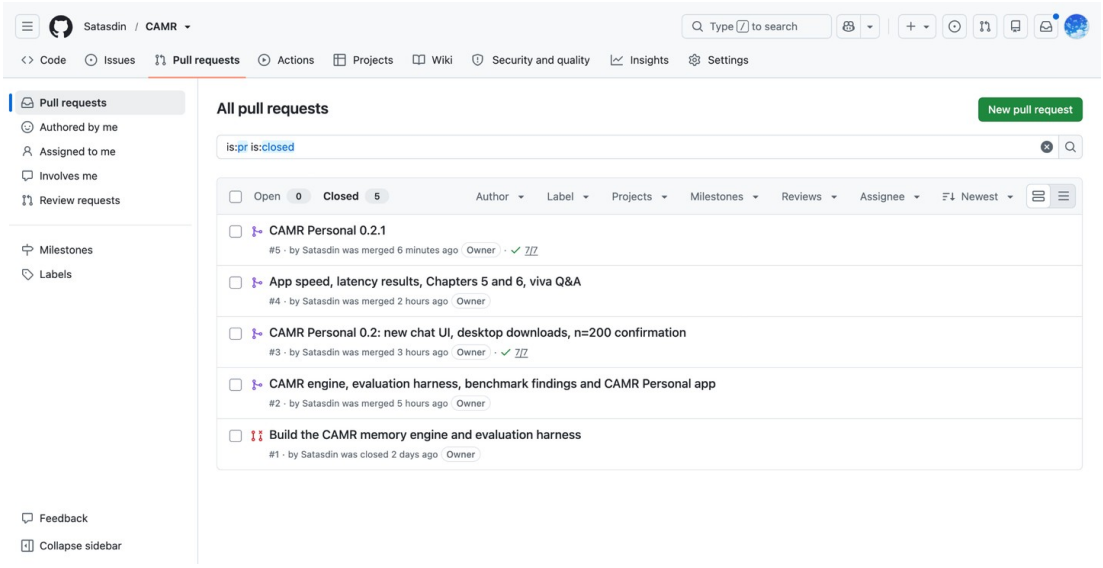


Figure 5.41: Pull requests used to merge each round of work (screenshot)

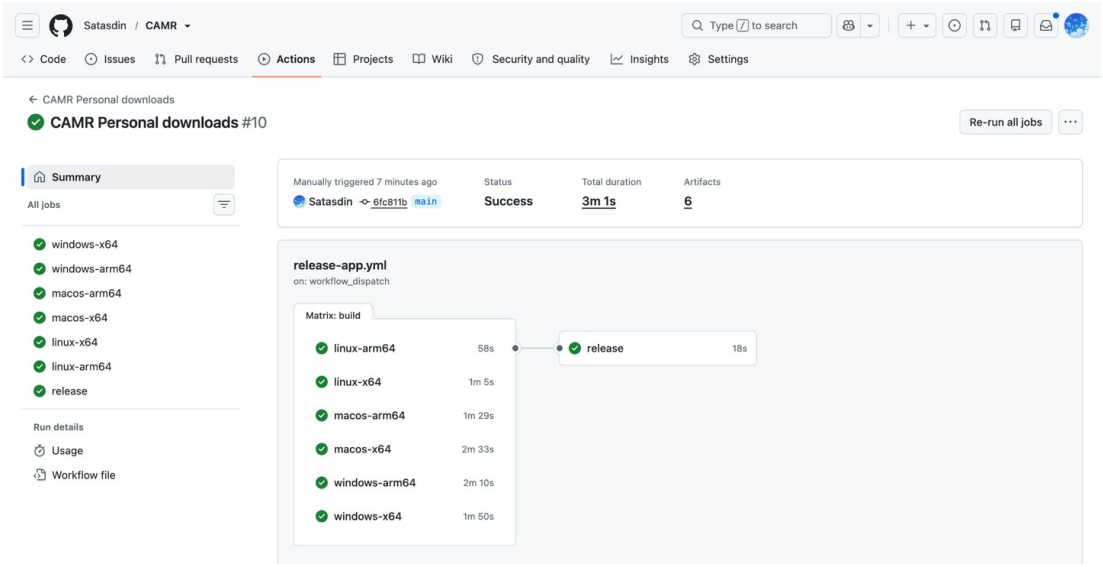


Figure 5.42: Continuous-integration run: tests, build and launch test on six platforms (screenshot)

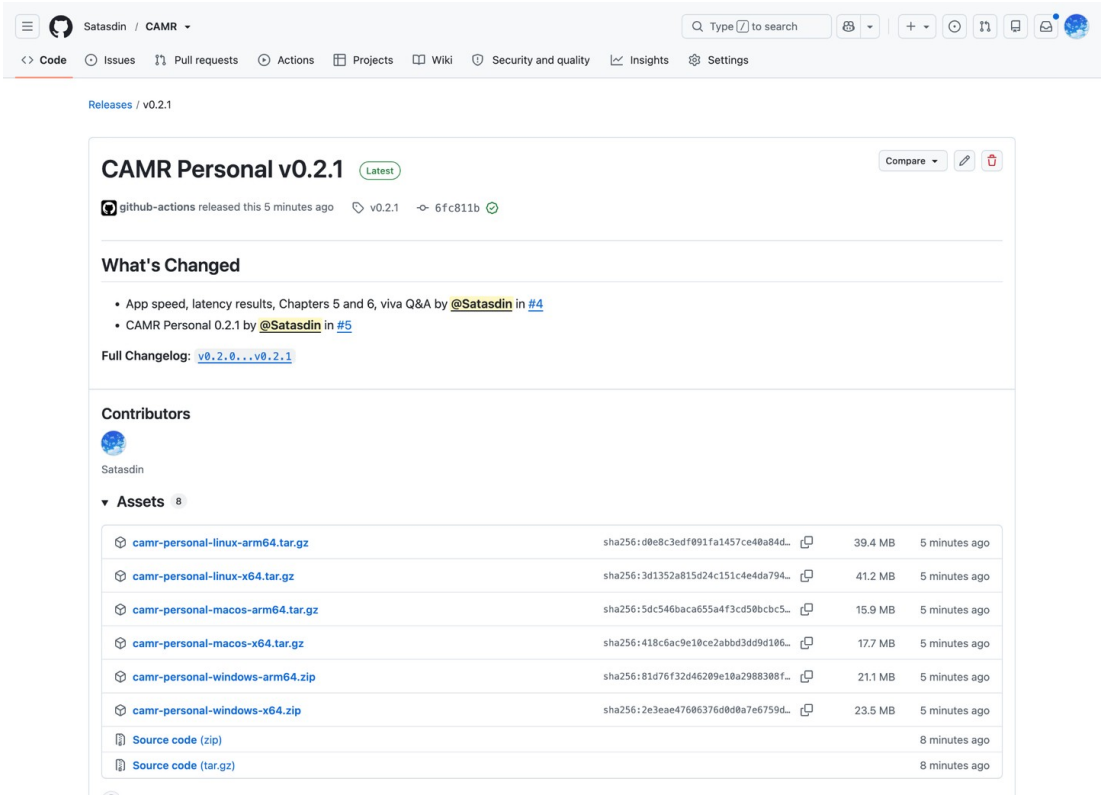


Figure 5.43: Release v0.2.1 with the six downloadable files and their checksums (screenshot)

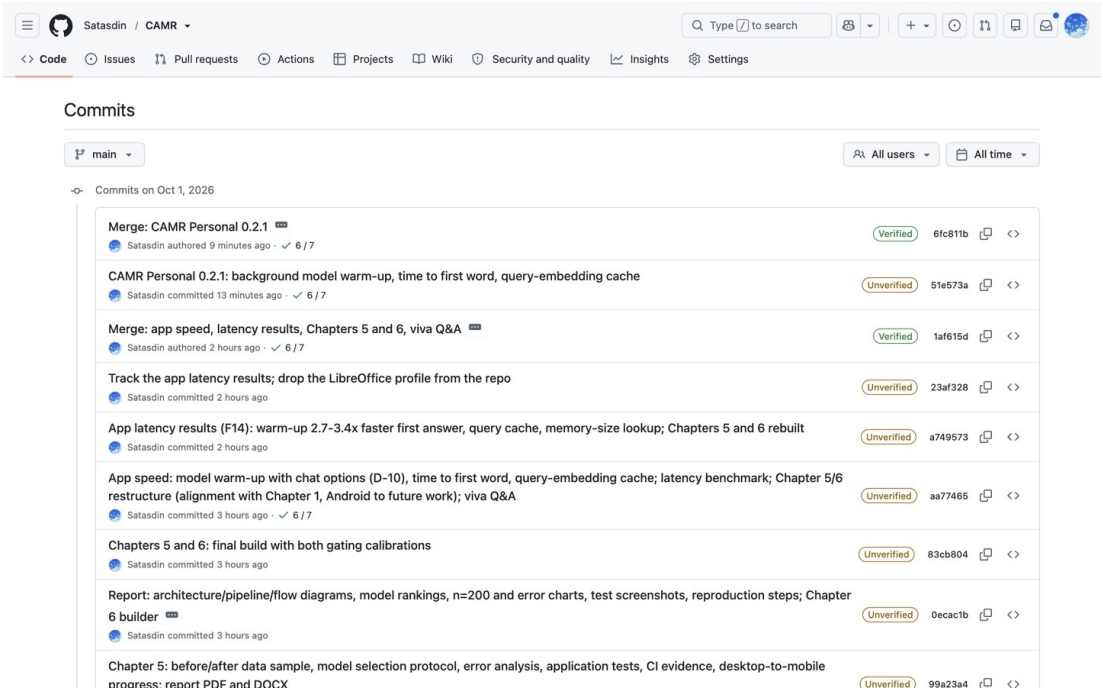


Figure 5.44: Commit history on the main branch (screenshot)

Table 5.22: Continuous-integration evidence: build and launch test per platform (GitHub Actions run 36899712846, release v0.2.1)

Target	Runner	Tests	Build	Launch test	Result
Windows x64	windows-2025	pass	pass	pass	success
Windows ARM64	windows-11-arm	pass	pass	pass	success
macOS Apple Silicon	macos-15	pass	pass	pass	success
macOS Intel	macos-15-intel	pass	pass	pass	success
Linux x64	ubuntu-24.04	pass	pass	pass	success
Linux ARM64	ubuntu-24.04-arm	pass	pass	pass	success

5.7 Author Oversight and Use of AI-Assisted Tools

Claude Code, an AI coding assistant provided by Anthropic, was used throughout the implementation and evaluation stages of this project. Under the student's direction it assisted with writing the engine and harness code and its tests, running the benchmark experiments on the cloud machines, producing the figures and screenshots, and drafting this chapter from the logged results. The student defined the research problem, objectives, requirements and design (Chapters 1–4), supplied the cloud model access, and decided which experiments were run, including the comparison with cloud models, the wider model-size sweep, the learned escalation policy and the use of real rather than synthetic benchmark data.

The student reviewed the intermediate results and screenshots as the work progressed and redirected it where it fell short; most importantly, the student required that this chapter report only experiments that had actually been run and benchmarked with real models, and an earlier draft written before the results existed was discarded for that reason. The AI-assisted output was checked through the automated test suite (148 tests), the per-query logs, the read-only inspector, and the findings log, which names the results file behind every reported number, so that each claim can be traced and re-run. The student made the final decisions and takes full responsibility for the submitted work.

5.8 Evaluation Results, Discussion and Limitations

The results are organised by research objective. Source files are in `results/` and every table is reproduced in `docs/FINDINGS.md`.

5.8.1 Retrieval Quality (Objective ii)

Before any language model was involved, the read path was evaluated on 500 HotpotQA and 500 2WikiMultiHopQA questions against one store of 38,054 notes built from 23,524 real paragraphs (170 MB). The measure was whether *both* gold supporting paragraphs reached the context, because a multi-hop question cannot be answered from one of them.

Table 5.23: Full-support recall and tokens supplied, 500 questions each (results/retrieval_eval/tables/retrieval_eval.json)

Read path (512 tokens unless noted)	**2Wiki: both gold**	**Tokens**	**HotpotQA: both gold**	**Tokens**
Similarity only (baseline)	35.8%	471	70.8%	470
+ recency and importance (Eq. 4.1)	34.6%	470	65.4%	467
+ gating	33.4%	408	64.6%	384
+ entity bridging, no gating	75.2%	472	70.0%	470
Similarity + gating + bridging	**78.4%**	**439**	**70.8%**	**420**
Baseline @ 1,024 tokens	41.4%	980	79.8%	978
Similarity + gating + bridging @ 1,024	**84.0%**	**821**	**83.0%**	**728**

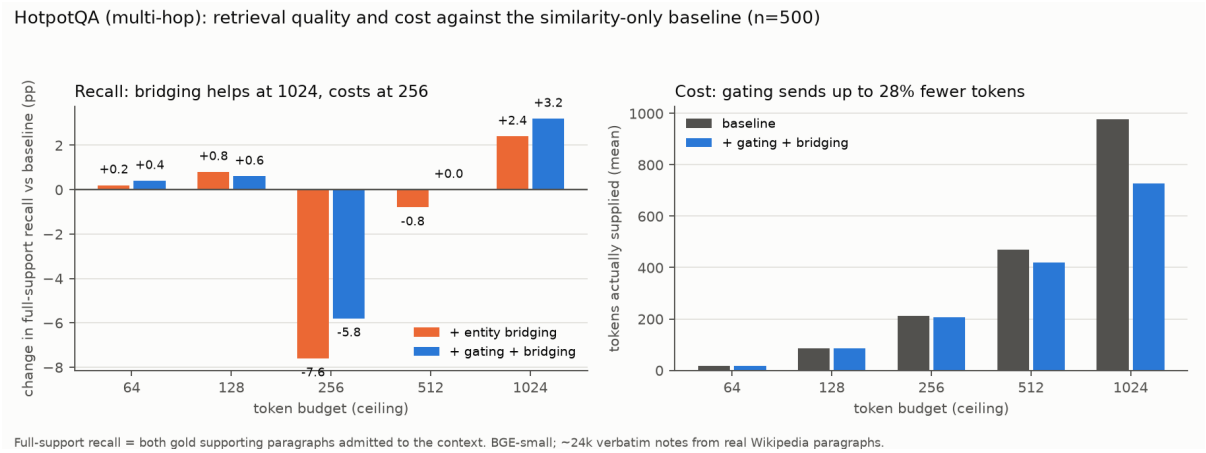


Figure 5.45: HotpotQA: change in full-support recall and tokens supplied relative to the baseline

Entity bridging was the decisive mechanism on 2WikiMultiHopQA: full-support recall rose from 35.8% to 78.4% at the same budget, with 7% fewer tokens. Its questions are compositional (for example, *who is the father of the director of film X?*). The second paragraph shares almost no words with the question, so similarity search cannot find it, but the first paragraph names it, and that is the hop bridging follows. On HotpotQA the gain depended on the budget: none at 512 tokens, with 11% fewer tokens, and +3.2 points at 1,024 tokens, with 26% fewer tokens. On both benchmarks the composite score of Equation 4.1 lowered recall. On independent questions, recency and importance carry no signal about the current question, which confirms the risk anticipated in section 4.4.7.

5.8.2 Tuning the Read Path on Validation Data

The margin δ and the budget B were fixed by design in the first study. To check them properly, they were tuned on a **validation** half of the 500 HotpotQA and 500 2WikiMultiHopQA retrieval questions (500 questions) and the chosen setting was scored once on the other, **held-out** half. Because a larger budget always recovers more evidence, the objective charges for tokens at the same rate μ as the learned router's reward (Equation 5.15):

$$J(\delta, B) = \mathrm{SR}(\delta, B) - \mu \frac{\overline{T}(\delta, B)}{1000}, \mu = 0.1 \tag{5.30}$$

Figure 5.46 shows the 24-point grid. Recall rises with the budget and with a wider margin, and so does the token bill; the objective balances them.

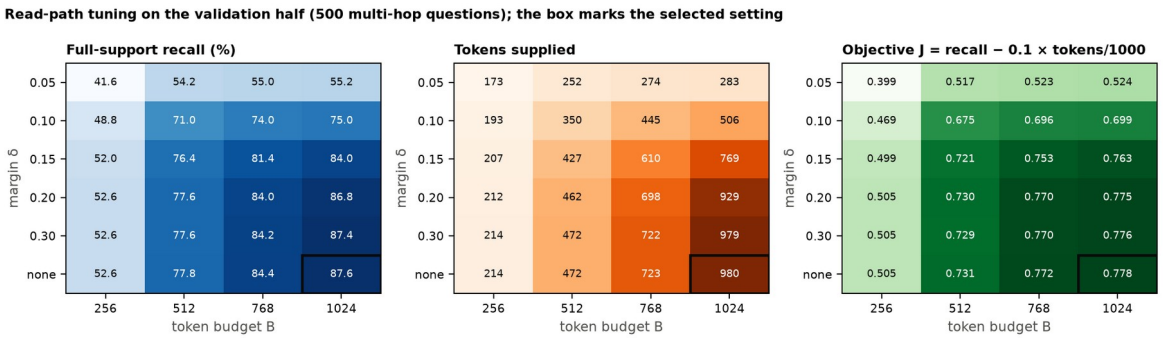


Figure 5.46: Read-path grid on the validation half: full-support recall, tokens supplied and the objective J; the box marks the selected setting

Table 5.24: Read path on the held-out half: tuned setting against the study's default

Setting	**δ**	**B**	**Full-support recall**	**Tokens**	**J**
Selected on validation	none	1024	85.6%	979	0.758
Study default	0.15	512	72.8%	431	0.685

Validation selected no margin and B = 1024: at a price of μ = 0.1 per 1,000 tokens, more evidence is worth its tokens. On the held-out half it raised full-support recall from 72.8% to 85.6%, but supplied 979 tokens per question instead of 431. The choice is sensitive to the price of a token: the study's setting wins as soon as a token costs more than μ ≈ 0.20 per 1,000, and the margin's main effect is to save tokens (at B = 1,024, δ = 0.15 supplies about a fifth fewer tokens for a few points of recall). More importantly, this objective measures *retrieval*, not answers. The end-to-end budget sweep (section 5.8.8) showed that the 0.5B model answers *worse* when given more notes, because it is distracted by them. The read path should therefore be tuned to the reader: a large budget with no margin suits strong readers and multi-hop questions, where missing evidence is the binding constraint; the study's small budget and margin suit tiny models. This is exactly the per-question trade-off the learned router makes, and the reason a single fixed setting cannot be best.

The abstain threshold τ is different: on these questions every memory is on-topic, so a higher τ can only lose recall. It was therefore set on separate data, the similarity of each question to an *off-topic* paragraph, as the 90th percentile of that distribution. Figure 5.47 shows the trade-off for both embedders: at the chosen τ almost every relevant note is kept, and about 90% of off-topic notes are refused.

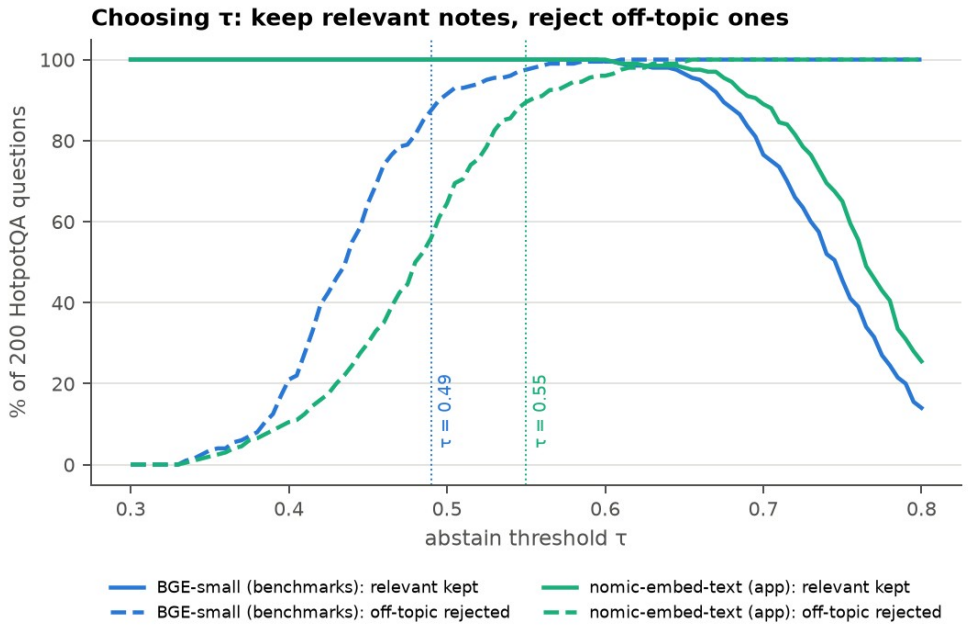


Figure 5.47: The abstain threshold τ: relevant notes kept against off-topic notes rejected on 200 HotpotQA questions, for the benchmark and the application embedders

5.8.3 How Much CAMR Adds to Standard RAG

To show the difference rather than describe it, CAMR was run head to head with a textbook retrieve-then-read RAG baseline (Equation 5.9) that uses the **same** embedder, store and prompt template. Only the choice of notes differs. `scripts/rag_vs_camr.py` produces both tables.

Table 5.25: Retrieval, 500 questions per benchmark: standard RAG against CAMR on the same 38,054-note store

Benchmark	Method	Both gold paragraphs in context	Context precision	Notes	Tokens
HotpotQA	RAG top-3	56.2%	57.0%	3.0	231
HotpotQA	RAG top-5	68.2%	39.2%	5.0	391
HotpotQA	RAG top-10	78.6%	21.9%	10.0	800
HotpotQA	CAMR	70.8%	39.1%	5.6	420
HotpotQA	CAMR + entity gate	69.6%	39.3%	5.5	410
2Wiki	RAG top-3	28.0%	60.3%	3.0	216
2Wiki	RAG top-5	34.2%	42.0%	5.0	373
2Wiki	RAG top-10	39.2%	23.4%	10.0	767
2Wiki	CAMR	78.4%	51.5%	6.0	439
2Wiki	CAMR + entity gate	78.4%	51.5%	6.0	439

The two benchmarks separate where CAMR adds something and where it does not. On **2WikiMultiHopQA**, whose second paragraph is reached only through a name in the first, standard RAG almost always found *one* gold paragraph (100.0% at top-5) but rarely both: 28.0%, 34.2% and 39.2% at k = 3, 5 and 10. Doubling k to 10 doubled the tokens (767) and added only 5 points, because more of the same kind of retrieval finds more first-hop paragraphs, not the second hop. CAMR put both in the context for 78.4% of questions with 439 tokens, twice the recall of top-10 RAG at 43% fewer tokens, and a larger share of its notes were gold (51.5% against 23.4%). On **HotpotQA** the difference was small: CAMR 70.8% with 420 tokens against top-5 RAG's 68.2% with 391, at the same precision (39.1% and 39.2%); top-10 RAG recovered more (78.6%) by sending 800 tokens. HotpotQA questions usually name both entities, so similarity alone finds both paragraphs, and on such questions CAMR's read path is essentially RAG with a token cap.

Table 5.26: Answers on 30 held-out PopQA questions: standard RAG against CAMR, with the answer in memory and with the question's own document removed

Model	Condition	Accuracy	Context words	Abstained	Wrong answer copied from the notes
qwen2.5:0.5b	Model alone	6.7%	0	–	–
qwen2.5:0.5b	RAG top-5, answer in memory	70.0%	309	0.0%	20.0%
qwen2.5:0.5b	CAMR, answer in memory	80.0%	179	0.0%	16.7%
qwen2.5:0.5b	CAMR + entity gate, answer in memory	80.0%	179	0.0%	16.7%
qwen2.5:0.5b	RAG top-5, answer NOT in memory	6.7%	311	0.0%	76.7%
qwen2.5:0.5b	CAMR, answer NOT in memory	10.0%	358	3.3%	73.3%
qwen2.5:0.5b	CAMR + entity gate, answer NOT in memory	10.0%	60	80.0%	13.3%
llama3.2:3b	Model alone	23.3%	0	–	–
llama3.2:3b	RAG top-5, answer in memory	73.3%	309	0.0%	26.7%
llama3.2:3b	CAMR, answer in memory	73.3%	179	0.0%	23.3%
llama3.2:3b	CAMR + entity gate, answer in memory	73.3%	179	0.0%	23.3%
llama3.2:3b	RAG top-5, answer NOT in memory	13.3%	311	0.0%	20.0%
llama3.2:3b	CAMR, answer NOT in memory	10.0%	358	3.3%	16.7%
llama3.2:3b	CAMR + entity gate, answer NOT in memory	20.0%	60	80.0%	6.7%

When the answer is in memory, CAMR matched or beat RAG with about 40% less text. The 0.5B model reached 80.0% with CAMR against 70.0% with RAG, reading 179 words of notes instead of 309: the extra chunks RAG always adds were distraction for a weak reader. The 3B model scored the same with both (73.3%). **When the answer is not in memory, the first version of CAMR failed exactly as RAG did.** With the question's own document

removed, the store still held notes about other films, people and albums that were as similar to the question as the missing note, so the similarity gate abstained on only 3.3% of questions. Both systems filled the prompt with unrelated notes, and the 0.5B model answered with a wrong name taken from them 76.7% (RAG) and 73.3% (CAMR) of the time. The irrelevant notes even made the 3B model worse than answering alone (23.3% alone, 13.3% with RAG, 10.0% with CAMR): they overrode facts it already knew. A similarity threshold cannot tell *a different film by a similar director* from *this film*.

Overcoming the hurdle: an entity-anchored gate. The remedy follows from the failure: if the question *names* something (*The Loudwater Mystery*, *Fahrenheit*) and none of the candidate notes mentions that name, the notes are about something else, however similar they look, and memory should be withheld. `camr.memory.entities` extracts names from the question (runs of capitalised words, quoted titles) and the engine abstains when no eligible note mentions any of them; questions that name nothing, such as *when is my dentist appointment?*, are unaffected. It needs no model call and adds no measurable time. Because the gate was designed after seeing the test questions fail, it was judged first on the 60 *validation* questions of the learning split, which played no part in its design (table below). There it changed nothing when the answer was in memory (68.3% and 75.0% for the two models, as without it), and when the answer was missing it abstained on 78.3% of questions, cutting wrong answers copied from the notes from 73.3% to 15.0% (0.5B) and from 23.3% to 6.7% (3B). On the held-out test questions the result held: abstention 80.0%, copied wrong answers 73.3% → 13.3% (0.5B) and 16.7% → 6.7% (3B), and the 3B model's accuracy without the document recovered from 10.0% to 20.0%, close to its 23.3% alone. The first run of the gate lost one answerable test question, *Who was the producer of Peck's Bad Girl?*, because the possessive was normalised differently in the question and in the note (defect D-15); the numbers here are after the fix, and the first run is kept in `results/rag_vs_camr/answers_before_D15.json`. After it, the gate cost nothing on answerable questions. On the multi-hop retrieval questions, where the evidence does name the question's entities, the gate changed full-support recall from 70.8% to 69.6% on HotpotQA and from 78.4% to 78.4% on 2Wiki (abstaining on 2.2% and 0.0% of questions). Standard RAG has no step at which this decision could be made; that is the difference between retrieving text and deciding whether a small model should read it.

Table 5.27: The entity-anchored gate on the 60 validation questions, which played no part in designing it

Model	Condition	Accuracy	Context words	Abstained	Wrong answer copied from the notes
qwen2.5:0.5b	Model alone	11.7%	0	–	–
qwen2.5:0.5b	RAG top-5, answer in memory	61.7%	302	0.0%	33.3%
qwen2.5:0.5b	CAMR, answer in memory	68.3%	164	0.0%	23.3%
qwen2.5:0.5b	CAMR + entity gate, answer in memory	68.3%	164	0.0%	23.3%
qwen2.5:0.5b	RAG top-5, answer NOT in memory	10.0%	305	0.0%	65.0%
qwen2.5:0.5b	CAMR, answer NOT in memory	10.0%	375	0.0%	73.3%
qwen2.5:0.5b	CAMR + entity gate, answer NOT in memory	13.3%	69	78.3%	15.0%
llama3.2:3b	Model alone	11.7%	0	–	–
llama3.2:3b	RAG top-5, answer in memory	71.7%	302	0.0%	20.0%
llama3.2:3b	CAMR, answer in memory	75.0%	164	0.0%	15.0%
llama3.2:3b	CAMR + entity gate, answer in memory	75.0%	164	0.0%	15.0%
llama3.2:3b	RAG top-5, answer NOT in memory	11.7%	305	0.0%	23.3%
llama3.2:3b	CAMR, answer NOT in memory	11.7%	375	0.0%	23.3%
llama3.2:3b	CAMR + entity gate, answer NOT in memory	15.0%	69	78.3%	6.7%

Earlier results in this chapter measure the remaining differences. Against the same store with a fixed budget and no gating (the *control* configuration), the full engine was 10 points more accurate on single-hop facts with 42% fewer tokens and answered 3.4 times faster (section 5.8.8). A fixed RAG budget for every question (*always 512 tokens*) reached 45.8% on the held-out policy questions; choosing per question, with escalation, reached 66.7% (section 5.8.12). And a fixed corpus cannot learn from the conversation: with chat memory switched off, the assistant could not recall an allergy mentioned eight turns earlier; with CAMR's write path it could (section 5.5.5).

5.8.4 Gap Closed and the Residual Gap (Objective iv)

Table 5.28: Accuracy by condition, pilot sample (results/pilot/tables/summary.json, gap_vs_kimi.md)

Task (n)	0.5B alone	0.5B + CAMR	gpt-oss-20b	20B + same notes	Kimi K3	Kimi K3 + CAMR
Single-hop, PopQA (30)	10.0%	**66.7%**	23.3%	76.7%	66.7%	83.3%
Multi-hop, HotpotQA (30)	6.7%	**30.0%**	30.0%	53.3%	53.3%	63.3%
Reasoning, GSM8K (12)	33.3%	16.7%	100%	100%	100%	100%

Against Kimi K3, the engine closed the **entire** single-hop gap (gap closed 1.00) and **half** of the multi-hop gap (0.50, 95% CI 0.20–0.90). It closed none of the reasoning gap (−0.25). The 0.5B model with CAMR answered single-hop questions in 1.5 s against Kimi's 7.7 s. The residual gap (20B with the same notes minus 0.5B with them) was 10.0 points on single-hop, 23.3 on multi-hop and 83.3 on reasoning: the more a task needs reading and reasoning, the more model size still buys. This answers research question iii. Memory helps knowledge-bound tasks and does not help reasoning-bound ones. The same notes also lifted the frontier model (Kimi K3 rose from 66.7% to 83.3% on PopQA), so the engine helps beside any model.

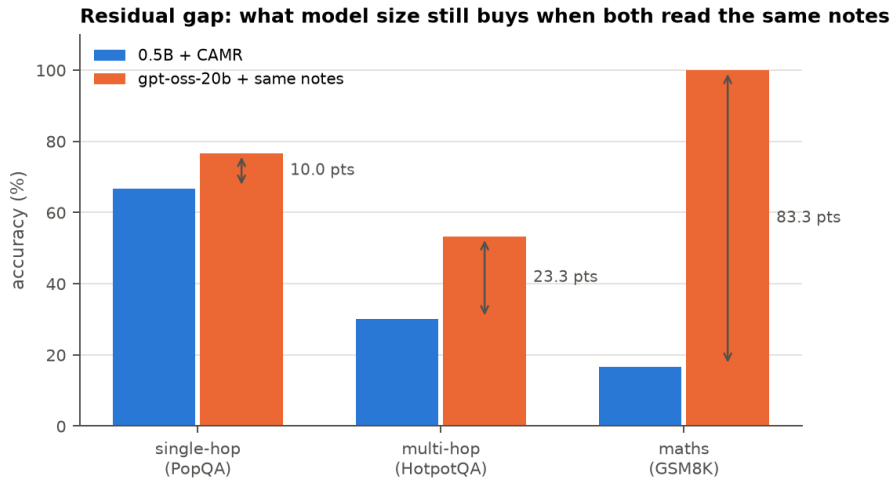


Figure 5.48: Residual gap: accuracy of the 0.5B model and of gpt-oss-20b when both read the same notes (pilot)

5.8.5 What the Models Actually Answered: Worked Examples

Aggregate accuracy hides how an answer is won or lost. This subsection follows single questions through the engine, with and without memory, using the logged prompts and answers verbatim (abridged only where marked …). The examples were chosen to show each mechanism and each failure mode, not to flatter the engine; `scripts/extract_examples.py` pulls them from the result databases.

Example 1: a long-tail fact, and an ambiguous name. *What genre is Fahrenheit?* PopQA's intended entity is the Taiwanese band *Fahrenheit*, and the accepted answers are mandopop, Mandarin popular music, mandapop.

Table 5.29: Example 1: answers to a long-tail question with and without memory (pilot logs)

Model and condition	Answer	Correct	Prompt tokens	Seconds
qwen2.5:0.5b alone	Science fiction	no	51	0.1
qwen2.5:0.5b + CAMR	Mandopop	yes	392	1.2
gpt-oss:20b alone	Science fiction.	no	89	8.9
Kimi K3 (cloud)	Action-adventure game	no	107	7.5

Without memory, each model answered for a *different* Fahrenheit: the 0.5B and 20B models chose the novel's genre, Kimi K3 the video game. With CAMR, the 0.5B model read the stored note about the band and answered correctly in 1.2 s. Memory supplied not only a missing fact but the user's intended referent, which no amount of parameters can guess. This is the personal-assistant case in miniature: *my* dentist, *my* supervisor.

Example 2: a two-hop question solved by bridging. *Jamie Harris is best known for his role as The Hook-Handed Man in a movie directed by who ?* (gold: Brad Silberling). The engine admitted 6 notes; 2 of them came only through entity bridging (*Lemony Snicket's A Series of Unfortunate Events*, similarity 0.52), reached because an admitted seed note named it. llama3.2:3b answered *Robert Rodriguez* alone and *Brad Silberling.* with CAMR (correct), in 6.9 s; Kimi K3 knew it closed-book (8.6 s).

Example 3: a two-hop question solved by bridging. *What year did Francesca Caccini's father die?* (gold: 1618). The engine admitted 6 notes; 1 of them came only through entity bridging (*La liberazione di Ruggiero*, similarity 0.46), reached because an admitted seed note named it. llama3.2:3b answered *1628* alone and *1618* with CAMR (correct), in 6.8 s; Kimi K3 knew it closed-book (5.0 s).

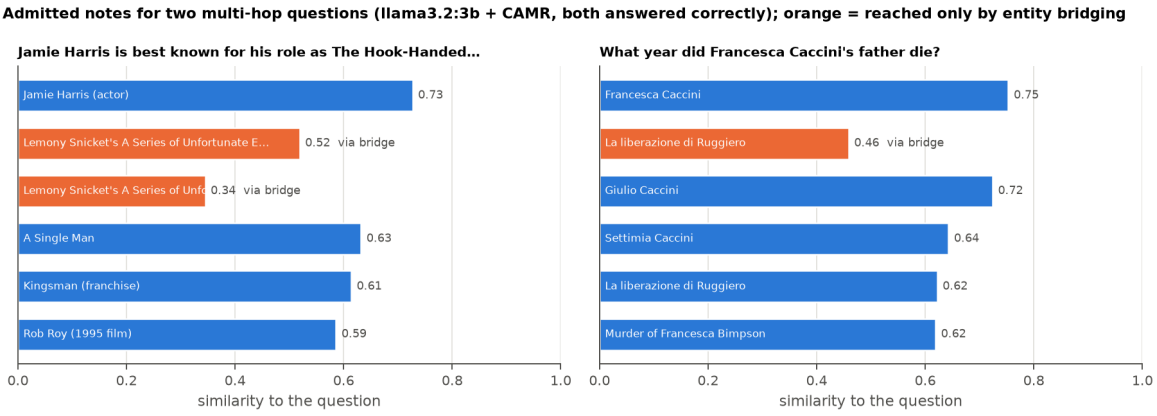


Figure 5.49: Admitted notes for the two multi-hop examples; orange notes were reached only by entity bridging (Equation 5.6)

Example 4: a maths problem where worked-example memory helps. *Three friends spent $20.25 on 3 tickets to the fair. They also spent $4.50 less on food than on the tickets. They also went on 2 different rides which costs $33 for each ride. If they agreed to split all the costs evenly, how much did each of them pay?* The gold solution is a chain of four operations:

$$\text{food} = 20.25 - 4.50 = 15.75\,,\ \text{rides} = 2 \times 33 = 66\,,\ \frac{20.25 + 15.75 + 66}{3} = \frac{102}{3} = 34 \qquad (5.31)$$

For reasoning questions CAMR retrieves solved GSM8K *training* problems instead of facts. Here it retrieved two: a state-fair ticket problem and *three friends split the cost of a movie*. Table 5.30 shows the decisive step of each answer.

Table 5.30: Example 4: the same GSM8K problem with and without worked-example memory (logged answers)

Model and condition	Opening of the answer (abridged)	Final	Correct
phi4-mini:3.8b alone	First, let's calculate the total cost of the tickets: / 3 tickets * $20.25 = $60.75 / Next, let's calculate the total cost of the food: / $4.50 less than the tickets = $2 …	47.50	no
phi4-mini:3.8b + CAMR worked examples	First, let's calculate the total cost of the tickets: / 3 tickets cost $20.25. / Next, let's calculate the total cost of the food: / They spent $4.50 less on food than on …	34	yes
qwen2.5:14b alone	First, calculate the total cost of the tickets: / \[3 \times 20.25 = 60.75 \] / Next, calculate the cost of the food: / \[20.25 - 4.50 = 15.75 \] …	47.50	no
qwen2.5:14b + CAMR worked examples	First, calculate the total cost of the tickets: / - The cost of 3 tickets is $20.25. / Next, calculate the cost of the food: / - They spent $4.50 less on food than on the …	34	yes
qwen2.5:0.5b + CAMR worked examples	#### $7.50 / - Problem: Three friends went out to watch a movie. Mitch paid for their tickets at $7 each. On the other hand, Jam paid for the 2 boxes of popcorn at $1.5 w …	$7.50	no
Kimi K3 (cloud)	**Step 1: Cost of tickets** / Tickets = $20.25 / **Step 2: Cost of food** / Food = $20.25 − $4.50 = $15.75 …	34	yes

Alone, both phi4-mini (3.8B) and qwen2.5:14b read *$20.25 on 3 tickets* as a price per ticket and computed 3 × 20.25 = 60.75, which led to 47.50. With the two worked examples in the prompt, both read it as the total and reached 34, matching Kimi K3. The 0.5B model, given the same examples, printed an answer and then began copying an example problem: the format of the examples overwhelmed it.

Example 5: where the same memory hurts. *Henry is making cookies for a local baking competition. He wants to make twice as many as he did last year. When he finishes baking, he realizes he actually bak…* The 0.5B model alone solved it (answer ending *So, Henry baked 50 c*). With worked examples it answered *#### 100* and then repeated the problem text. Across the pilot, exemplar memory moved the 0.5B model from 33.3% to 16.7% on GSM8K,

and over 19 models it helped 8, was neutral for 4 and hurt 7. This is why the learned router sends maths to a stronger model instead of adding memory (section 5.8.12).

5.8.6 Error Analysis: Where Answers Are Lost

Table 5.31 follows each question through the pipeline (from `tables/coverage.md`): is the gold answer in the store, among the retrieved candidates, in the context actually sent, and was the answer correct? The last column is accuracy *given* that the answer was in the context.

Table 5.31: Failure decomposition for the 0.5B model with the full engine (pilot)

Benchmark (n)	Answer in store	In candidates	In context	Correct	Correct when in context
PopQA (30)	93.3%	93.3%	90.0%	66.7%	74.1%
PopQA, 128-token budget (30)	93.3%	93.3%	86.7%	76.7%	84.6%
HotpotQA (30)	96.7%	93.3%	83.3%	30.0%	36.0%

Retrieval was rarely the problem. On HotpotQA the answer reached the context for 83% of questions, but the 0.5B model used it correctly only 36% of the time. Of its 21 misses, 16 (76%) had the gold answer in front of it. On PopQA a smaller budget raised correct-when-in-context from 74% to 85%: fewer notes, less distraction. Errors therefore fall into three groups, illustrated in Table 5.33: **correct** answers read from the note; **incomplete** answers that pick a related entity from the right evidence; and **unsupported** answers, where the model ignored or lacked the evidence and fell back on its own (wrong) knowledge.

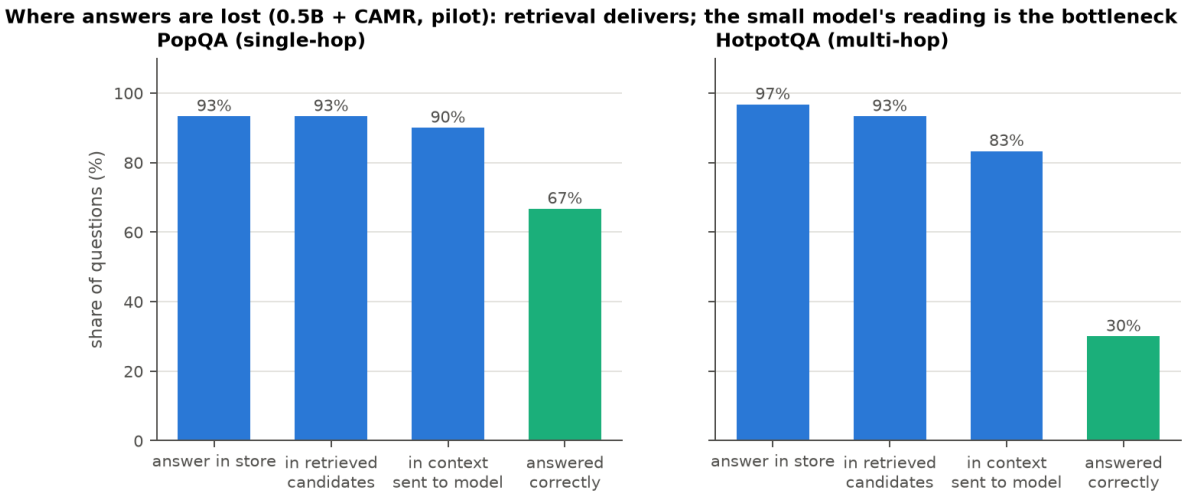


Figure 5.50: Where answers are lost for the 0.5B model with CAMR: store, candidates, context and correct answer

Table 5.32: Examples of correct, incomplete and unsupported answers (from the query logs)

Type	Question (abridged)	Gold	Answer given
Correct	Who was the screenwriter for *The Graduate*?	Calder Willingham, Buck Henry	0.5B + CAMR: Buck Henry (alone: Linda Wachowski)
Correct	McLaren MP4/11 was driven by which Finnish driver?	Mika Häkkinen	0.5B + CAMR: Mika Häkkinen (gpt-oss-20b alone: Kimi Räikkönen)
Incomplete	City with the Nusretiye Clock Tower and …?	Istanbul, Turkey	Beyoğlu (a district of Istanbul; evidence was in context)
Incomplete	Event where Tyson Gay and Rodney Martin both represented the US?	4 × 100 m relay	100 m sprint (evidence in context)
Unsupported	Who was the director of *The Loudwater Mystery*? (no memory)	Walter West	0.5B alone: John Sturges (invented)
Unsupported	When is my dentist appointment? (empty memory)	Thursday 9 October 14:30	next Monday (invented); with the note: correct

5.8.7 Knowledge Growth with Frozen Weights

The knowledge corpus was fed into one store in three stages, and the same frozen 0.5B model answered the same questions after each stage (Figure 5.51). On long-tail facts, accuracy rose from 6.7% to 16.7%, 33.3% and 66.7% as memory grew to 25%, 50% and 100%. That matches Kimi K3 and is roughly three times the 20B model. Median decode speed stayed between 44 and 63 tokens/s at every stage (Figure 5.52). The cost of memory is in prefill (270–460 extra prompt tokens) and a bounded 3–17% per-token decode slowdown, measured across the model sweep, from attending over a longer prompt.

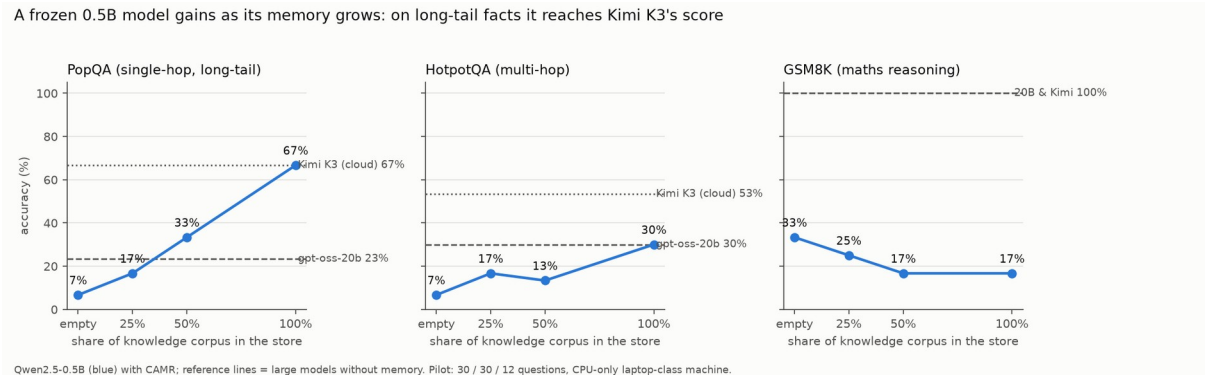


Figure 5.51: Accuracy of the frozen 0.5B model as the memory store grows

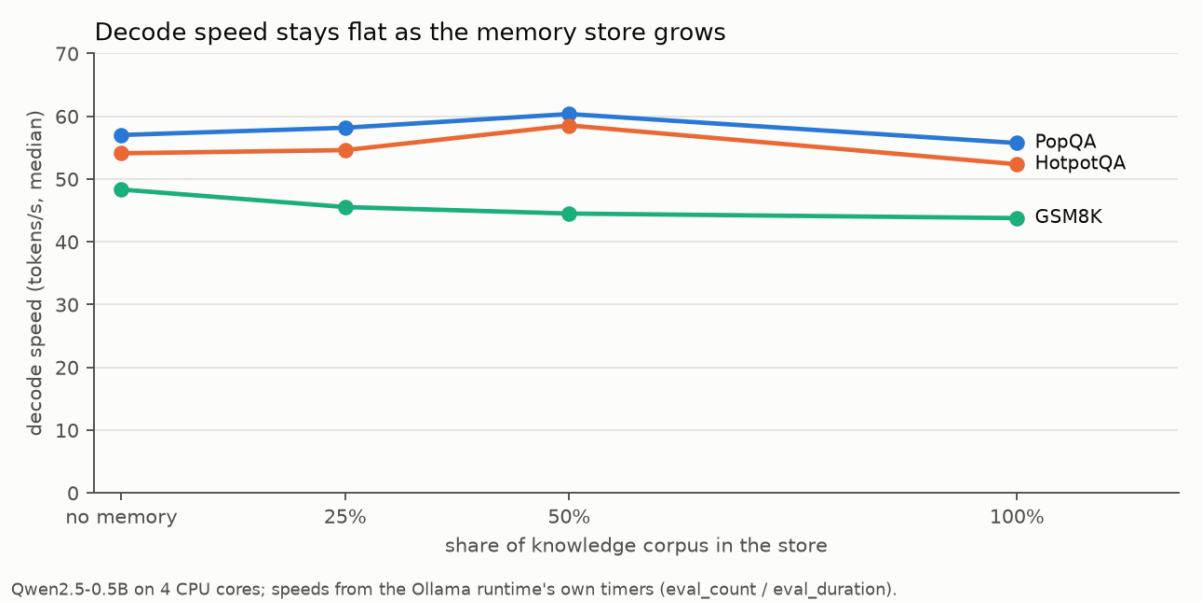


Figure 5.52: Decode speed of the frozen 0.5B model at each store size

5.8.8 Ablation and Token Budget (Objective v)

Table 5.33: Ablation and budget sweep, 0.5B model (ablation.md, budget_sweep.md)

Variant	Single-hop acc. (tokens)	Multi-hop acc. (tokens)	Reasoning acc.	Answer time (single / multi)
Control: similarity only, fixed budget	56.7% (464)	26.7% (472)	–	2,343 / 2,469 ms
Full engine: gating + bridging	66.7% (267)	30.0% (380)	–	688 / 1,480 ms
Budget 0	10.0% (0)	3.3% (0)	16.7%	–
Budget 128	**76.7% (82)**	30.0% (86)	16.7%	–
Budget 512	66.7% (267)	**33.3% (380)**	20.8%	–
Budget 1,024	66.7% (434)	26.7% (557)	16.7%	–

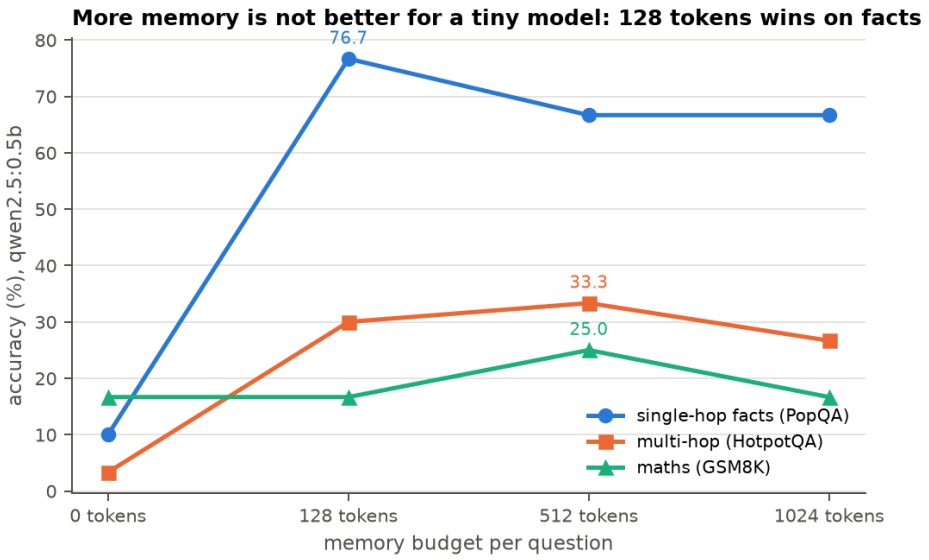


Figure 5.53: Accuracy of the 0.5B model by memory budget (pilot)

The full engine was more accurate than the control and 1.7–3.4 times faster per answer, because gating sent fewer tokens. For a 0.5B model, the budget that maximised gap closed per token was **128 tokens** for single-hop facts. Larger budgets *lowered* accuracy, because the small model was distracted by extra notes. Multi-hop questions peaked at 512 tokens, and reasoning never benefited. This answers research question iv: the best budget depends on the task type and the model, which motivated the learned policy.

5.8.9 Small Models on Their Own: the Baseline Ranking

Before memory, the 19 candidate models were ranked by their own accuracy (Figure 5.54). The ranking follows size, but loosely: the rank correlation between parameter count and mean accuracy was ρ = 0.82. gemma3:12b led (51.7% mean), followed by qwen2.5:7b (47.8%) and gemma2:9b (47.2%); qwen2.5:0.5b was last (13.9%). The models' weakness was knowledge, not arithmetic. Alone, no model knew more than 33.3% of the long-tail facts (Kimi K3 knew 66.7%), and the mean across all 19 was 17.7%. Several, however, already scored 75–92% on GSM8K. Multi-hop questions were also out of reach: the best was qwen2.5:14b at 46.7%, below Kimi K3's 53.3%. Model families differed markedly at the same size. qwen2.5:3b was 16th of 19 on its own (6.7% on facts) while llama3.2:3b was 8th, and mistral:7b was 14th, below several 1–3B models.

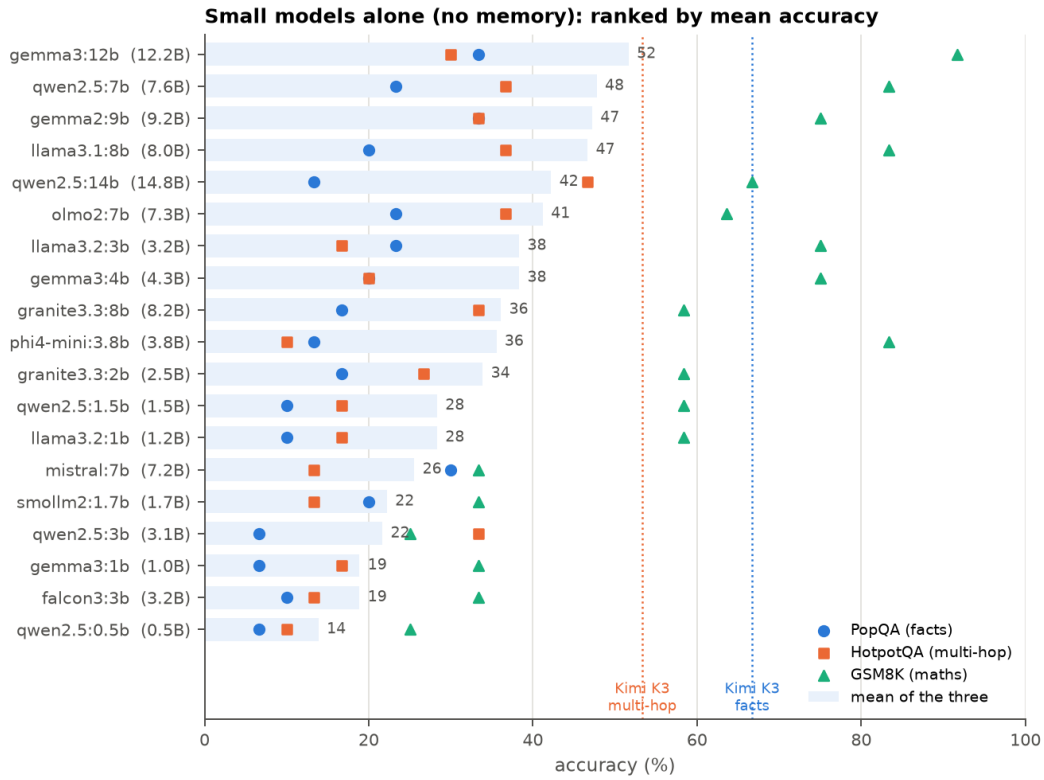


Figure 5.54: Ranking of the 19 small models alone (no memory): mean accuracy and per-benchmark scores, with Kimi K3 for reference

With CAMR the ranking changed (Figure 5.55). The size advantage on facts largely disappeared: the rank correlation between size and long-tail accuracy fell from 0.67 ($p = 0.002$) alone to 0.35 ($p = 0.14$, not significant) with memory, and mean fact accuracy rose from 17.7% to 74.2%. The models that gained most climbed the ranking: phi4-mini moved from 10th to 5th, falcon3:3b from 17th to 10th, and qwen2.5:14b from 5th to 2nd. Models with strong parametric knowledge but weaker reading of supplied evidence fell: gemma2:9b from 3rd to 8th, llama3.1:8b from 4th to 9th. Multi-hop accuracy with memory still tracked size ($\rho = 0.78$), because combining two pieces of evidence depends on the model as a reader. Figure 5.56 shows each model's gain and Figure 5.57 its speed.

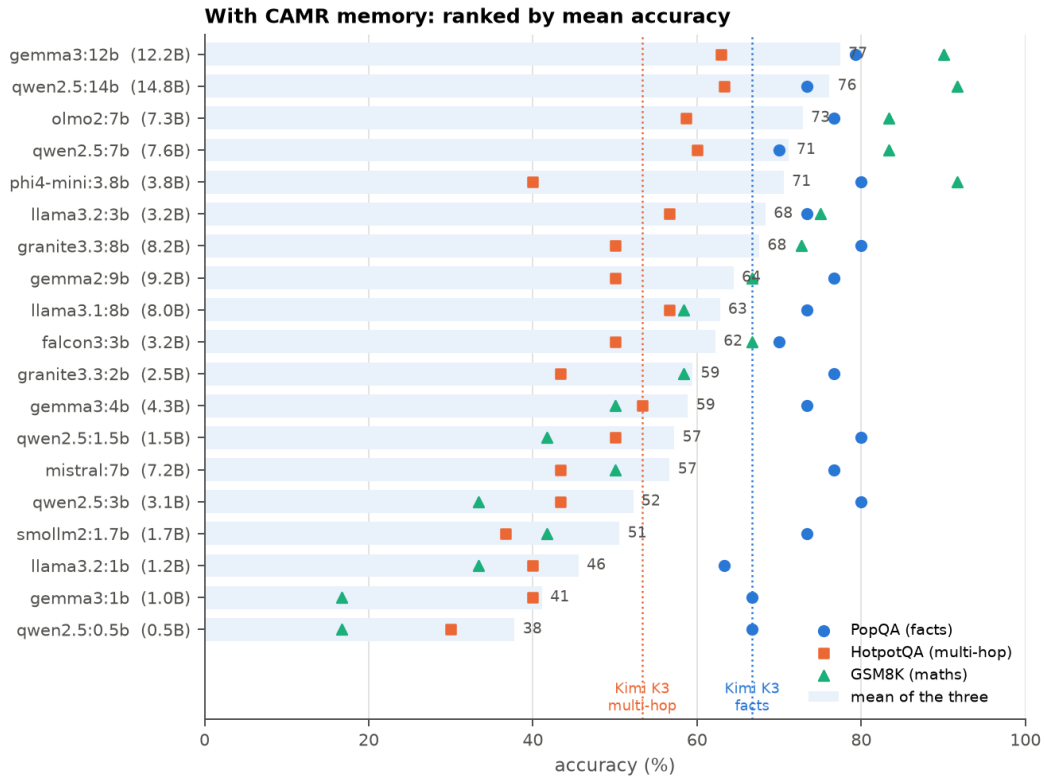


Figure 5.55: Ranking of the same 19 models with CAMR memory

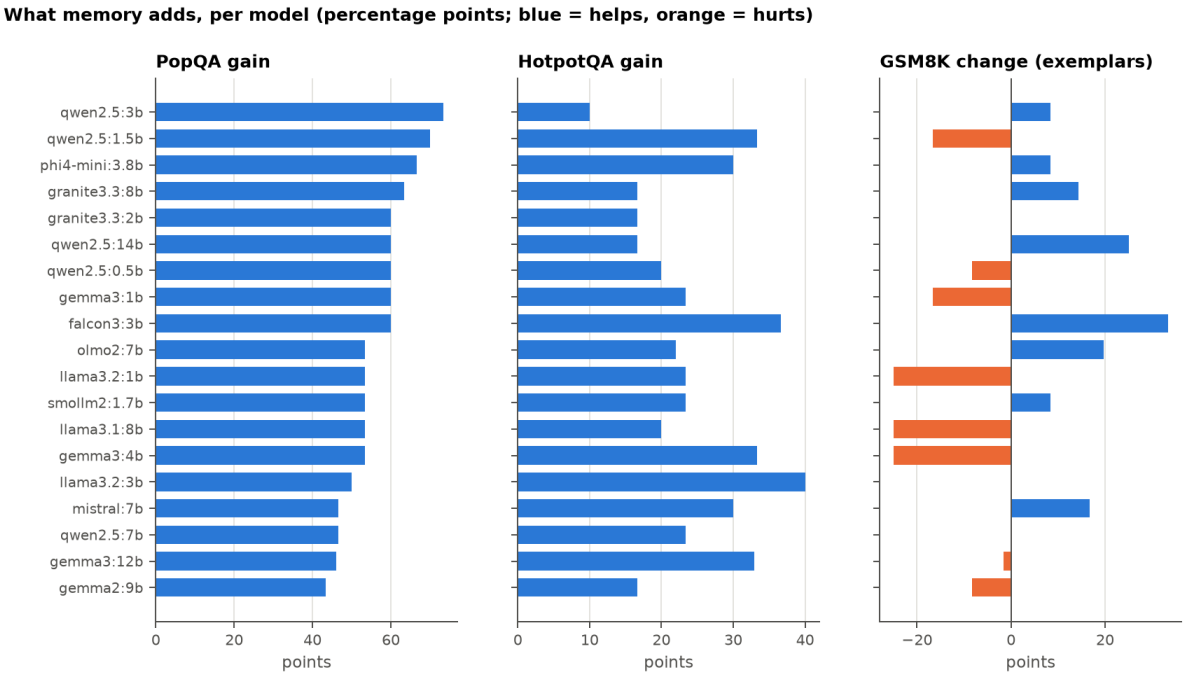


Figure 5.56: What memory adds to each model, per benchmark (percentage points)

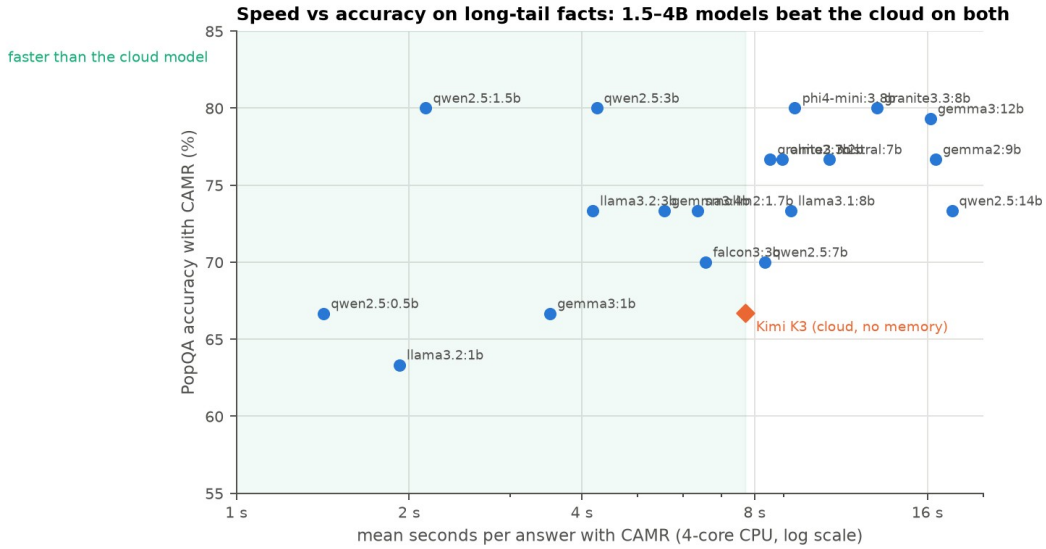


Figure 5.57: Speed against accuracy with CAMR on long-tail facts; the shaded region is faster than the cloud model

5.8.10 Which Model Sizes Suit the Engine

The same store, questions and ceilings were used with 19 small models from 0.5B to 14.8B parameters (Table 5.34, Figure 5.58). Accuracies are comparable across machines; decode speeds are comparable only within a machine.

Table 5.34: Model-size sweep: alone → with CAMR (results/pilot/tables/model_sweep.json)

Model (parameters, quantisation)	PopQA (%)	HotpotQA (%)	GSM8K (%)	Decode tok/s (PopQA)	s / answer with CAMR (PopQA)
qwen2.5:0.5b (494.03M, Q4_K_M)	6.7 → 66.7	10.0 → 30.0	25.0 → 16.7	50.3 → 48.9	1.4
gemma3:1b (999.89M, Q4_K_M)	6.7 → 66.7	16.7 → 40.0	33.3 → 16.7	24.9 → 23.9	3.5
llama3.2:1b (1.2B, Q8_0)	10.0 → 63.3	16.7 → 40.0	58.3 → 33.3	36.2 → 33.7	1.9
qwen2.5:1.5b (1.5B, Q4_K_M)	10.0 → 80.0	16.7 → 50.0	58.3 → 41.7	21.4 → 19.9	2.1
smollm2:1.7b (1.7B, Q8_0)	20.0 → 73.3	13.3 → 36.7	33.3 → 41.7	15.2 → 13.8	6.4
granite3.3:2b (2.5B, Q4_K_M)	16.7 → 76.7	26.7 → 43.3	58.3 → 58.3	14.2 → 12.2	8.5
qwen2.5:3b (3.1B, Q4_K_M)	6.7 → 80.0	33.3 → 43.3	25.0 → 33.3	13.3 → 11.0	4.2
falcon3:3b (3.2B, Q4_K_M)	10.0 → 70.0	13.3 → 50.0	33.3 → 66.7	14.2 → 14.5	6.6
llama3.2:3b (3.2B, Q4_K_M)	23.3 → 73.3	16.7 → 56.7	75.0 → 75.0	11.8 → 10.4	4.2
phi4-mini:3.8b (3.8B, Q4_K_M)	13.3 → 80.0	10.0 → 40.0	83.3 → 91.7	12.6 → 10.7	9.4
gemma3:4b (4.3B, Q4_K_M)	20.0 → 73.3	20.0 → 53.3	75.0 → 50.0	9.9 → 9.1	5.6
mistral:7b (7.2B, Q4_K_M)	30.0 → 76.7	13.3 → 43.3	33.3 → 50.0	5.4 → 5.1	10.8
olmo2:7b (7.3B, Q4_K_M)	23.3 → 76.7	36.7 → 58.6	63.6 → 83.3	5.8 → 5.3	8.9
qwen2.5:7b (7.6B, Q4_K_M)	23.3 → 70.0	36.7 → 60.0	83.3 → 83.3	6.1 → 6.0	8.3
llama3.1:8b (8.0B, Q4_K_M)	20.0 → 73.3	36.7 → 56.7	83.3 → 58.3	5.7 → 5.4	9.3
granite3.3:8b (8.2B, Q4_K_M)	16.7 → 80.0	33.3 → 50.0	58.3 → 72.7	5.0 → 4.6	13.1

Model (parameters, quantisation)	PopQA (%)	HotpotQA (%)	GSM8K (%)	Decode tok/s (PopQA)	s / answer with CAMR (PopQA)
gemma2:9b (9.2B, Q4_0)	33.3 → 76.7	33.3 → 50.0	75.0 → 66.7	6.7 → 6.5	16.5
gemma3:12b (12.2B, Q4_K_M)	33.3 → 79.3	30.0 → 63.0	91.7 → 90.0	3.9 → 3.7	16.2
qwen2.5:14b (14.8B, Q4_K_M)	13.3 → 73.3	46.7 → 63.3	66.7 → 91.7	3.2 → 3.0	17.7
Kimi K3 (cloud), no memory	*66.7*	*53.3*	*100*	–	*7.7*

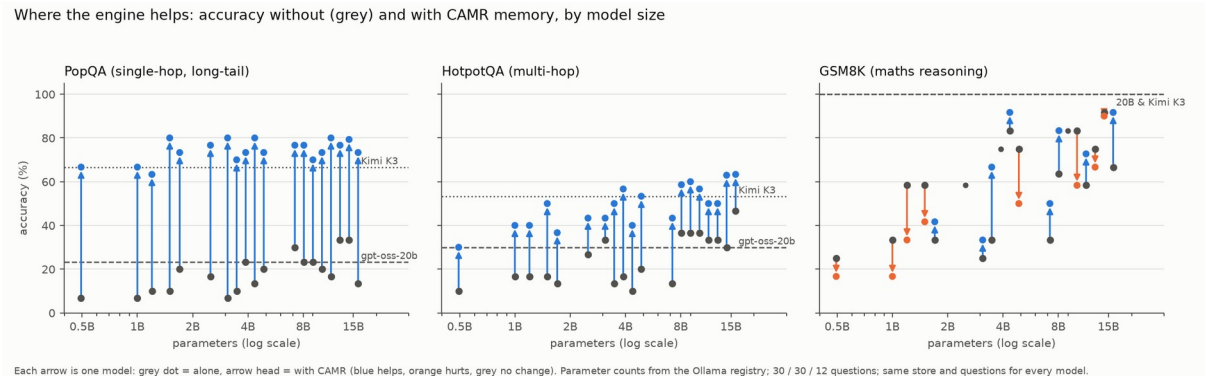


Figure 5.58: Accuracy without and with CAMR by model size, against the Kimi K3 ceiling

All 19 models gained on knowledge tasks: +43 to +73 points on PopQA and +10 to +40 on HotpotQA. With CAMR, 18 of 19 reached or beat Kimi K3 on long-tail facts; only llama3.2:1b (63.3%) fell short. Parametric knowledge barely grew with size: even the 12–15B models alone knew 13–33% of the long-tail facts, while every model from 1.5B upward reached 70–80% with memory. Multi-hop questions depended on the model as a reader. Seven of 19 reached Kimi K3's 53.3%: llama3.2:3b, gemma3:4b and five of the eight 7–15B models, with the best at 63% (qwen2.5:14b, gemma3:12b). Family mattered as much as size, since mistral:7b reached only 43.3%. Speed set the upper bound. On the 4-core CPU, every model of 7B or more with memory answered more slowly than Kimi K3 (8–24 s against 7.7 s on PopQA), so **1.5–4B is the range that suits the engine on this class of device**: faster than the cloud model, at or above it on facts, and at it on multi-hop from about 3B. Worked-example memory for maths helped 8 models, was neutral for 4 and hurt 7, depending on family rather than size, so reasoning routing must be learned per model. A few runs on machine D had one to three queries fail with a runtime error; their accuracy is over the answered questions.

5.8.11 Confirmation at Larger Scale: 200 HotpotQA Questions

To test whether the pilot's multi-hop results held beyond 30 questions, the study was repeated on 200 HotpotQA questions with the read path fixed in advance (similarity, gating and bridging, 512 tokens). Nothing was tuned on these questions. Three small models and Kimi

K3 were run with and without the same notes. 95% confidence intervals come from 2,000 bootstrap resamples of the questions.

Table 5.35: HotpotQA, n = 200: exact match, alone and with CAMR (results/hotpot200)

Model	Alone	With CAMR	Gain (95% CI)	s / answer with CAMR
qwen2.5:0.5b	3.5%	**26.5%**	+23.0 pts (+17.0 to +29.0)	2.4
qwen2.5:1.5b	10.5%	**41.0%**	+30.5 pts (+23.0 to +38.0)	2.8
llama3.2:3b	19.5%	**54.0%**	+34.5 pts (+27.0 to +42.0)	5.8
Kimi K3 (cloud)	51.8%	**66.3%**	+14.6 pts (+9.0 to +20.6)	9.5

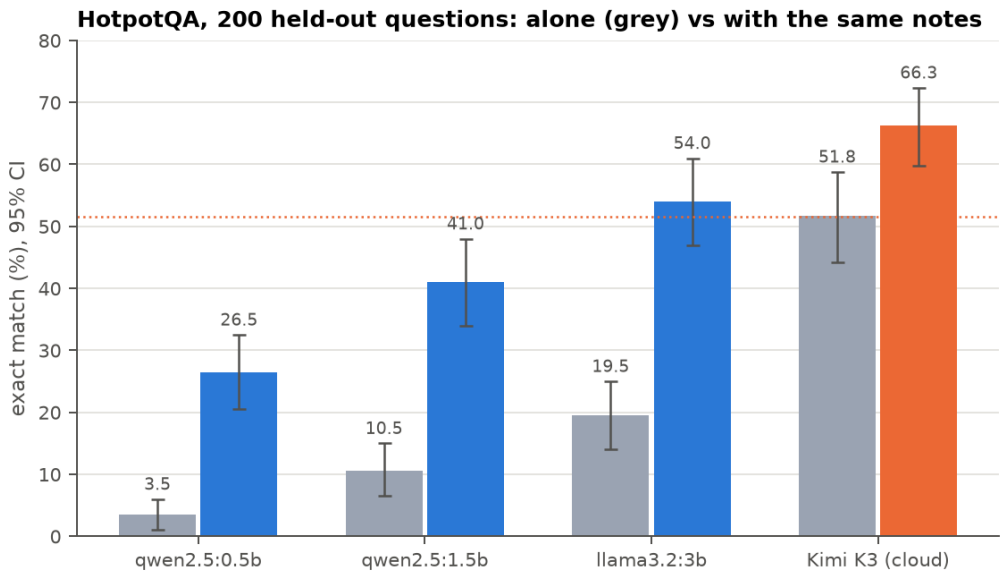


Figure 5.59: HotpotQA, 200 held-out questions: exact match alone and with the same notes, 95% bootstrap intervals

The larger sample confirmed the pilot. Every small model gained substantially, and every 95% interval excludes zero. With CAMR, llama3.2:3b reached 54.0% exact match against Kimi K3's closed-book 51.8% (n = 199), and answered in 5.8 s against 18.4 s. On multi-hop questions, a 3B model on a laptop CPU with memory therefore matches a frontier cloud model at about a third of the answer time. qwen2.5:1.5b reached 41.0% and qwen2.5:0.5b 26.5%. The ordering by size is the pilot's finding that multi-hop gains depend on how well the model reads the evidence. With the same notes, Kimi K3 rose from 51.8% to 66.3%. The residual gap between the best small model and the frontier model reading the same evidence is the share of multi-hop performance that memory cannot buy. Compared with the pilot (30 questions: 30.0%, 50.0% and 56.7% for the three models with memory), the estimates moved

by at most 9 points, within the pilot's uncertainty. Accuracy was identical when the 0.5B run was repeated for clean timings (26.5% both times).

5.8.12 Learning When to Use Memory and When to Escalate

A single fixed configuration cannot be best for every question. The learned policy chose an action per question from pre-answer features and was evaluated on the 72 held-out questions (Table 5.36, Figure 5.60).

Table 5.36: Engine policies on the held-out split (results/learn/tables/rl_policies.md, rl_cascade.md)

Policy (first study)	**Accuracy**	**Sent to cloud**	**Mean s / answer**
Always 0.5B alone	13.9%	0%	1.1
Always 0.5B + 128 tokens	41.7%	0%	1.2
Always 0.5B + 512 tokens	45.8%	0%	2.2
Always Kimi K3	63.9%	100%	12.6
Learned router (ridge, cloud cost 0.3)	61.1%	16.7%	2.2
Learned router (LinUCB, after 5 epochs)	56.9%	16.7%	2.2
Router + grounding cascade	**65.3%**	29.2%	3.1
Oracle (best action in hindsight)	79.2%	23.6%	2.6

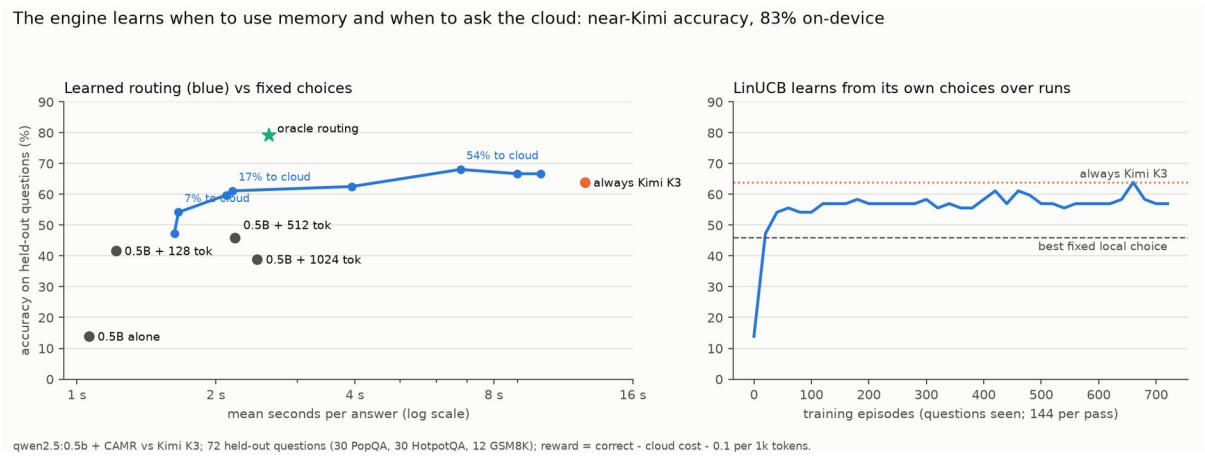


Figure 5.60: Learned routing compared with fixed choices, and the LinUCB learning curve over 5 epochs

The policy learned without supervision to send every reasoning question to the cloud (12 of 12, all correct) and to answer factual questions locally with a small budget. In this first study, with the grounding cascade, which answers locally first and escalates when the answer's words are not found in its notes, and its threshold set by inspecting the held-out results, the hybrid reached 65.3% against always-Kimi's 63.9%. 71% of questions never left the device, and mean latency was 4.0 times lower. LinUCB, which sees only the outcome of the action it chose, rose from 13.9% to 54.2% held-out accuracy within the first 40 episodes and then

fluctuated between 54% and 64% over the five epochs. The gap to the oracle (79.2%) is mostly on multi-hop questions, where pre-answer features cannot predict whether the small model will read the evidence correctly.

Tuning the router properly. The first version fixed λ = 1, α = 0.5 and 5 passes by hand, and its cascade threshold (0.2) was read off the held-out questions, which flatters the result. All four were re-chosen by 5-fold cross-validation on the 144 training questions only (Equation 5.19), and the 72 held-out questions were then scored once (Figure 5.61, Table 5.37).

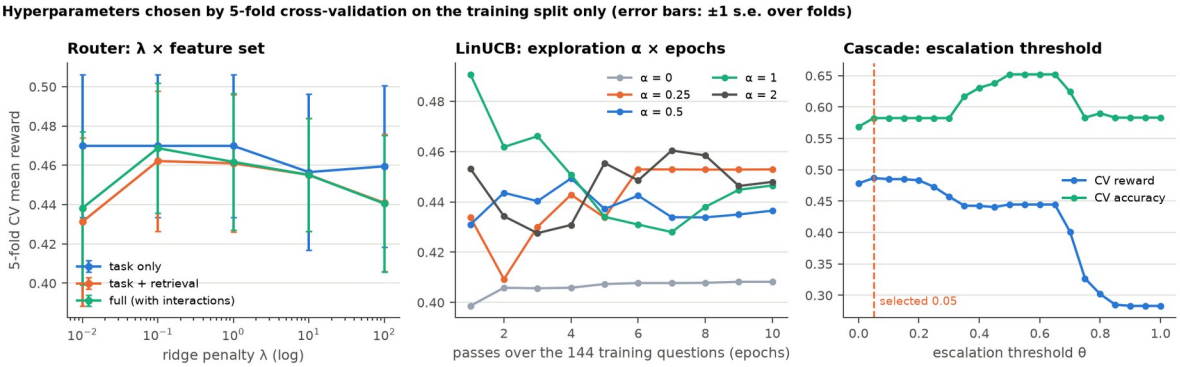


Figure 5.61: Cross-validated tuning of the learned router, LinUCB and the cascade threshold

Table 5.37: Hyperparameters before and after cross-validated tuning

Hyperparameter	First study (by hand)	Selected by 5-fold CV	CV mean reward
Feature set	full (11 features)	task only	0.470 ± 0.081
Ridge penalty λ	1.0	0.01	(same run)
LinUCB exploration α	0.5	1	0.491 ± 0.048
LinUCB passes (epochs)	5	1	(same run)
Cascade threshold θ	0.2 (chosen on held-out data)	0.05	0.487

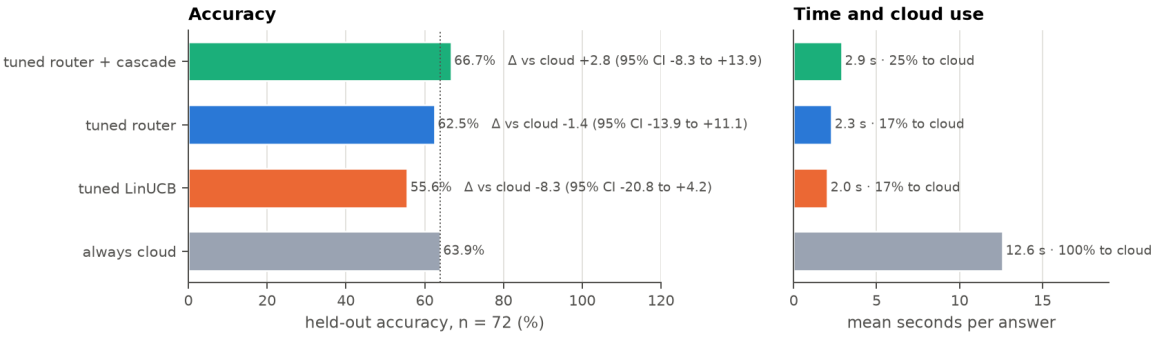


Figure 5.62: Tuned policies on the 72 held-out questions: accuracy with the paired 95% interval of the difference from always using the cloud, and time per answer

Two results stand out. First, cross-validation preferred the **simplest** router: task type alone, with the retrieval features adding nothing measurable on 144 questions (the fold-to-fold spread, about ±0.08, is larger than the differences between feature sets). With so little training data the richer model mainly fits noise; the retrieval features may pay off with more data. Second, the honestly tuned cascade is at least as good as the hand-set one: 66.7% held-out accuracy with 25% of questions sent to the cloud and 2.9 s per answer, against 63.9% and 12.6 s for always asking Kimi K3. The difference in accuracy, +2.8 points (95% CI -8.3 to +13.9), is not significant: the defensible claim is *equal accuracy at a quarter of the cloud calls and about a fifth of the time*. LinUCB, which learns only from the actions it takes, stayed below the offline router (55.6%); its best setting was one pass, and more passes did not help, a sign that 144 questions are too few for exploration to pay off.

5.8.13 Fine-Tuning Compared with Memory

The problem statement (section 1.2) argued that fine-tuning is the wrong tool for giving a small on-device model new knowledge: it needs compute users do not have, must be repeated whenever knowledge changes, and can damage what the model already does well. CAMR itself never changes the model's weights. To test that argument rather than assume it, the obvious alternative was run as a **baseline**: the same 0.5B model was fine-tuned on exactly the documents CAMR stores, and the two were compared on the same held-out questions.

Method. Low-Rank Adaptation (LoRA; Hu et al., 2022) freezes each weight matrix W and learns a low-rank update, so only a small fraction of parameters is trained:

$$W' = W + \frac{\alpha_{\text{LoRA}}}{r} B A, B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll \min(d, k) \tag{5.32}$$

The adapters were trained with the causal language-modelling loss over the document tokens $x_1 \ldots x_T$:

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \log p_\theta(x_t \mid x_1, \ldots, x_{t-1}) \tag{5.33}$$

Table 5.38: Fine-tuning set-up (scripts/finetune_baseline.py)

Setting	Value
Model	Qwen2.5-0.5B-Instruct (Hugging Face weights, float32 on CPU; the same base as qwen2.5:0.5b)
Training data	The 237 PopQA documents CAMR stores for the pilot and learning splits (20,622 trained tokens per epoch); no questions or answers
Formats compared	*raw*: title and text as plain text; *chat*: the document as the assistant's answer to "What do you know about <title>?" (loss on the answer only)

Setting	Value
LoRA	rank r = 16, α = 32, dropout 0.05, on all attention and MLP projections; 8,798,208 trainable parameters (about 1.8% of the model)
Optimisation	AdamW with warm-up then a constant learning rate; batch 8 (micro-batch 2 × 4 accumulation steps, because batch 8 exhausted the 14 GB memory limit), gradient clipping 1.0, gradient checkpointing, seed 13
Grid	*chat* format at learning rates 1e-4 and 3e-4; *raw* format at 3e-4 only, because the learning rate made no difference in the chat format and each configuration costs over an hour on the CPU
Epochs	Up to 8; checkpoints evaluated after epochs 1, 2, 4 and 8
Validation (selection)	60 PopQA questions of the learning split's *training* set, closed book
Test (reported once)	60 held-out PopQA questions (pilot 30 + learning evaluation 30), closed book and with CAMR's logged prompts; GSM8K (48 questions) for side effects
Decoding	Greedy, the same chat template for every condition

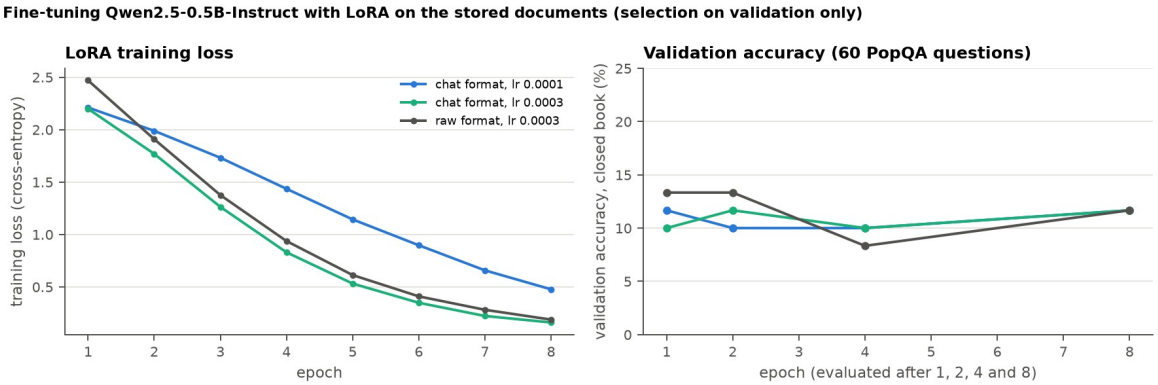


Figure 5.63: LoRA training loss and validation accuracy per epoch for each configuration

Table 5.39: Memory against fine-tuning on the same documents (held-out test, one runtime for all conditions)

Condition	PopQA (long-tail facts)	GSM8K (side effect)	Cost to add the knowledge	Artefact
Base model alone	10.0%	45.8%	–	–
Base + CAMR (no training)	71.7%	45.8%	6 s to store	2.6 MB store
LoRA fine-tuned (raw, lr 0.0003, 1 epochs)	13.3%	45.8%	7 min to train	35 MB adapter
LoRA fine-tuned + CAMR	75.0%	45.8%	both	both

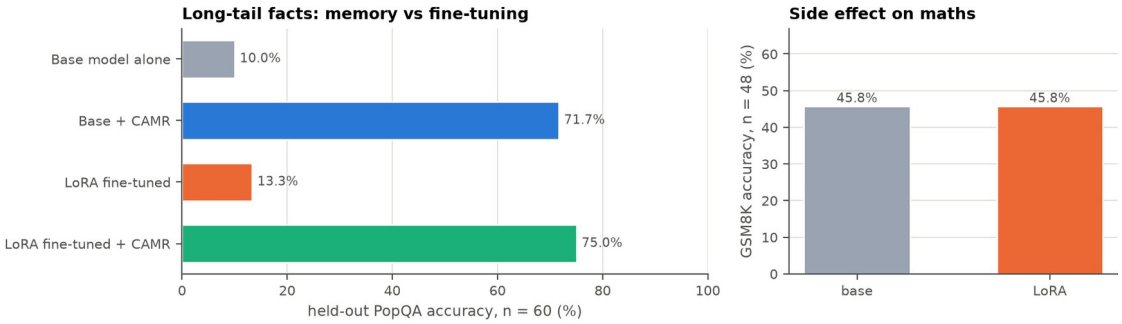


Figure 5.64: Held-out accuracy of the base model, CAMR, LoRA and both; and GSM8K accuracy before and after fine-tuning

Memory beat fine-tuning by 58 points on the same documents. Fine-tuning moved closed-book accuracy from 10.0% to 13.3% (+2 questions of 60); giving the untrained model CAMR's notes moved it to 71.7%. Across the whole grid, validation accuracy never rose above 13.3% while the training loss fell from about 2.2 to as low as 0.16 (Figure 5.63): the adapters learned to *reproduce* the documents, not to answer questions about them. This matches Ovadia et al. (2023), who found unsupervised fine-tuning a poor way to inject new knowledge compared with retrieval, which puts the exact sentence in front of the model. The costs differ by orders of magnitude. CAMR stored the 237 documents as 305 notes in 6 s (2.6 MB), and a new note is usable the moment it is saved. The selected adapter needed 7.5 min of training on four CPU cores (35 MB), but finding it took a 3.5-hour search over formats, learning rates and epochs, and any change to the user's knowledge means training again. GSM8K accuracy was unchanged (45.8% before and after): one epoch of low-rank training on short documents caused no measurable forgetting on this sample. Adding the adapter to CAMR changed accuracy from 71.7% to 75.0% (+2 questions), which is within noise at this sample size: fine-tuning adds little once the notes are there.

5.8.14 Learning Over Time: a Frozen 0.5B Model That Looks Things Up

Design. A stream of 200 PopQA questions that no earlier experiment had used (30 subjects are asked about twice, 40–90 questions apart) was put, in order, to qwen2.5:0.5b with memory that starts **empty** and eager learning on (Equation 5.25; the same BGE embedder, gate, entity gate and 512-token budget as the benchmarks). Before the stream ran, the one free choice in the lookup, how to order the meanings of an ambiguous name, was made on 60 other fresh questions: ordering by the embedder's similarity put the answer in the fetched text for 55.0% of them against 35.0% for shared words. Then, offline, every fact was asked again **in different words** (a second template per relation), so what the model gets right in this

second pass it can only know from what it learned. Closed-book baselines answered the same questions, and the resident memory of each model's process was measured (`scripts/eager_learning.py`).

Table 5.40: The learning stream: accuracy on 200 new questions and the model process's resident memory

System	As asked	Reworded	RAM (GB)
qwen2.5:0.5b alone	10.0%	10.5%	0.7
qwen2.5:1.5b alone	11.0%	–	2.0
llama3.2:3b alone	19.0%	18.0%	4.3
gpt-oss:20b alone	20.5%	–	13.2
qwen2.5:0.5b + eager learning (memory starts empty)	**43.5%** (online)	41.5% (offline)	0.7
llama3.2:3b reading the memory the 0.5B model learned	–	45.0% (offline)	4.3

The frozen 0.5B model with eager learning answered 43.5% of the stream, against 10.0% for the same model closed-book, 19.0% for a model six times its size, and 20.5% for gpt-oss:20b, a model forty times its size whose process held 13.2 GB of RAM against 0.7 GB. It looked 158 questions up, found nothing for 9, and answered 35 from what it had already learned without any request; of the 30 subjects asked about twice, 25 second questions needed no lookup. Offline, in other words, it still answered 79 of the 87 facts it had got right online (90.8%): what it learned stayed learned. The same memory, read by llama3.2:3b, gave 45.0% on the reworded questions against 18.0% for that model alone: memory learned once serves any model. Accuracy did not climb along the stream (45.0% in the first half, 42.0% in the second), which is expected: almost every question is about a new subject, so the gain is from learning each fact when it is first needed, and the evidence of learning is the second pass. **Where answers are still lost:** the fetched text contained an accepted answer for 52.0% of questions, and when it did the 0.5B model was right 76.0% of the time. The ceiling is therefore set mostly by the lookup (PopQA's long-tail subjects are often ambiguous, e.g. *Kill*, *Ram*, *The Other*, and its answer keys are strict), not by the reader. Memory grew to 333 notes, 2.7 MB on disk. Times per question in this run (median 29 s) are dominated by waiting out rate limits on a shared cloud machine, not by the engine, and are not reported as the application's speed.

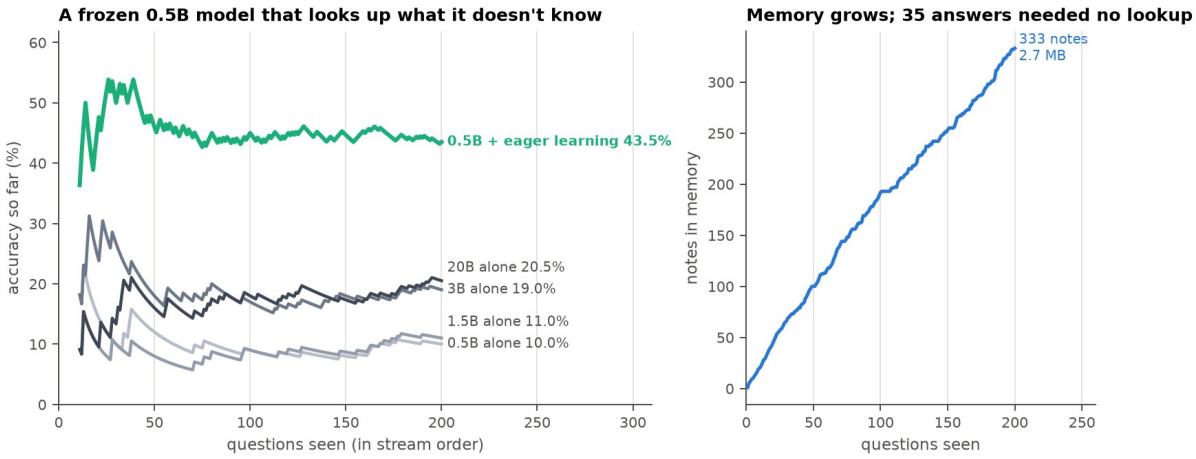


Figure 5.65: The learning stream: running accuracy of the frozen 0.5B model with eager learning against closed-book models (left), and the notes it stored (right)

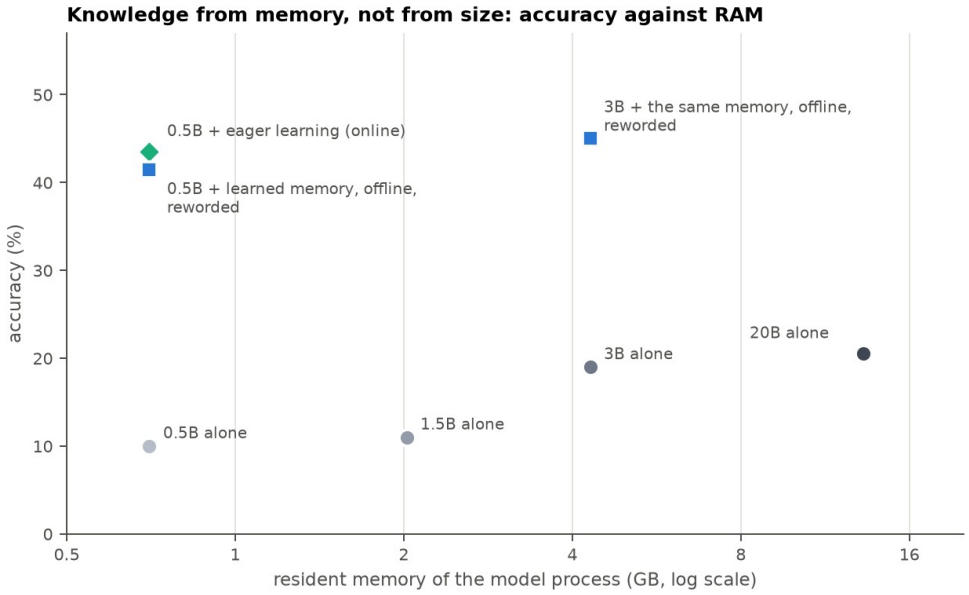


Figure 5.66: Accuracy against the resident memory of the model process: knowledge comes from memory on disk, not from model size

After defect D-21 was fixed (Wikipedia had refused most of the fetcher's requests because it did not send its User-Agent), the whole stream was run again from an empty memory with the same DuckDuckGo answers replayed from the cache: pages per question rose from 1.07 to 1.91 and accuracy went from 43.5% to 37.5%. The first run is the one reported in Table 5.40, since the stream's design was fixed before it.

Table 5.41: How much a lookup should give the 0.5B reader, chosen on 60 development questions

Pages kept per lookup	128-token budget	512-token budget
1	20.0%	20.0%
2	18.3%	15.0%

Pages kept per lookup	128-token budget	512-token budget
3	36.7%	33.3%

The remedy was chosen on the 60 development questions, never the stream: keeping three pages per lookup mattered most (36.7% and 33.3% against at most 20.0% with one or two, because the gate needs candidates to choose between), and a 128-token budget was slightly better than 512 (36.7% against 33.3%, two questions of 60), so 3 pages at 128 tokens was adopted. Run once on the stream with the corrected fetcher, it scored 43.0%, against 37.5% with the earlier settings, and 42.5% offline on the reworded questions. The first run, made before any of this was known, remains the headline result in Table 5.40; the tuned run shows what the corrected engine does when its reading is sized for a 0.5B model.

Eager, not evasive, in the application. The benchmark prompt above asks for a short answer, so the model never declined there. In the application the question is what happens when memory cannot help with a question about the user. The chat prompt was therefore extended so that such a gap is answered by asking to be taught; Table 5.42 scores it on the same development, held-out and fresh cases as Section 5.5.5 (prompt D is the one the application used before).

Table 5.42: Asking to be taught instead of only saying *I don't know*: correct cases and unknown personal facts answered with an invitation to teach (development 6, held-out 7, fresh 8 cases; 5 unknown)

Prompt	Model	Dev	Held-out	Fresh	Asks to be taught
D (before)	llama3.2:3b	6/6	7/7	8/8	0/5
D (before)	qwen2.5:1.5b	6/6	7/7	8/8	0/5
E	llama3.2:3b	5/6	7/7	8/8	5/5
E	qwen2.5:1.5b	6/6	7/7	8/8	3/5
F (adopted)	llama3.2:3b	6/6	7/7	8/8	5/5
F (adopted)	qwen2.5:1.5b	6/6	7/7	8/8	4/5

5.8.15 External Weights Beside the Model: Token Memory

The kNN-LM of Equation 5.23 was built for the frozen Qwen2.5-0.5B-Instruct from the same 237 documents the LoRA study trained on: 22,554 (hidden state, next token) pairs, 40.6 MB, built in 210 s on the CPU. The document format (plain text, or the chat format the LoRA study used), the mixing weight $\lambda \in \{0.25, 0.5, 0.75\}$ and the temperature $\tau \in \{0.02, 0.05\}$ were chosen on the 60 validation questions; no setting did better than 13.3% there, and larger λ (leaning more on the memory) did worse. The chosen setting ($\lambda = 0.25$, $\tau = 0.02$, raw text)

was tested once on the 60 held-out questions (Table 5.43, Figure 5.67), in one runtime (transformers, float32, greedy).

Table 5.43: Three places for a frozen model's knowledge, on the same 237 documents and 60 held-out questions

Where the knowledge lives	External weights	Held-out accuracy
Frozen 0.5B alone	–	10.0%
Inside: LoRA (Section 5.8.13)	8.8 M trained weights	13.3%
Beside: kNN-LM	22,554 entries, 41 MB	13.3%
In the prompt: CAMR	notes + vectors	75.0%
kNN-LM and CAMR together	both	75.0%

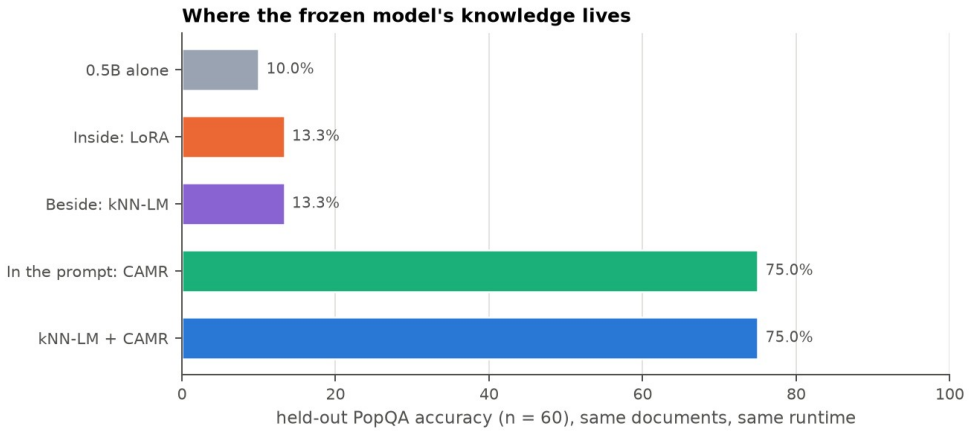


Figure 5.67: Held-out accuracy by where the frozen model's knowledge lives

Weights beside the model help no more than weights inside it. The token memory reached 13.3% against 10.0% for the model alone, the same as LoRA (13.3%), while CAMR reached 75.0%; adding the token memory to CAMR changed nothing (75.0%). The reason is visible in Equation 5.23: the memory is keyed by the model's state *while reading a document*, and a question puts the model in a different state, so the nearest keys rarely continue with the answer. This matches the finding that kNN-LMs help memory-intensive language modelling but not knowledge-intensive question answering (Geng et al., 2024). The literal reading of *external weights used as memory, with the model's parameters as compute* therefore works best when the external weights are notes the model reads (CAMR), not numbers it mixes into its output. (CAMR's 75.0% here differs from the 71.7% of Section 5.8.13 for the same prompts because this script decodes one question at a time instead of in padded batches.)

5.8.16 What Sleep Buys: a Learned Index and Learning to Fetch

The learned index on real memory. On the 38,054-note store of the retrieval evaluation, with its 1,000 real questions, exact search found every gold paragraph for 63.7% of them. The index (195 clusters, built in 2.0 s) was compared at four probe sizes (Table 5.44).

Table 5.44: Exact search against the learned index on 38,054 real notes and 1,000 real questions (top 20)

Search	Share of memory read	ms / question	Agreement with exact	Full-support recall
Exact	100%	0.7	100%	63.7%
Index, n_probe = 4	2.1%	0.2	63.5%	36.7%
Index, n_probe = 8	4.1%	0.3	75.2%	43.6%
Index, n_probe = 16	8.2%	0.7	85.9%	50.3%
Index, n_probe = 32	16.4%	1.2	93.4%	55.8%

At the size of this store the index does not pay. Exact search over 38,054 vectors held in RAM took 0.71 ms. Probing 8 clusters was faster (0.34 ms) but found every gold paragraph for only 43.6% of questions against 63.7%; probing 32 recovered most of that (55.8%) and was slower than exact search (1.23 ms). An index trades evidence for speed, and at tens of thousands of notes there is no speed to buy.

It pays as memory grows. The store was grown to 400,000 notes with perturbed copies of its own vectors (distractors only; every gold paragraph is a real one) and each size was searched in a fresh process with the default probe (8% of the clusters). At 150,000 notes exact search took 15.2 ms and the index 6.8 ms; at 400,000 notes, 40.1 against 25.5 ms, with the process at 1,419 against 981 MB, because exact search holds the whole matrix in RAM. The price is evidence: full-support recall 60.8% against 63.7%. One caveat on memory: over 1,000 questions the index's process still grew by 586 MB, because the operating system keeps the clusters it has read in its page cache; those pages can be dropped when memory is needed, unlike the exact search's matrix, but they count while they stay. Equation 5.22 therefore holds per question, not per session. **Decision:** sleep builds the index only once memory passes 100,000 notes; below that, exact search is faster and finds more, so a personal memory is never indexed.



Figure 5.68: Search time and resident memory as memory grows: exact search against the learned index (distractor notes added to the real store)

Learning to fetch on unseen questions: a negative result. The query adapter (Equation 5.27) was trained on half of the HotpotQA questions and half of the 2WikiMultiHopQA questions (question → its gold paragraphs), the epoch chosen on a validation part of that half, and tested on the other half. The untrained engine already ranked a gold paragraph first for 88.0% and 97.2% of the test questions, and within the top 5 for 97.6% and 100.0%. On validation no epoch beat the untrained adapter, so the selected adapter is the identity; trained for 40 epochs regardless, it lowered recall@1 to 83.2% and 80.8% (rank 16). With a strong embedder there is little for a small adapter to learn from a few hundred questions, and much to overfit; this is why sleep keeps an adapter only when held-out recall rises (Equation 5.29).

Learning to fetch from its own experience. After the learning stream, the engine slept on what it had experienced: pairs (question as first asked → the notes that held its answer) for 86 questions, with 22 kept aside. Inside sleep, the adapter's own held-out check moved recall@1 from 83.3% to 77.8% and recall@5 from 94.4% to 94.4%, so the adapter was **rejected** and retrieval stayed exactly as it was: on the reworded questions recall@1 remained 83.7% for facts it had learned and 81.8% for the ones kept aside. Both studies point the same way: with a strong embedder and a few hundred questions, a learned query map has little to add, and the acceptance test is what keeps sleep from making retrieval worse.

5.8.17 Cost on the Device (Research Question v)

On machine A, median end-to-end latency was 188 ms for the 0.5B model alone and 375 ms with CAMR. The engine added a median of 187 ms and about 536 prompt tokens. Peak memory was 712 MB for the engine and 1987 MB for the model runtime, and the store file

was 47.8 MB. NFR-02 was **not met**: retrieval was 56% of answer time against a 10% target. In isolation the engine's retrieval took 29 ms (embedding 21.5 ms, search 6.2 ms, bridging 1.2 ms, packing 0.4 ms). The rest was contention between the embedder and the model runtime for the same four cores. Partitioning the threads cut the p90 from 157 ms to 50 ms. Because a 0.5B model generates in about 180 ms, the 10% target is reachable only for models whose generation takes 450 ms or more, roughly 1.5B parameters and above on this CPU.

The large local alternative behaved worse on the same device. gpt-oss:20b decoded at about 6.3 tokens/s and needed 19.5–38 s per answer. When a 0.7 GB process ran beside it, it collapsed to 0.4–0.9 tokens/s as its memory-mapped weights were evicted. On a 16 GB device, a small model with external memory is the practical design, not merely the cheaper one.

5.8.18 Responsiveness of the Application

The benchmark profile above measured the engine. A user of CAMR Personal waits for something else: the first word of the answer. Three changes were made to shorten that wait, and each was measured with real models through Ollama on the same 4-core CPU class (`scripts/app_latency.py`, results in `docs/results/app_latency.json`). (1) **Warm-up.** Ollama loads a model on first use, which dominated the first answer. The app now loads the chosen model in the background when it starts and when the user switches model, while the user is still typing. (2) **Time to first word** is recorded for every answer and shown under it, because streaming makes this, not the total time, the wait a user notices. (3) **Query-embedding cache.** A repeated question reuses its embedding instead of calling the embedding model again. The memory held 282 notes: the five demonstration notes and the PopQA documents, with six questions per setting.

Table 5.45: Time a CAMR Personal user waits, per model (budget 384 tokens; medians over six questions)

Model	First word after reboot	First word, cold (cached file)	First word after warm-up	Warm-up (in background)	First word, typical	Whole answer, typical	Memory lookup
qwen2.5:0.5b	2.4 s	1.9 s	0.6 s	1.0 s	0.44 s	0.8 s	218 ms
qwen2.5:1.5b	16.8 s	2.4 s	0.9 s	1.3 s	0.55 s	1.7 s	243 ms
llama3.2:3b	172.7 s	5.0 s	1.7 s	2.4 s	0.85 s	2.6 s	263 ms

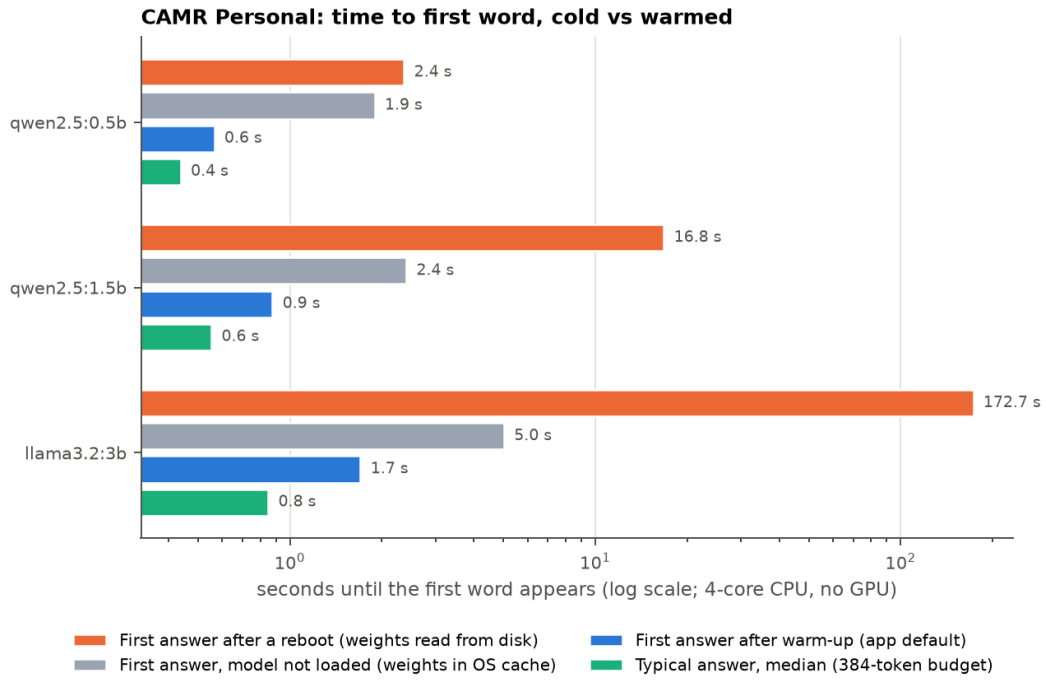


Figure 5.69: Time to the first word: cold start, after the app's warm-up, and a typical answer

Table 5.46: Effect of the memory budget on waiting time (medians)

Model	Budget (tokens)	Prompt tokens	First word	Whole answer
qwen2.5:0.5b	128	168	0.41 s	0.8 s
qwen2.5:0.5b	384	168	0.44 s	0.8 s
qwen2.5:0.5b	768	168	0.42 s	0.8 s
qwen2.5:1.5b	128	168	0.57 s	1.8 s
qwen2.5:1.5b	384	168	0.55 s	1.7 s
qwen2.5:1.5b	768	168	0.51 s	1.6 s
llama3.2:3b	128	160	0.78 s	2.4 s
llama3.2:3b	384	160	0.85 s	2.6 s
llama3.2:3b	768	160	0.82 s	2.4 s

Warm-up. With the model unloaded but its file in the operating system's cache, the first word took 1.9 → 0.6 s, 2.4 → 0.9 s, 5.0 → 1.7 s for qwen2.5:0.5b, qwen2.5:1.5b, llama3.2:3b once the app had warmed the model, which it does while the user types. After a reboot, when the weights must be read from disk, the cold first answer took 173 s for llama3.2:3b on this container's disk; warm-up moves that load out of the conversation, but cannot hide it if the user asks before it finishes. The first warmed answer is still slower than a typical one because nothing of the prompt has been processed yet. **Budget.** The budget made no measurable difference here: on personal notes, gating admitted only the one or two notes that matched, so every budget sent the same prompt (about 168 tokens) and the budget never bound. It matters

when many notes are relevant, as in the benchmarks (section 5.8.8). **Lookup.** Inside a conversation the memory lookup took a median of 218–263 ms, against 48 ms with nothing else running: the embedding model and the chat model share the same four cores, the contention already diagnosed in section 5.8.17. A repeated question skipped the embedding and took 2.8 ms instead of 189 ms. Growing memory 21-fold, from 282 to 5,976 notes, raised the lookup only from 48 to 57 ms (vector search 2 → 11 ms), so memory can keep growing well beyond the model's context window without the assistant becoming noticeably slower. In every case the user sees the first word in under 2 s once the model is loaded.

Table 5.47: Memory lookup time as memory grows (nomic-embed-text through Ollama)

Memory	**Notes**	**Lookup (median)**	**of which embedding**	**of which vector search**
Demonstration + PopQA	282	48 ms	45 ms	2.1 ms
+ HotpotQA paragraphs	5,976	57 ms	42 ms	11.3 ms
Repeated question (cache)	—	2.8 ms	0 (cached)	—

5.8.19 Alignment with the Proposal (Chapter 1)

Each objective and research question of Chapter 1 was checked against what was built and measured. All five research questions are answered (section 6.2). Where the work departed from the proposal, the departure, its reason and its effect on the findings are recorded in Table 5.48, so that no deviation is hidden in the results.

Table 5.48: Deviations from the approved proposal and their effect

Proposal (Chapter 1)	**What was done**	**Reason**	**Effect on the findings**
Obj. ii: a write policy that *distils* interactions into compact notes	Distilling (structured) policy implemented and unit-tested; experiments used verbatim notes	Benchmark documents are already compact; distilling would add a model call per note and a confound to the retrieval study	Results describe verbatim memory; distillation is untested at scale
Obj. ii: rank notes by similarity, recency *and* importance (Eq. 4.1)	Eq. 4.1 implemented and evaluated; the default became similarity + gating + bridging	Eq. 4.1 lowered full-support recall by 1.2–5.4 points (5.8.1)	A measured improvement on the proposal; recency is untested on sessioned data
Obj. iii and scope: one physical consumer laptop as the device	CPU-only cloud containers of laptop class (4 vCPU, 16 GB, no GPU), fully specified	Several identical machines were needed to run the 19-model sweep in time	Accuracy transfers (greedy, deterministic engine); latency is indicative, as limitation 1 expected
Scope: small models of 1–3B parameters	All 1–3B models tested, plus 0.5B and 7–14.8B	To find where the useful size range starts and ends	An extension: confirms 1.5–4B as the range that suits the engine (5.8.10)
Scope: a read-only inspector, not an end-user application	Inspector delivered as scoped; CAMR Personal added after the experiments	To put the findings in users' hands and collect feedback	An extension; it adds no claim to the research results
Delimitation: no training or fine-tuning of any language model	CAMR trains none. One LoRA adapter was trained as a *comparison baseline* only (section 5.8.13)	To test the problem statement's argument against fine-tuning with evidence	Strengthens RQ ii; the engine's own claims are unchanged (frozen weights)

Proposal (Chapter 1)	What was done	Reason	Effect on the findings
Delimitation: no port to mobile phones	No mobile results are reported; an unrun Android prototype is described as future work (6.4)	Kept within the delimitation	None
NFR-02: retrieval ≤ 10% of answer time	Measured and reported as not met for 0.5B; diagnosed	A 0.5B model answers in about 180 ms	Met only for models of about 1.5B and above (5.8.17)

5.8.20 Limitations

i. **Sample size.** Pilot samples were 30/30/12 questions per benchmark. One question is 3.3 points, so differences of one or two questions are within noise. Confidence intervals are reported for gap closed. The multi-hop results were confirmed on 200 questions, and the retrieval study used 500 questions per benchmark; the single-hop and reasoning results rest on the pilot sample.

ii. **Stochastic cloud ceiling.** Kimi accepts only temperature 1, so its scores are single samples. Small-model floors also varied between repeats (20/30 identical answers), so floors are best read as a range.

iii. **Oracle-like PopQA corpus.** PopQA documents are the subject's own Wikipedia summary, which favours retrieval. HotpotQA, with eight distractors per question and extra distractor paragraphs, is the more realistic test.

iv. **Benchmark questions are independent.** Recency could not show a benefit. A sessioned benchmark (e.g. LongMemEval) would be needed to evaluate it fairly.

v. **Write policy.** Only verbatim notes were evaluated end to end. The structured policy is implemented and tested but was not run at scale.

vi. **Hardware.** All measurements are from CPU cloud containers of laptop class, not from a physical laptop or phone. Decode speeds differ between machines.

vii. **No user study yet.** The application was tested for correct and safe behaviour, not for usefulness or usability with real users.

viii. **Policy learning.** The router was trained on 144 questions with costs set by the researcher. Even after cross-validated tuning, its accuracy is statistically equal to always using the cloud model; the robust result is equal accuracy with most questions kept on the device and much lower latency. Fold-to-fold variation was larger than the differences between hyperparameter settings.

ix. **Fine-tuning baseline.** One base model (0.5B), one unsupervised training format family and a CPU float32 runtime were tested; supervised question–answer data or a larger

model could fine-tune better. The comparison shows what a user's own device can do with the same documents, not the limit of fine-tuning.

x. **Live scenarios.** The application scenarios are five hand-designed conversations, run once each with one model; they illustrate behaviour and failure modes and are not a statistical evaluation.

Conclusions, recommendations and future work, including feedback from real users of CAMR Personal and a phone version of the assistant, are presented in Chapter 6.